

ets_marl - Default RQ Analysis - MAIN UPDATED

May 12, 2026

1 ETS MARL — Default-Config Research-Question Analysis

This notebook focuses on what the **default configuration** alone can tell us about the thesis research questions. It deliberately does **not** attempt cross-config (variant) comparisons — those will be addressed in a separate experiments notebook.

Research questions (full text in docs/research_questions.md and the top-level README.md):

- **Main RQ** — How can MARL be used to simulate the EU ETS, and what does the simulation reveal about its regulatory mechanisms and the strategic behaviour of power-sector participants?
- **Sub-RQ 1** — What environment / algorithm design choices are required to build a *stable, behaviourally credible* simulation?
- **Sub-RQ 2** — How sensitive are market outcomes (price stability, compliance, decarbonisation pace) to the regulatory mechanisms of the EU ETS? (*Default-only descriptive baseline; cross-variant claims deferred.*)
- **Sub-RQ 3** — How do financial vs environmental objectives shape company strategies, and how do those aggregate into market-level outcomes?

Structure

1. Setup — locate project root.
2. Imports.
3. Multi-sweep loader (allows pulling in **several** default-config sweeps at once — extends the single-spec pattern from ets_marl - Sweep Analysis.ipynb).
4. Discovery & loading.
5. Sanity overview (counts, episode coverage, default-only filter).
6. **§RQ1** — convergence, cross-seed reproducibility, behavioural credibility, pathology.
7. **§RQ2** — within-default regulatory mechanism behaviour (cap → price, MSR / TNAC, penalty incidence, decarbonisation pace).
8. **§RQ3** — strategic behaviour by archetype and reward weighting, aggregated to system-level outcomes.

The notebook is intentionally compact: each section ships the headline plot or table that supports the corresponding RQ claim, and stops there.

```
[42]: # Install notebook dependencies if needed.  
# Set INSTALL_NOTEBOOK_DEPS = True (or run the cell with required packages  
      ↪missing)  
# to pip-install everything in notebooks/requirements-notebooks.txt using the
```

```

# CURRENT kernel's interpreter. This is more reliable than `!pip install -r
↳ {path}`
# (IPython magics don't expand `{var}` at runtime) and works on Azure / Colab /
↳ local.
import subprocess, sys, importlib, re
from pathlib import Path

INSTALL_NOTEBOOK_DEPS = False # flip to True to force-reinstall

# Map PyPI distribution name -> import module name when they differ.
_DIST_TO_IMPORT = {
    "pyyaml": "yaml",
    "scikit-learn": "sklearn",
    "imbalanced-learn": "imblearn",
    "opencv-python": "cv2",
    "pillow": "PIL",
    "beautifulsoup4": "bs4",
    "ipython": "IPython",
}

def _find_requirements():
    req_name = "requirements-notebooks.txt"
    here = Path.cwd()
    candidates = [here / req_name, here / "notebooks" / req_name]
    candidates.extend(p / "notebooks" / req_name for p in here.parents)
    return next((p for p in candidates if p.exists()), None)

def _parse_requirements(req_path):
    """Extract distribution names from a requirements file (skip comments,
    ↳ options, env markers)."""
    dists = []
    for raw in req_path.read_text(encoding="utf-8").splitlines():
        line = raw.strip()
        if not line or line.startswith("#") or line.startswith("-"):
            continue
        # Strip env markers and extras: "pkg[extra]>=1.0; python_version>='3.9'"
        line = line.split(";", 1)[0].strip()
        m = re.match(r"^([A-Za-z0-9_\.\\-]+)", line)
        if m:
            dists.append(m.group(1).lower())
    return dists

def _missing_required(dists):
    missing = []
    for dist in dists:
        mod = _DIST_TO_IMPORT.get(dist, dist.replace("-", "_"))
        try:

```

```

        importlib.import_module(mod)
    except ImportError:
        missing.append(dist)
    return missing

req_path = _find_requirements()
if req_path is None:
    raise FileNotFoundError("Could not find notebooks/requirements-notebooks.
↪txt")

dists = _parse_requirements(req_path)
missing = _missing_required(dists)

if INSTALL_NOTEBOOK_DEPS or missing:
    if missing:
        print(f"Missing packages: {missing}")
        print(f"Installing notebook dependencies from: {req_path}")
        subprocess.check_call([sys.executable, "-m", "pip", "install", "-r",
↪str(req_path)])
    else:
        print(f"All {len(dists)} notebook dependencies present; skipping install.")
        print("(Set INSTALL_NOTEBOOK_DEPS = True to force a reinstall.)")

```

All 14 notebook dependencies present; skipping install.
(Set INSTALL_NOTEBOOK_DEPS = True to force a reinstall.)

1.1 1. Setup — Locate Project Root

```

[43]: import os, sys
from pathlib import Path

def _is_project_root(path: str) -> bool:
    return os.path.isdir(os.path.join(path, "src")) and os.path.isdir(os.path.
↪join(path, "configs"))

_candidates = ["..", ".", os.path.join("..", ".."),
               os.path.join("Thesis-Energy-Auction", "ets_marl_happo_current"),
↪"Thesis-Energy-Auction"]
_found = _is_project_root(".")
if not _found:
    for _c in _candidates:
        if _is_project_root(_c):
            os.chdir(_c); _found = True; break
if not _found:
    raise FileNotFoundError(f"Cannot find project root (needs src/ + configs/).
↪\nCWD: {os.getcwd()} | Contents: {os.listdir('.')}")

```

```
PROJECT_ROOT = Path.cwd().resolve()
sys.path.insert(0, str(PROJECT_ROOT))
print(f"Working directory: {PROJECT_ROOT}")
```

Working directory: C:\Users\danie\Documents\Python_Scripts\Master Thesis\Thesis-Energy-Auction

1.2 2. Imports

```
[44]: import re
import yaml
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
from src.utils.run_data import load_run_csv, glob_run_logs # parquet-cached
↳ log loader
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style('whitegrid')
pd.set_option('display.max_columns', 200)
pd.set_option('display.width', 200)

from src.utils.sweep import load_sweep_spec, expand_dotted_overrides, deep_merge
```

1.3 3. Sweep Specs to Load

List one or more sweep specs (paths relative to the project root). All of them are loaded and merged into a single dict keyed by (`sweep_name`, `variant_name`, `seed`). The default below loads only `default_seeds.yaml`, but additional default-config sweeps can be appended freely — every run that survives the §5 filter is treated as another seed of the *same* default-config population.

If a sweep contains variants whose `overrides` deviate from the base config, those variants are flagged in §5 and excluded from the analysis by default (this notebook is for the default config only). To include them anyway, set `STRICT_DEFAULT_ONLY = False` in §5.

```
[45]: # Add additional default-config sweeps here as the project produces them.
SWEEP_SPECS = [
    PROJECT_ROOT / 'configs' / 'sweeps' / 'default_seeds.yaml',
    # PROJECT_ROOT / 'configs' / 'sweeps' / 'default_seeds_old.yaml',
    # PROJECT_ROOT / 'configs' / 'sweeps' / 'default_seeds_expanded.yaml'
]

# Optional: override the output_dir from the spec (useful if results were
# moved). One entry per spec, or None to use the spec's own output_dir.
```

```

OUTPUT_DIR_OVERRIDES = [
    PROJECT_ROOT / 'results' / 'sweeps' / 'default',
    # PROJECT_ROOT / 'results' / 'sweeps',
    # PROJECT_ROOT / 'results' / 'sweeps' / 'default'
]
assert len(OUTPUT_DIR_OVERRIDES) == len(SWEEP_SPECS), "one override (or None)␣
↳per spec"

for sp in SWEEP_SPECS:
    if not Path(sp).exists():
        warnings.warn(f"Spec not found: {sp}")

print(f"Loading {len(SWEEP_SPECS)} sweep spec(s):")
for sp, od in zip(SWEEP_SPECS, OUTPUT_DIR_OVERRIDES):
    print(f"  - {sp} (override={od})")

```

Loading 1 sweep spec(s):

```

- C:\Users\danie\Documents\Python_Scripts\Master Thesis\Thesis-Energy-
Auction\configs\sweeps\default_seeds.yaml
(override=C:\Users\danie\Documents\Python_Scripts\Master Thesis\Thesis-Energy-
Auction\results\sweeps\default)

```

1.4 4. Multi-Sweep Discovery & Loading

```

[46]: from src.utils.run_data import log_exists, default_ep_columns,␣
↳default_yr_columns
from tqdm.auto import tqdm

def _resolve_run_files(variant_name, seed, variant_dir, output_dir):
    """Return (training_log, year_log) paths or (None, None). Tagged then␣
↳untagged."""
    search_dirs = []

    # Existing locations
    for d in (variant_dir, output_dir):
        if d is not None:
            d = Path(d)
            if d not in search_dirs:
                search_dirs.append(d)

    # Extra fallback: results/sweep/<tag>[/<variant>]
    # (tag inferred from output_dir name, e.g. default_seeds)
    if output_dir is not None:
        tag = Path(output_dir).name
        alt_base = PROJECT_ROOT / "results" / "sweep" / tag
        for d in (alt_base / variant_name, alt_base):

```

```

        if d not in search_dirs:
            search_dirs.append(d)

    training_names = [f"training_log_{variant_name}_s{seed}.csv",
↪f"training_log_s{seed}.csv"]
    year_names = [f"year_log_{variant_name}_s{seed}.csv", f"year_log_s{seed}.
↪csv"]

    tlog = ylog = None
    for base in search_dirs:
        if tlog is None:
            for n in training_names:
                cand = base / n
                if log_exists(cand):
                    tlog = cand
                    break
        if ylog is None:
            for n in year_names:
                cand = base / n
                if log_exists(cand):
                    ylog = cand
                    break
        if tlog is not None and ylog is not None:
            break

    return tlog, ylog

# Project both logs onto the analysis-superset columns at load time. The
# year_log carries 270+ diagnostic columns; pulling only the ~80 actually
# used by this notebook cuts in-RAM size by ~5-10x. Unknown columns are
# silently dropped by load_run_csv.
EP_COLUMNS = default_ep_columns()
YR_COLUMNS = list(dict.fromkeys(
    default_yr_columns()
    + [f"sec_price_mult_A{i}" for i in range(1, 17)]
))

def _load_csv_or_none(path, columns=None):
    if path is None: return None
    return load_run_csv(path, columns=columns)

def _empty_ep(yr=None):
    if yr is not None and 'episode' in yr.columns:
        return pd.DataFrame({'episode': np.sort(yr['episode'].dropna().
↪unique())})
    return pd.DataFrame({'episode': pd.Series(dtype='int64')})

```

```

def _empty_yr(ep=None):
    if ep is not None and 'episode' in ep.columns and len(ep) > 0:
        return pd.DataFrame({'episode': ep['episode'].astype('int64').values,
                              'year': np.zeros(len(ep), dtype='int64')})
    return pd.DataFrame({'episode': pd.Series(dtype='int64'), 'year': pd.
        ↳Series(dtype='int64')})

specs = []
for spec_path, override in zip(SWEEP_SPECS, OUTPUT_DIR_OVERRIDES):
    if not Path(spec_path).exists():
        continue
    spec = load_sweep_spec(str(spec_path))
    sweep_name = Path(spec_path).stem
    with open(spec['base_config'], encoding='utf-8') as f:
        base_cfg = yaml.safe_load(f)
    output_dir = Path(override) if override is not None else (PROJECT_ROOT /
        ↳spec['output_dir']).resolve()
    specs.append({'name': sweep_name, 'spec': spec, 'base_cfg': base_cfg,
        ↳'output_dir': output_dir})

# Pre-enumerate every (sweep, variant, seed) we'll attempt so tqdm has a real
↳total.
load_jobs = []
for s in specs:
    for v in s['spec']['variants']:
        seeds = v['seeds'] if v['seeds'] is not None else s['spec']['seeds']
        variant_dir = s['output_dir'] / v['name']
        merged_cfg = deep_merge(s['base_cfg'], expand_dotted_overrides(v.
        ↳get('overrides') or {}))
        flat_overrides = expand_dotted_overrides(v.get('overrides') or {})
        for seed in seeds:
            load_jobs.append((s, v, seed, variant_dir, merged_cfg,
        ↳flat_overrides))

# Build (sweep, variant, seed) -> run dict.
runs = {}
discovery = []
for s, v, seed, variant_dir, merged_cfg, flat_overrides in tqdm(
    load_jobs, desc='Loading sweep CSVs', unit='run', leave=True
):
    tlog, ylog = _resolve_run_files(v['name'], seed, variant_dir,
        ↳s['output_dir'])
    row = {
        'sweep': s['name'], 'variant': v['name'], 'seed': seed,
        'has_overrides': bool(flat_overrides),
        'training_log': str(tlog) if tlog else None,
    }

```

```

        'year_log': str(ylog) if ylog else None,
        'has_training_log': tlog is not None,
        'has_year_log': ylog is not None,
        'found': (tlog is not None) or (ylog is not None),
    }
    discovery.append(row)
    if not row['found']:
        continue
    ep_df = _load_csv_or_none(tlog, columns=EP_COLUMNS)
    yr_df = _load_csv_or_none(ylog, columns=YR_COLUMNS)
    if ep_df is None and yr_df is None:
        continue
    if ep_df is None: ep_df = _empty_ep(yr_df)
    if yr_df is None: yr_df = _empty_yr(ep_df)
    runs[(s['name'], v['name'], seed)] = {
        'ep_df': ep_df, 'yr_df': yr_df, 'config': merged_cfg,
        'has_overrides': bool(flat_overrides),
        'overrides': flat_overrides,
        'has_training_log': bool(tlog), 'has_year_log': bool(ylog),
    }

discovery_df = pd.DataFrame(discovery)
n_found = int(discovery_df['found'].sum()) if not discovery_df.empty else 0
print(f"Discovered {len(discovery_df)} (sweep, variant, seed) triples; loaded_{n_found}")
if not runs:
    raise RuntimeError("No runs loaded. Check SWEEP_SPECS /_{OUTPUT_DIR_OVERRIDES}")
discovery_df

# Free chunked CSV/parquet readers held by pandas/pyarrow before continuing.
import gc; gc.collect()

```

Loading sweep CSVs: 0%| | 0/4 [00:00<?, ?run/s]

Discovered 4 (sweep, variant, seed) triples; loaded 4.

[46]: 144

1.5 5. Default-Only Filter & Sanity Overview

```

[47]: STRICT_DEFAULT_ONLY = True # if True, drop runs whose variant has any override.

if STRICT_DEFAULT_ONLY:
    n_before = len(runs)
    runs = {k: v for k, v in runs.items() if not v['has_overrides']}
    n_after = len(runs)

```



```

    if n_after < n_before:
        print(f"Filtered out {n_before - n_after} run(s) whose variant_
↳ overrides the base config.")
    if n_after == 0:
        raise RuntimeError("STRICT_DEFAULT_ONLY filtered out every run. Set_
↳ STRICT_DEFAULT_ONLY = False to inspect non-default variants.")

# Treat every surviving run as a default-config seed. Build a flat list.
ALL_RUN_KEYS = sorted(runs.keys())
print(f"\nUsable default-config runs: {len(ALL_RUN_KEYS)}")
for (sw, var, seed), run in sorted(runs.items()):
    n_eps = len(run['ep_df'])
    n_yrs = len(run['yr_df'])
    print(f" {sw}/{var}/seed={seed:>4} ({n_eps:>6} episodes, {n_yrs:>7}_
↳ year-rows)")

# Pick a reference config. All defaults agree on schema; just take the first.
REFERENCE_CONFIG = runs[ALL_RUN_KEYS[0]]['config']
N_AGENTS = int(REFERENCE_CONFIG.get('companies', {}).get('n_agents', 8))
N_BOTS = int(REFERENCE_CONFIG.get('companies', {}).get('n_bot_agents', 0))
N_TOTAL = N_AGENTS + N_BOTS
N_YEARS = int(REFERENCE_CONFIG.get('environment', {}).get('n_years', 12))
YEAR_DISPLAY_START = 1
YEAR_DISPLAY_END = N_YEARS
print(f"\nReference config: n_agents={N_AGENTS}, n_bot_agents={N_BOTS},_
↳ n_years={N_YEARS}")

def _year_log_to_display_years(yr_df):
    if yr_df is None or yr_df.empty or 'year' not in yr_df.columns:
        return yr_df, False
    year = pd.to_numeric(yr_df['year'], errors='coerce')
    finite_year = year.dropna()
    if finite_year.empty:
        return yr_df, False
    if finite_year.min() >= YEAR_DISPLAY_START:
        return yr_df, False
    if finite_year.min() == 0 and finite_year.max() <= max(N_YEARS - 1, 0):
        out = yr_df.copy()
        out['year'] = year + YEAR_DISPLAY_START
        if out['year'].notna().all() and np.allclose(out['year'], np.
↳ round(out['year'])):
            out['year'] = out['year'].astype('int64')
        return out, True
    raise ValueError(f"Unexpected year range in year_log: {finite_year.min()}..
↳ {finite_year.max()}")

```

```

n_shifted_year_frames = 0
for run in runs.values():
    run['yr_df'], shifted = _year_log_to_display_years(run.get('yr_df'))
    n_shifted_year_frames += int(shifted)

if n_shifted_year_frames:
    print(f"Converted {n_shifted_year_frames} year_log frame(s) from years_
↳0-{N_YEARS - 1} to years {YEAR_DISPLAY_START}-{YEAR_DISPLAY_END}.")
else:
    print(f"Year_log frames already use display years_
↳{YEAR_DISPLAY_START}-{YEAR_DISPLAY_END}.")

# Archetype/reward labelling (default config: 4 archetypes x {financial, ESG}).
ARCHETYPES = ['Coal-heavy', 'Gas-dominant', 'Transitioner', 'Green-leader']
def _agent_archetype(idx0):
    return ARCHETYPES[idx0 // 2] if idx0 // 2 < len(ARCHETYPES) else_
↳f'Custom-{idx0//2}'
def _agent_weighting(idx0):
    # Even indices (A1, A3, ...) are pure financial; odd are balanced ESG. (Per_
↳README.)
    return 'Financial' if idx0 % 2 == 0 else 'Balanced ESG'

AGENT_LABELS = [f'A{i+1} ({_agent_archetype(i)}, {_agent_weighting(i)})' for i_
↳in range(N_AGENTS)]
print("Agent roster (1-based):")
for lbl in AGENT_LABELS:
    print(f" {lbl}")

```

Usable default-config runs: 4

default_seeds/default/seed=1729	(120000 episodes, 1440000 year-rows)
default_seeds/default/seed=5041	(120000 episodes, 1440000 year-rows)
default_seeds/default/seed=6561	(120000 episodes, 1440000 year-rows)
default_seeds/default/seed=8191	(120000 episodes, 1440000 year-rows)

Reference config: n_agents=8, n_bot_agents=0, n_years=12

Converted 4 year_log frame(s) from years 0-11 to years 1-12.

Agent roster (1-based):

```

A1 (Coal-heavy, Financial)
A2 (Coal-heavy, Balanced ESG)
A3 (Gas-dominant, Financial)
A4 (Gas-dominant, Balanced ESG)
A5 (Transitioner, Financial)
A6 (Transitioner, Balanced ESG)
A7 (Green-leader, Financial)
A8 (Green-leader, Balanced ESG)

```

1.5.1 5a. Helpers

A handful of helpers reused across §RQ1–§RQ3. We intentionally treat every loaded run as one seed of the same default-config population (no variant axis) — the helpers therefore aggregate over ALL_RUN_KEYS without any variant grouping.

```
[60]: ROLL = 50 # episode-axis smoothing window
TAIL_EPISODES = 1000 # converged-window definition for steady-state metrics

def agent_cols(df, prefix, max_idx=N_AGENTS):
    """Return columns 'prefix_A1' ... 'prefix_A{max_idx}' that exist in df,
    sorted by agent index."""
    rx = re.compile(rf'{re.escape(prefix)}_A(\d+)\$')
    out = []
    for c in df.columns:
        m = rx.match(c)
        if m and int(m.group(1)) <= max_idx:
            out.append((int(m.group(1)), c))
    return [c for _, c in sorted(out)]

def per_run_series(metric_fn, source='ep'):
    """Return dict {run_key: pd.Series indexed by episode} from each run."""
    out = {}
    for k, run in runs.items():
        df = run['yr_df'] if source == 'yr' else run['ep_df']
        if source == 'run':
            s = metric_fn(run)
        else:
            s = metric_fn(df)
        if s is None or len(s) == 0:
            continue
        s = s.copy()
        if source == 'ep' and 'episode' in run['ep_df'].columns:
            s.index = run['ep_df']['episode']
        out[k] = s
    return out

def stack_runs(metric_fn, source='ep'):
    """Return wide DataFrame of episode-indexed series, one column per run."""
    series = per_run_series(metric_fn, source=source)
    if not series:
        return pd.DataFrame()
    return pd.concat(series, axis=1).sort_index()

def fan_chart(df, ax, color='steelblue', label='runs', smooth=ROLL):
    """Plot mean ±1 std fan + thin per-run lines for a wide DataFrame."""
    if df.empty:
```

```

        ax.text(0.5, 0.5, 'no data', ha='center', va='center', transform=ax.
↳transAxes)

        return

    sm = df.rolling(smooth, min_periods=1).mean()
    mean = sm.mean(axis=1)
    std = sm.std(axis=1).fillna(0.0)
    for col in sm.columns:
        ax.plot(sm.index, sm[col].values, color=color, lw=0.6, alpha=0.30)
    ax.fill_between(sm.index, (mean - std).values, (mean + std).values,
                    color=color, alpha=0.20, label=f'{label} ±1 std (n={sm.
↳shape[1]})')
    ax.plot(sm.index, mean.values, color=color, lw=2.2, label=f'{label} mean')
    ax.grid(True, alpha=0.3); ax.legend(fontsize=8, loc='best')

def tail_episodes(ep_df, n=TAIL_EPISODES):
    if ep_df.empty: return ep_df
    return ep_df.tail(min(n, len(ep_df)))

def tail_year_rows(run, n=TAIL_EPISODES):
    ep = tail_episodes(run['ep_df'], n)
    if ep.empty or 'episode' not in run['yr_df'].columns:
        return run['yr_df'].head(0)
    return run['yr_df'][run['yr_df']['episode'].isin(ep['episode'].unique())]

def format_year_axis(ax, *, pad=0.35):
    start = globals().get('YEAR_DISPLAY_START', 1)
    end = globals().get('YEAR_DISPLAY_END', globals().get('N_YEARS', 12))
    ax.set_xlim(start - pad, end + pad)
    ax.set_xticks(np.arange(start, end + 1, dtype=int))

```

1.6 §RQ1 — Stable & Behaviourally Credible Simulation

Three sub-checks:

1. **Convergence** — system reward, composite quality score, and PPO actor/critic losses settle by training end.
2. **Cross-seed reproducibility** — fan charts on headline metrics across loaded seeds; the spread of converged-window means quantifies how reliably the same equilibrium is reached.
3. **Behavioural credibility** — final-window clearing price sits in the EU-ETS-plausible range (~€60–110 per the README's design notes), compliance rate plateaus high, and warning-flag counts in the converged window are not pathological.

1.6.1 RQ1.1 — Convergence dynamics (rewards, quality score, losses)

```
[49]: def _system_reward(df):
    cols = agent_cols(df, 'reward')
    return df[cols].sum(axis=1) if cols else pd.Series(dtype=float)

def _system_base_reward(df):
    cols = agent_cols(df, 'reward_base')
    return df[cols].sum(axis=1) if cols else pd.Series(dtype=float)

def _quality(df):
    return df['quality_score'] if 'quality_score' in df.columns else pd.
    ↪Series(dtype=float)

def _mean_actor_loss(df):
    cols = agent_cols(df, 'actor_loss')
    return df[cols].mean(axis=1) if cols else pd.Series(dtype=float)

def _mean_critic_loss(df):
    cols = agent_cols(df, 'critic_loss')
    return df[cols].mean(axis=1) if cols else pd.Series(dtype=float)

fig, axes = plt.subplots(2, 2, figsize=(16, 9))
fan_chart(stack_runs(_system_reward), axes[0, 0], color='steelblue',
    ↪label='system reward')
axes[0, 0].set(title='System total reward ( $\Sigma$  reward_A*)', xlabel='Episode',
    ↪ylabel='reward')

fan_chart(stack_runs(_quality), axes[0, 1], color='seagreen',
    ↪label='quality_score')
axes[0, 1].set(title='Composite quality score', xlabel='Episode', ylabel='Q
    ↪[-5, 5]')
axes[0, 1].set_ylim(0, 3)

fan_chart(stack_runs(_mean_actor_loss), axes[1, 0], color='firebrick',
    ↪label='actor loss')
axes[1, 0].set(title='Mean actor loss across agents', xlabel='Episode',
    ↪ylabel='loss')

fan_chart(stack_runs(_mean_critic_loss), axes[1, 1], color='darkorange',
    ↪label='critic loss')
axes[1, 1].set(title='Mean critic loss across agents', xlabel='Episode',
    ↪ylabel='loss')

plt.suptitle('RQ1.1 - Training-time convergence (mean  $\pm$ 1 std across loaded
    ↪seeds)',
    fontsize=13, y=1.02)
```

```
plt.tight_layout(); plt.show()
```

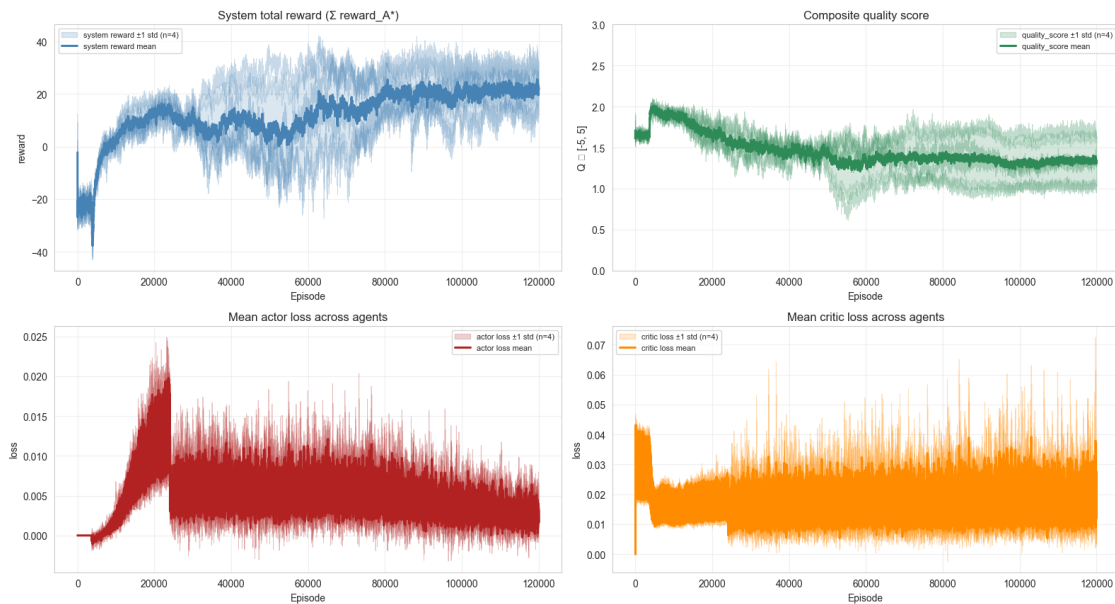
C:\Users\danie\AppData\Local\Temp\ipykernel_28804\61372440.py:36: UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.

```
plt.tight_layout(); plt.show()
```

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```

RQ1.1 — Training-time convergence (mean $\pm$ 1 std across loaded seeds)



1.6.2 RQ1.2 — Cross-seed reproducibility on headline metrics

```
[9]: # Per-seed converged-window means for a small set of headline metrics.
def _converged_metrics_for(run):
    ep = tail_episodes(run['ep_df'])
    yr = tail_year_rows(run)
    out = {}

    rew_cols = agent_cols(ep, 'reward')
    out['system_reward'] = float(ep[rew_cols].sum(axis=1).mean()) if
↪rew_cols else np.nan
    out['quality_score'] = float(ep['quality_score'].mean()) if
↪'quality_score' in ep.columns else np.nan

    if 'clearing_price' in yr.columns:
        out['mean_clearing_price'] = float(yr['clearing_price'].mean())
```

```

elif 'clearing_price_last' in ep.columns:
    out['mean_clearing_price'] = float(ep['clearing_price_last'].mean())
else:
    out['mean_clearing_price'] = np.nan

sf_cols = agent_cols(yr, 'shortfall')
out['compliance_rate'] = float((yr[sf_cols] <= 1e-6).mean(axis=1).mean())
if sf_cols else np.nan

pen_cols = agent_cols(ep, 'penalty')
out['system_penalty'] = float(ep[pen_cols].sum(axis=1).mean()) if pen_cols
else np.nan

inv_cols = agent_cols(ep, 'invest_cost')
out['system_invest'] = float(ep[inv_cols].sum(axis=1).mean()) if inv_cols
else np.nan

gf_cols = agent_cols(ep, 'green_frac')
out['mean_final_green_frac'] = float(ep[gf_cols].mean(axis=1).mean()) if gf_cols
else np.nan

if 'secondary_volume' in ep.columns:
    out['secondary_volume'] = float(ep['secondary_volume'].mean())
else:
    out['secondary_volume'] = np.nan

return out

per_seed_rows = []
for k, run in runs.items():
    row = {'sweep': k[0], 'variant': k[1], 'seed': k[2]}
    row.update(_converged_metrics_for(run))
    per_seed_rows.append(row)
per_seed_df = pd.DataFrame(per_seed_rows)
print(f"Per-seed converged-window means (last {TAIL_EPISODES} episodes):")
display(per_seed_df.round(3))

agg_cols = ['system_reward', 'quality_score', 'mean_clearing_price',
            'compliance_rate',
            'system_penalty', 'system_invest', 'mean_final_green_frac',
            'secondary_volume']
summary = per_seed_df[agg_cols].agg(['mean', 'std', 'min', 'max']).round(3)
summary.loc['cv (%)'] = (summary.loc['std'] / summary.loc['mean'].abs() * 100).
    round(1)
print(f"\nCross-seed summary (n={len(per_seed_df)}):")
summary

```

Per-seed converged-window means (last 1000 episodes):

	sweep	variant	seed	system_reward	quality_score	mean_clearing_price	compliance_rate	system_penalty	system_invest	mean_final_green_frac	secondary_volume
0	default_seeds	default	1729	20.079	1.712	66.					
	↪963	0.875	1920.884	5139.825	0.735						
	↪	43.932									
1	default_seeds	default	8191	31.476	1.331	47.					
	↪796	0.853	2529.066	6426.351	0.765						
	↪	40.209									
2	default_seeds	default	6561	12.398	1.345	56.					
	↪253	0.797	3793.327	6858.905	0.771						
	↪	27.110									
3	default_seeds	default	5041	24.875	1.015	46.					
	↪519	0.787	4301.157	5121.147	0.748						
	↪	30.116									

Cross-seed summary (n=4):

	system_reward	quality_score	mean_clearing_price	compliance_rate	system_penalty	system_invest	mean_final_green_frac	secondary_volume
mean	22.207	1.351	54.383	0.828				
3136.108	5886.557	0.755	35.342					
std	8.037	0.285	9.434	0.043				
1100.688	890.749	0.017	8.012					
min	12.398	1.015	46.519	0.787				
1920.884	5121.147	0.735	27.110					
max	31.476	1.712	66.963	0.875				
4301.157	6858.905	0.771	43.932					
cv (%)	36.200	21.100	17.300	5.200				
35.100	15.100	2.300	22.700					

Reading the table. A coefficient of variation (cv (%)) below ~10% on the headline outcomes (mean clearing price, compliance rate, mean final green fraction) is the bar we use to call the simulation *reproducible* across seeds — anything significantly larger means the RQ2 / RQ3 claims need to be hedged seed-by-seed.

1.6.3 RQ1.3 — Behavioural credibility in the converged window

```
[10]: # (a) Final-window clearing-price distribution; (b) per-seed compliance plateau.
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

ax = axes[0]
boxes = []
labels = []
for k, run in runs.items():
```



```

yr = tail_year_rows(run)
if 'clearing_price' not in yr.columns or yr.empty:
    continue
boxes.append(yr['clearing_price'].dropna().values)
labels.append(f"{k[0][:8]}/s{k[2]}")
if boxes:
    ax.boxplot(boxes, tick_labels=labels, showfliers=False)
    ax.axhspan(60, 110, color='green', alpha=0.10, label='EU-ETS plausible band_
↳( €60-110/t)')
    ax.set(title='Converged-window clearing price by seed', ylabel='€/tCO ',
↳xlabel='Run')
    ax.legend(fontsize=8); ax.tick_params(axis='x', rotation=30)
else:
    ax.text(0.5, 0.5, 'No clearing_price column in any year_log', ha='center',
↳va='center', transform=ax.transAxes)

ax = axes[1]
for k, run in runs.items():
    sf_cols = agent_cols(run['yr_df'], 'shortfall')
    if not sf_cols or 'episode' not in run['yr_df'].columns:
        continue
    rate = (run['yr_df'][sf_cols] <= 1e-6).mean(axis=1).
↳groupby(run['yr_df']['episode']).mean()
    rate_smooth = rate.rolling(ROLL, min_periods=1).mean()
    ax.plot(rate_smooth.index, rate_smooth.values, lw=1.0, alpha=0.85,
        label=f"{k[0][:8]}/s{k[2]}")
ax.set(title='Compliance rate over training (mean across (agent, year) cells)',
    xlabel='Episode', ylabel='compliance rate [0,1]')
ax.set_ylim(0.0, 1.02); ax.grid(True, alpha=0.3); ax.legend(fontsize=8,
↳loc='best')

plt.suptitle('RQ1.3 - Behavioural credibility: realistic price band +
↳compliance plateau',
    fontsize=13, y=1.02)
plt.tight_layout(); plt.show()

```

C:\Users\danie\AppData\Local\Temp\ipykernel_28804\2922376889.py:36: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

plt.tight_layout(); plt.show()

C:\Users\danie\AppData\Local\Temp\ipykernel_28804\2922376889.py:36: UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.

plt.tight_layout(); plt.show()

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

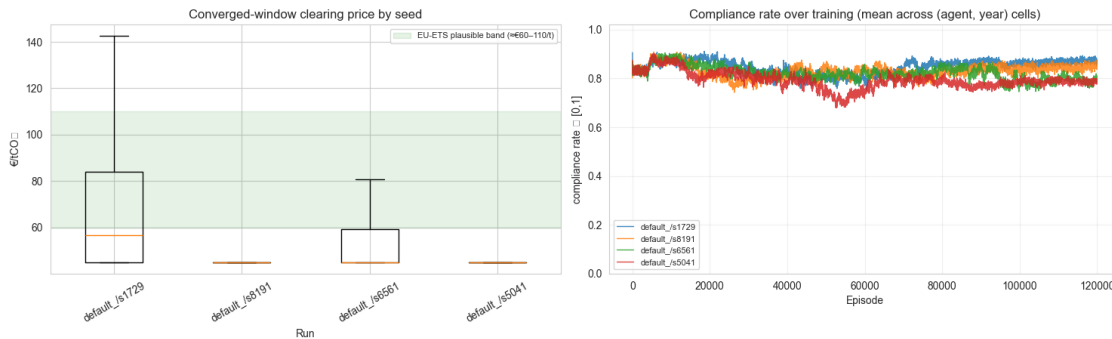
fig.canvas.print_figure(bytes_io, **kw)

C:\Users\danie\AppData\Roaming\Python\Python312\site-

packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.

```
fig.canvas.print_figure(bytes_io, **kw)
```

RQ1.3 — Behavioural credibility: realistic price band + compliance plateau



1.6.4 RQ1.4 — Pathology check: warnings in the converged window

```
[11]: # Sum the warn_* columns across the converged window for each run.
WARN_PREFIX = 'warn_'
warn_rows = []
warn_columns_seen = set()
for k, run in runs.items():
    ep = tail_episodes(run['ep_df'])
    warn_cols = [c for c in ep.columns if c.startswith(WARN_PREFIX)]
    warn_columns_seen.update(warn_cols)
    if not warn_cols:
        continue
    row = {'run': f"{k[0][:8]}/s{k[2]}"}
    # Per-episode mean warning count (interpretable as "how often per episode
    # does this warn fire").
    row.update({c: float(ep[c].mean()) for c in warn_cols})
    warn_rows.append(row)

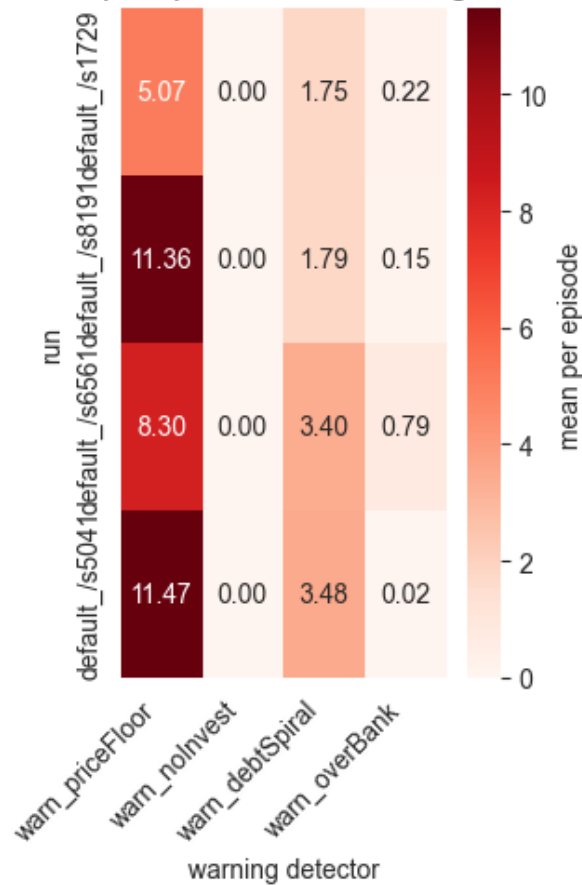
if warn_rows:
    warn_df = pd.DataFrame(warn_rows).set_index('run').fillna(0.0)
    # Drop columns that are exactly zero across all runs.
    warn_df = warn_df.loc[:, (warn_df > 0).any(axis=0)]
    fig, ax = plt.subplots(figsize=(min(14, 1.5 + 0.45 * len(warn_df.columns)),
    4 + 0.3 * len(warn_df)))
    if warn_df.empty:
        ax.text(0.5, 0.5, 'No non-zero warnings in the converged window - clean
        run.',
                ha='center', va='center', transform=ax.transAxes, fontsize=12)
    ax.axis('off')
```

```

else:
    sns.heatmap(warn_df.round(2), annot=True, cmap='Reds', fmt='.2f',
    ↳cbar_kws={'label': 'mean per episode'}, ax=ax)
    ax.set(title=f'Mean warning-flag counts per episode in the converged_
    ↳window (last {TAIL_EPISODES} eps)',
           xlabel='warning detector', ylabel='run')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout(); plt.show()
    print(f'Inspected {len(warn_columns_seen)} warning columns; showing only_
    ↳the {warn_df.shape[1]} with > 0 events.')
else:
    print('No warn_* columns present in episode logs.')

```

Mean warning-flag counts per episode in the converged window (last 1000 eps)



Inspected 6 warning columns; showing only the 4 with > 0 events.

1.6.5 RQ1.5 — Whole-training overview (all loaded runs)

A single 2×2 panel summarising the entire training trajectory across *every* loaded run (i.e. before the §RQ2 default-seeds filter kicks in). Top-left: system reward; top-right: final-year clearing price (per-episode `clearing_price` at the maximum simulation year); bottom-left: compliance rate; bottom-right: mean green generation share. The shaded band on each panel is ± 1 **standard deviation around the mean** of the loaded runs at every smoothed episode, with thin lines showing the individual runs.

The clearing-price panel adds two horizontal references taken from `configs/default.yaml`: the auction reserve (floor) price and the final-year penalty rate (`penalty.rate × (1 + inflation_rate)^(N_YEARS - 1)`). The green-share panel marks the cross-agent mean of the *initial* (display year 1) green fractions read from `companies.initial_mix × technologies.is_green`. Compliance and green-share use a tight padded y-axis so changes during training are easy to read.

```
[79]: # 2x2 whole-training overview across all loaded runs.
# Episode-axis fan charts (mean  $\pm 1$  std band across runs, smoothed by ROLL).

def _final_year_clearing_by_ep(run):
    """Per-episode clearing price in the final simulation year."""
    yr = run['yr_df']
    needed = {'clearing_price', 'episode', 'year'}
    if not needed.issubset(yr.columns):
        return pd.Series(dtype=float)
    idx = yr.groupby('episode')['year'].idxmax()
    s = yr.loc[idx].set_index('episode')['clearing_price']
    return s.sort_index()

def _compliance_rate_by_ep(run):
    yr = run['yr_df']
    sf_cols = agent_cols(yr, 'shortfall')
    if not sf_cols or 'episode' not in yr.columns:
        return pd.Series(dtype=float)
    return (yr[sf_cols] <= 1e-6).mean(axis=1).groupby(yr['episode']).mean()

def _mean_green_frac_by_ep(df):
    cols = agent_cols(df, 'green_frac')
    return df[cols].mean(axis=1) if cols else pd.Series(dtype=float)

# Display year 1 (start-of-simulation) mean green fraction from the default_
↳ config.
_mix = REFERENCE_CONFIG.get('companies', {}).get('initial_mix', []) or []
_is_green = REFERENCE_CONFIG.get('technologies', {}).get('is_green', []) or []
_green_idx = [i for i, g in enumerate(_is_green) if g]
if _mix and _green_idx:
    _agent_start_green = [sum(row[i] for i in _green_idx) for row in _mix[:
↳ N_AGENTS]]
```

```

START_GREEN_FRAC = float(np.mean(_agent_start_green))
else:
    START_GREEN_FRAC = float('nan')

def _std_fan(df_wide, ax, *, color, label, smooth=ROLL):
    """Plot mean  $\pm 1$  std band (across runs) of `df_wide` on `ax`, plus thin
    per-run lines."""
    if df_wide.empty:
        ax.text(0.5, 0.5, 'no data', ha='center', va='center', transform=ax.
        transAxes)
        return
    sm = df_wide.rolling(smooth, min_periods=1).mean()
    mean = sm.mean(axis=1)
    std = sm.std(axis=1).fillna(0.0)
    n = sm.shape[1]
    for col in sm.columns:
        ax.plot(sm.index, sm[col].values, color=color, lw=0.6, alpha=0.30)
    ax.fill_between(sm.index, (mean - std).values, (mean + std).values,
                    color=color, alpha=0.20, label=f'{label}  $\pm 1$  std (n={n})')
    ax.plot(sm.index, mean.values, color=color, lw=2.2, label=f'{label} mean')
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=8, loc='best')

def _padded_ylim_std(df_wide, *, smooth=ROLL, pad_frac=0.10, include=None,
                    lo_clip=None, hi_clip=None):
    """Tight padded (ymin, ymax) around the smoothed mean  $\pm 1$  std band."""
    if df_wide.empty:
        return None
    sm = df_wide.rolling(smooth, min_periods=1).mean()
    mean = sm.mean(axis=1)
    std = sm.std(axis=1).fillna(0.0)
    lo = float((mean - std).min())
    hi = float((mean + std).max())
    if include is not None and not np.isnan(include):
        lo = min(lo, float(include))
        hi = max(hi, float(include))
    span = max(hi - lo, 1e-9)
    pad = pad_frac * span
    ylo, yhi = lo - pad, hi + pad
    if lo_clip is not None:
        ylo = max(ylo, lo_clip)
    if hi_clip is not None:
        yhi = min(yhi, hi_clip)
    return (ylo, yhi)

```

```

# Reserve (auction floor) price and final-year penalty rate from the
# default config - used as horizontal reference lines on the clearing-

```

```

# price panel, and reported in the companion table.
_ets_cfg = REFERENCE_CONFIG.get('ets', {}) or {}
RESERVE_PRICE = float(_ets_cfg.get('reserve_price', float('nan')))
_pen_cfg = REFERENCE_CONFIG.get('penalty', {}) or {}
_pen_base = float(_pen_cfg.get('rate', float('nan')))
_pen_infl = float(_pen_cfg.get('inflation_rate', 0.0) or 0.0)
# Final display year is N_YEARS; inflation still accrues over N_YEARS - 1
↳ intervals.
FINAL_YEAR_PENALTY_RATE = (
    _pen_base * (1.0 + _pen_infl) ** max(N_YEARS - 1, 0)
    if not np.isnan(_pen_base) else float('nan')
)

# Episode-indexed wide frames (one column per run).
overview_panels = {
    'System reward': (stack_runs(_system_reward,
↳ source='ep'), 'reward', 'steelblue'),
    'Final-year clearing price': (stack_runs(_final_year_clearing_by_ep,
↳ source='run'), '€/tCO2', 'firebrick'),
    'Compliance rate': (stack_runs(_compliance_rate_by_ep,
↳ source='run'), 'compliance in [0,1]', 'seagreen'),
    'Mean green generation share': (stack_runs(_mean_green_frac_by_ep,
↳ source='ep'), 'green frac in [0,1]', 'darkorange'),
}

fig, axes = plt.subplots(2, 2, figsize=(15, 8))
for ax, (title, (df_wide, ylabel, color)) in zip(axes.ravel(), overview_panels.
↳ items()):
    _std_fan(df_wide, ax, color=color, label=title)
    ax.set(title=title, xlabel='Episode', ylabel=ylabel)
    if title == 'Final-year clearing price':
        _refs = [v for v in (RESERVE_PRICE, FINAL_YEAR_PENALTY_RATE) if not np.
↳ isnan(v)]
        ylim = _padded_ylim_std(df_wide, pad_frac=0.08,
                                include=None)
        if ylim is not None:
            lo, hi = ylim
            for r in _refs:
                lo = min(lo, r); hi = max(hi, r)
            ax.set_ylim(lo, hi)
        if not np.isnan(RESERVE_PRICE):
            ax.axhline(RESERVE_PRICE, color='black', lw=1.0, linestyle='--',
↳ alpha=0.7,
                        label=f'reserve price ({RESERVE_PRICE:.1f} €/t)')
        if not np.isnan(FINAL_YEAR_PENALTY_RATE):

```

```

        ax.axhline(FINAL_YEAR_PENALTY_RATE, color='dimgray', lw=1.0,
↪linestyle=':', alpha=0.8,
                    label=f'final-year penalty rate_
↪({FINAL_YEAR_PENALTY_RATE:.1f} €/t)')
        if _refs:
            ax.legend(fontsize=8, loc='best')
        elif title == 'Compliance rate':
            ylim = _padded_ylim_std(df_wide, pad_frac=0.15, lo_clip=0.0, hi_clip=1.
↪02)

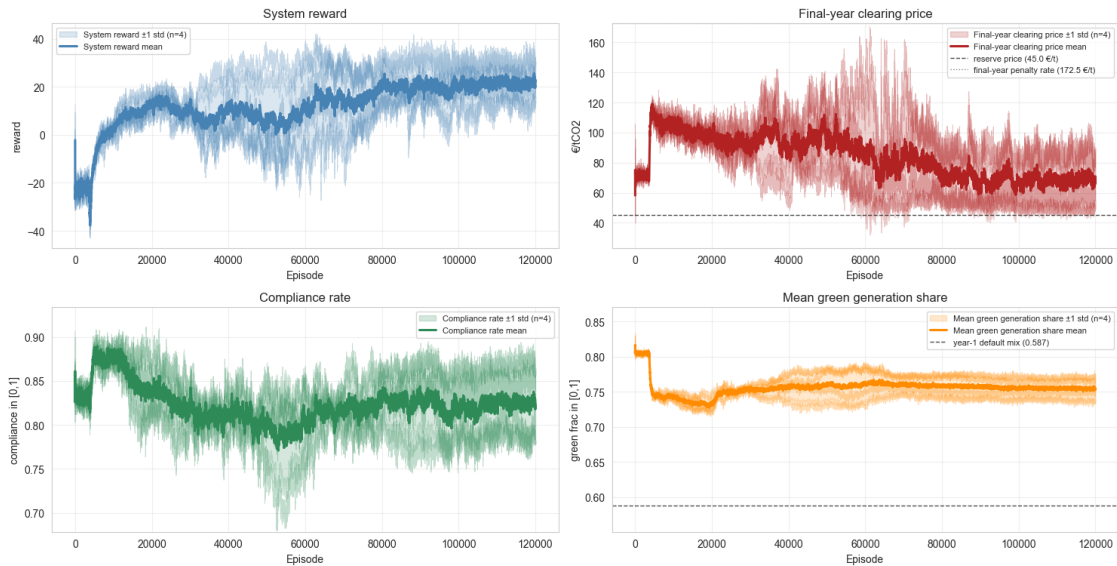
            if ylim is not None:
                ax.set_ylim(*ylim)
            elif title == 'Mean green generation share':
                ylim = _padded_ylim_std(df_wide, pad_frac=0.15,
↪include=START_GREEN_FRAC,
                                lo_clip=0.0, hi_clip=1.0)

            if ylim is not None:
                ax.set_ylim(*ylim)
            if not np.isnan(START_GREEN_FRAC):
                ax.axhline(START_GREEN_FRAC, color='black', lw=1.0, linestyle='--',
↪alpha=0.7,
                                label=f'year-1 default mix ({START_GREEN_FRAC:.3f})')
                ax.legend(fontsize=8, loc='best')

plt.suptitle(f'RQ1.5 - Whole-training overview across {len(runs)} loaded run(s)_
↪'
            f'(mean ±1 std across runs, ROLL={ROLL})', fontsize=13, y=1.02)
plt.tight_layout(); plt.show()

```

RQ1.5 — Whole-training overview across 4 loaded run(s) (mean ±1 std across runs, ROLL=50)



```
[51]: # Companion data table for the RQ1.5 overview figure: absolute values
# of the smoothed mean series (the line drawn in each panel) reduced
# to the headline scalars used in thesis prose.

_overview_rows = []
for title, (df_wide, ylabel, _color) in overview_panels.items():
    if df_wide.empty:
        _overview_rows.append({
            'panel': title, 'unit': ylabel, 'n_runs': 0, 'episodes': 0,
            'mean': np.nan, 'median': np.nan, 'std': np.nan,
            'min': np.nan, 'max': np.nan,
            'first': np.nan, 'last': np.nan,
        })
        continue
    sm = df_wide.rolling(ROLL, min_periods=1).mean()
    mean = sm.mean(axis=1)
    _overview_rows.append({
        'panel': title, 'unit': ylabel,
        'n_runs': int(sm.shape[1]), 'episodes': int(len(mean)),
        'mean': float(mean.mean()), 'median': float(mean.median()),
        'std': float(mean.std()), 'min': float(mean.min()), 'max': float(mean.
↳max()),
        'first': float(mean.iloc[0]), 'last': float(mean.iloc[-1]),
    })
overview_df = pd.DataFrame(_overview_rows)
print('Whole-training overview - summary of the smoothed mean series shown in
↳each panel:')
display(overview_df.round(4))
print('Reference lines used in the figure:')
print(f' Year-1 default-mix mean green fraction: {START_GREEN_FRAC:.4f}')
print(f' Auction reserve (floor) price: {RESERVE_PRICE:.2f} €/t')
print(f' Final-year penalty rate (year {N_YEARS}, base {_pen_base:.2f} @
↳{_pen_infl:.1%}/yr): '
      f'{FINAL_YEAR_PENALTY_RATE:.2f} €/t')
```

Whole-training overview - summary of the smoothed mean series shown in each panel:

			panel	unit	n_runs	episodes	mean	
	↳median	std	min	max	first	last		
0			System reward	reward	4	120000	11.9544	↳
	↳12.7906	9.7120	-37.7479	25.5983	-2.1723	22.8865		
1			Final-year clearing price	€/tCO	4	120000	83.7786	84.
	↳3478	13.7480	58.1888	119.0259	58.1888	67.0147		
2			Compliance rate compliance	[0,1]	4	120000	0.8232	0.
	↳8215	0.0206	0.7709	0.8894	0.8516	0.8221		


```

3 Mean green generation share green frac [0,1] 4 120000 0.7554 0.
↪7560 0.0115 0.7271 0.8165 0.8158 0.7534

```

Reference lines used in the figure:

Year-1 default-mix mean green fraction: 0.5875

Auction reserve (floor) price: 45.00 €/t

Final-year penalty rate (year 12, base 138.75 @ 2.0%/yr): 172.52 €/t

RQ1 summary. The convergence panels (1.1) plus the cross-seed table (1.2) and credibility plots (1.3 / 1.4) together provide the evidence base for Sub-RQ 1: with the default config, the simulation converges, reproduces a plausible EU-ETS price band and compliance plateau across loaded seeds, and the converged window is not dominated by pathology warnings.

1.7 §RQ2 — Regulatory-Mechanism Behaviour Within the Default Config

Scope caveat. Real *sensitivity* analysis (cap LRF / MSR on/off / penalty level) requires sweep variants and is deferred to a separate experiments notebook. What we *can* show with default-config data is a descriptive characterisation of how the regulatory mechanisms operate in equilibrium: how the cap maps to the clearing price, how the MSR withholds and releases, when the penalty actually binds, and how quickly the system decarbonises. This is the baseline that the sensitivity analysis will perturb.

Four panels:

- **2.1** — Cap → price linkage by year (mean across episodes/seeds in the converged window).
- **2.2** — TNAC trajectory and MSR reserve / flow behaviour.
- **2.3** — Penalty incidence: how often does it bind, and how big is it when it does?
- **2.4** — Decarbonisation pace: green-share trajectory and MAC-driven reductions.

1.7.1 RQ2.1 — Cap and clearing-price trajectory by year

```

[61]: # Average across all seeds and converged-window episodes.
year_frames = []
for k, run in runs.items():
    yr = tail_year_rows(run)
    if yr.empty: continue
    keep = ['episode', 'year', 'cap', 'clearing_price', 'auction_volume',
↪'tnac',
            'msr_reserve', 'msr_withhold_this_year', 'msr_release_this_year',
            'msr_total_cancelled', 'secondary_price']
    keep = [c for c in keep if c in yr.columns]
    if 'year' not in keep: continue
    df = yr[keep].copy(); df['__seed__'] = k[2]
    year_frames.append(df)
if not year_frames:
    raise RuntimeError('No year_log data available - RQ2 requires year-level_
↪logs.')
yr_pool = pd.concat(year_frames, ignore_index=True)

```

```

by_year = yr_pool.groupby('year').agg(['mean', 'std'])

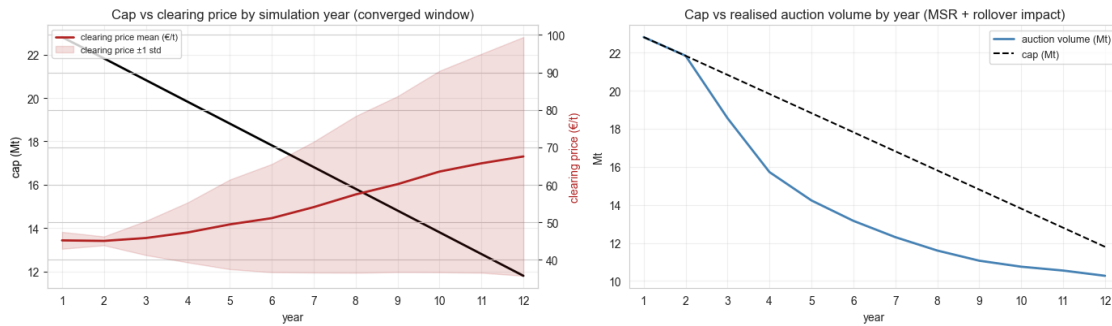
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))

ax = axes[0]
if ('cap', 'mean') in by_year.columns:
    ax.plot(by_year.index, by_year[('cap', 'mean')], lw=2, color='black',
    label='cap (Mt, mean)')
    ax.set_ylabel('cap (Mt)', color='black')
    ax2 = ax.twinx()
    if ('clearing_price', 'mean') in by_year.columns:
        clearing_mean = by_year[('clearing_price', 'mean')]
        clearing_std = by_year[('clearing_price', 'std')].fillna(0.0)
        ax2.plot(by_year.index, clearing_mean, lw=2, color='firebrick',
        label='clearing price mean (£/t)')
        ax2.fill_between(by_year.index,
        (clearing_mean - clearing_std).values,
        (clearing_mean + clearing_std).values,
        color='firebrick', alpha=0.15,
        label='clearing price  $\pm 1$  std')
        ax2.set_ylabel('clearing price (£/t)', color='firebrick')
        ax2.legend(fontsize=8, loc='best')
    ax.set(title='Cap vs clearing price by simulation year (converged window)',
    label='year')
    ax.grid(True, alpha=0.3)
    format_year_axis(ax)

ax = axes[1]
if ('auction_volume', 'mean') in by_year.columns:
    ax.plot(by_year.index, by_year[('auction_volume', 'mean')], lw=2,
    color='steelblue', label='auction volume (Mt)')
if ('cap', 'mean') in by_year.columns:
    ax.plot(by_year.index, by_year[('cap', 'mean')], lw=1.5, color='black',
    linestyle='--', label='cap (Mt)')
ax.set(title='Cap vs realised auction volume by year (MSR + rollover impact)',
    xlabel='year', ylabel='Mt')
ax.legend(fontsize=9); ax.grid(True, alpha=0.3)
format_year_axis(ax)

plt.suptitle('RQ2.1 - Regulatory cap schedule and its realised market impact',
    fontsize=13, y=1.02)
plt.tight_layout(); plt.show()

```



1.7.2 RQ2.2 — TNAC, MSR reserve, and MSR flow

```
[62]: fig, axes = plt.subplots(1, 3, figsize=(18, 4.5))

ax = axes[0]
if ('tnac', 'mean') in by_year.columns:
    tnac_mean = by_year[('tnac', 'mean')]
    tnac_std = by_year[('tnac', 'std')].fillna(0.0)
    ax.plot(by_year.index, tnac_mean, lw=2, color='darkblue', label='TNAC mean')
    ax.fill_between(by_year.index,
                    (tnac_mean - tnac_std).values,
                    (tnac_mean + tnac_std).values,
                    color='darkblue', alpha=0.15, label='TNAC ±1 std')
    ax.set(title='TNAC trajectory (Total Number of Allowances in Circulation)',
           xlabel='year', ylabel='Mt')
    ax.legend(fontsize=8); ax.grid(True, alpha=0.3)

ax = axes[1]
if ('msr_reserve', 'mean') in by_year.columns:
    ax.plot(by_year.index, by_year[('msr_reserve', 'mean')], lw=2,
            color='purple', label='MSR reserve')
if ('msr_total_cancelled', 'mean') in by_year.columns:
    ax.plot(by_year.index, by_year[('msr_total_cancelled', 'mean')], lw=2,
            color='red',
            linestyle='--', label='cumulative cancelled')
ax.set(title='MSR reserve & cancellations by year', xlabel='year', ylabel='Mt')
ax.legend(fontsize=9); ax.grid(True, alpha=0.3)

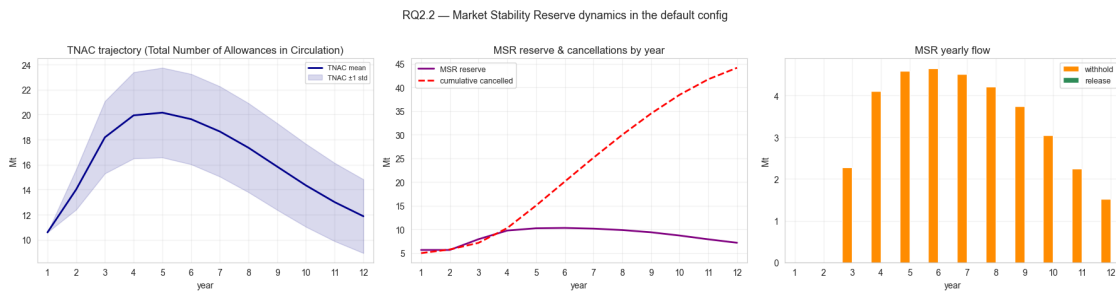
ax = axes[2]
years = by_year.index.values
if ('msr_withhold_this_year', 'mean') in by_year.columns:
    wh = by_year[('msr_withhold_this_year', 'mean')]
    ax.bar(years - 0.18, wh, width=0.35, color='darkorange', label='withhold')
```

```

if ('msr_release_this_year', 'mean') in by_year.columns:
    rel = by_year[('msr_release_this_year', 'mean')]
    ax.bar(years + 0.18, rel, width=0.35, color='seagreen', label='release')
ax.axhline(0, color='black', lw=0.8)
ax.set(title='MSR yearly flow', xlabel='year', ylabel='Mt')
ax.legend(fontsize=9); ax.grid(True, alpha=0.3)
for ax in axes:
    format_year_axis(ax)

plt.suptitle('RQ2.2 - Market Stability Reserve dynamics in the default config',
            fontsize=13, y=1.02)
plt.tight_layout(); plt.show()

```



1.7.3 RQ2.3 — Penalty incidence and magnitude

```

[16]: # (a) Share of (agent, year) cells with shortfall > 0; (b) mean per-cell
      ↪penalty when it bites.
share_rows = []
for k, run in runs.items():
    yr = tail_year_rows(run)
    sf_cols = agent_cols(yr, 'shortfall')
    pen_cols = agent_cols(yr, 'penalty')
    if not sf_cols: continue
    sf = yr[sf_cols].values.flatten()
    pen = yr[pen_cols].values.flatten() if pen_cols else None
    nc_mask = sf > 1e-6
    share_rows.append({
        'run': f'{k[0][:8]}/s{k[2]}',
        'pct_cells_short': 100.0 * nc_mask.mean() if sf.size else np.nan,
        'mean_shortfall_when_short_Mt': float(sf[nc_mask].mean()) if nc_mask.
    ↪any() else 0.0,
        'mean_penalty_when_short_Meur': float(pen[nc_mask].mean()) if (pen is
    ↪not None and nc_mask.any()) else 0.0,
    })
share_df = pd.DataFrame(share_rows)

```

```

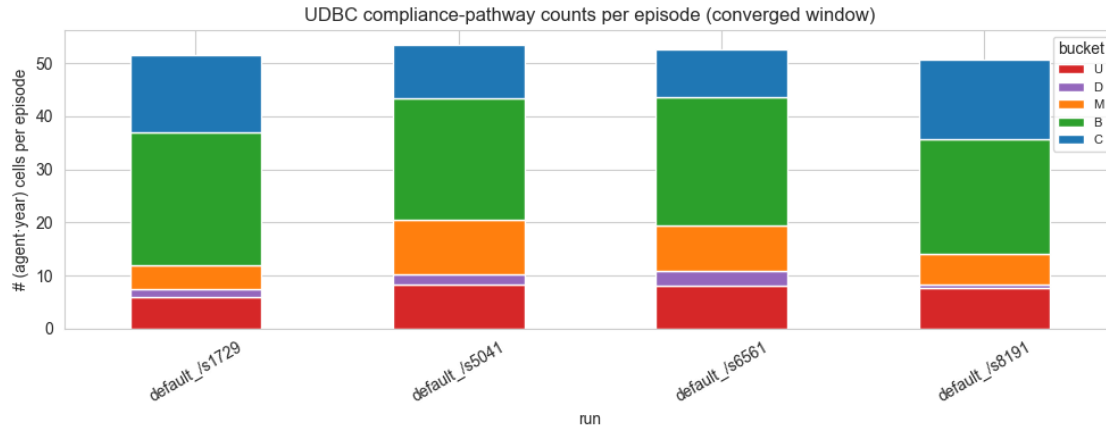
print(f'Penalty incidence in the converged window (last {TAIL_EPISODES}
↳ episodes):')
display(share_df.round(3))

# UDBC mix at the system level (averaged across seeds).
ud_rows = []
for k, run in runs.items():
    ep = tail_episodes(run['ep_df'])
    for b in ['U', 'D', 'M', 'B', 'C']:
        cols = agent_cols(ep, f'udbc_{b}_total')
        if cols:
            ud_rows.append({'run': f'{k[0][:8]}/s{k[2]}', 'bucket': b,
                            'mean_total_per_episode': float(ep[cols].
↳ sum(axis=1).mean())})
if ud_rows:
    ud_df = pd.DataFrame(ud_rows).pivot(index='run', columns='bucket',
↳ values='mean_total_per_episode').fillna(0.0)
    ud_df = ud_df[[c for c in ['U', 'D', 'M', 'B', 'C'] if c in ud_df.columns]]
    fig, ax = plt.subplots(figsize=(10, 4))
    ud_df.plot(kind='bar', stacked=True, ax=ax,
               color=['#d62728', '#9467bd', '#ff7f0e', '#2ca02c', '#1f77b4'])
    ax.set(title='UDBC compliance-pathway counts per episode (converged
↳ window)',
           ylabel='# (agent·year) cells per episode', xlabel='run')
    ax.legend(title='bucket', fontsize=8); ax.tick_params(axis='x', rotation=30)
    plt.tight_layout(); plt.show()
    print('U = pure under-bid; D = pure debt-cascade; M = mixed; B =
↳ bank-covered; C = secondary-covered.')

```

Penalty incidence in the converged window (last 1000 episodes):

	run	pct_cells_short	mean_shortfall_when_short_Mt	↳
				↳ mean_penalty_when_short_Meur
0	default_/s1729	12.494	1.002	↳
	↳ 160.154			
1	default_/s8191	14.730	1.123	↳
	↳ 178.846			
2	default_/s6561	20.312	1.229	↳
	↳ 194.530			
3	default_/s5041	21.330	1.333	↳
	↳ 210.048			



U = pure under-bid; D = pure debt-cascade; M = mixed; B = bank-covered; C = secondary-covered.

1.7.4 RQ2.4 — Decarbonisation pace

[63]: # (a) Mean green_frac by year across episodes/seeds; (b) MAC reduction by year.

```

yr_pool2 = []
for k, run in runs.items():
    yr = tail_year_rows(run)
    keep = ['year']
    gf_cols = agent_cols(yr, 'green_frac')
    mac_cols = agent_cols(yr, 'mac_reduction')
    em_cols = agent_cols(yr, 'emissions')
    if 'year' not in yr.columns: continue
    df = yr[['year']].copy()
    if gf_cols: df['green_frac_mean'] = yr[gf_cols].mean(axis=1)
    if mac_cols: df['mac_reduction_total'] = yr[mac_cols].sum(axis=1)
    if em_cols: df['emissions_total'] = yr[em_cols].sum(axis=1)
    yr_pool2.append(df)
if yr_pool2:
    pool = pd.concat(yr_pool2, ignore_index=True).groupby('year').agg(['mean', 'std'])

fig, axes = plt.subplots(1, 3, figsize=(18, 4.5))
ax = axes[0]
if ('green_frac_mean', 'mean') in pool.columns:
    green_mean = pool[('green_frac_mean', 'mean')]
    green_std = pool[('green_frac_mean', 'std')].fillna(0.0)
    ax.plot(pool.index, green_mean, lw=2, color='seagreen', label='mean')
    ax.fill_between(pool.index,
                    (green_mean - green_std).values,
                    (green_mean + green_std).values,

```

```

        color='seagreen', alpha=0.15, label='±1 std')
    ax.set(title='Mean green generation share by year', xlabel='year',
    ↪ylabel='green frac in [0,1]')
    ax.set_ylim(0, 1); ax.legend(fontsize=8); ax.grid(True, alpha=0.3)

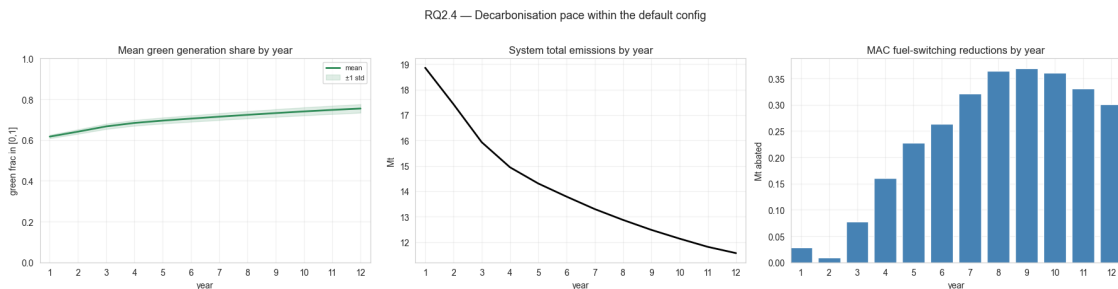
    ax = axes[1]
    if ('emissions_total', 'mean') in pool.columns:
        ax.plot(pool.index, pool[('emissions_total', 'mean')], lw=2,
    ↪color='black')
        ax.set(title='System total emissions by year', xlabel='year',
    ↪ylabel='Mt')
        ax.grid(True, alpha=0.3)

    ax = axes[2]
    if ('mac_reduction_total', 'mean') in pool.columns:
        ax.bar(pool.index, pool[('mac_reduction_total', 'mean')],
    ↪color='steelblue')
        ax.set(title='MAC fuel-switching reductions by year', xlabel='year',
    ↪ylabel='Mt abated')
        ax.grid(True, alpha=0.3)

    for ax in axes:
        format_year_axis(ax)

    plt.suptitle('RQ2.4 - Decarbonisation pace within the default config',
        fontsize=13, y=1.02)
    plt.tight_layout(); plt.show()

```



RQ2 summary. Within the default config, RQ2.1–2.4 give the *descriptive* baseline of how cap, MSR, penalty, and emission outcomes move together. The sensitivity question — *would these trajectories look qualitatively different under a tighter LRF, or with the MSR off?* — is what the cross-config experiments notebook will answer.

1.8 §RQ3 — Strategic Behaviour and Aggregation

The default config crosses 4 archetypes (coal-heavy, gas-dominant, transitioner, green-leader) with 2 reward weightings (pure financial, balanced ESG). That gives us the financial-vs-environmental contrast that RQ3 asks about, *inside one config*.

Five panels:

- **3.1** — Bidding behaviour by archetype $\times$ weighting.
- **3.2** — Investment behaviour by archetype $\times$ weighting.
- **3.3** — Secondary-market role by archetype.
- **3.4** — UDBC compliance pathway by archetype.
- **3.5** — Reward decomposition (financial vs ESG vs penalty) by weighting.

1.8.1 RQ3.1 — Bidding behaviour by archetype $\times$ reward-weighting

```
[18]: # Per-agent converged-window means, then group by (archetype, weighting).
def _per_agent_table(run, episode_cols, year_cols):
    """Return list of dicts (one per agent) with mean values from converged_
    ↪window."""
    ep = tail_episodes(run['ep_df'])
    yr = tail_year_rows(run)
    rows = []
    for i in range(N_AGENTS):
        row = {'agent_idx': i, 'archetype': _agent_archetype(i), 'weighting':
        ↪_agent_weighting(i)}
        for prefix in episode_cols:
            col = f'{prefix}_A{i+1}'
            row[prefix] = float(ep[col].mean()) if col in ep.columns else np.nan
        for prefix in year_cols:
            col = f'{prefix}_A{i+1}'
            row[prefix] = float(yr[col].mean()) if col in yr.columns else np.nan
        rows.append(row)
    return rows

agent_rows = []
for k, run in runs.items():
    for r in _per_agent_table(
        run,
        episode_cols=['bid_price', 'avg_bid_mult', 'avg_bid_coverage',
        ↪'sec_buy_vol', 'sec_sell_vol', 'sec_buy_avg_px',
        ↪'sec_sell_avg_px',
        ↪'sec_buy_years', 'sec_sell_years',
        ↪'invest_cost', 'green_frac', 'delta_green',
        ↪'mean_alloc', 'penalty', 'shortfall', 'reward',
        ↪'reward_base',
        ↪'inv_onshore_share', 'inv_offshore_share',
        ↪'inv_solar_share'],
```



```

        year_cols=['mac_reduction', 'mac_cost'],
    ):
        r.update({'sweep': k[0], 'seed': k[2]})
        agent_rows.append(r)
agent_df = pd.DataFrame(agent_rows)
print(f'Per-(run, agent) rows: {len(agent_df)}')

fig, axes = plt.subplots(1, 3, figsize=(18, 4.5))
order = [(arch, w) for arch in ARCHETYPES for w in ['Financial', 'Balanced',
↳ ESG']]
labels = [f'{a}\n{w}' for a, w in order]

def _bar(ax, col, title, ylabel):
    means = []; stds = []
    for arch, w in order:
        sub = agent_df[(agent_df['archetype'] == arch) & (agent_df['weighting'] == w)]
        ↳ [col]
        means.append(sub.mean()); stds.append(sub.std())
    x = np.arange(len(order))
    colors = ['#d62728' if w == 'Financial' else '#2ca02c' for _, w in order]
    ax.bar(x, means, yerr=stds, color=colors, capsize=3, alpha=0.85)
    ax.set_xticks(x); ax.set_xticklabels(labels, fontsize=8, rotation=30,
    ↳ ha='right')
    ax.set(title=title, ylabel=ylabel); ax.grid(True, alpha=0.3, axis='y')

_bar(axes[0], 'bid_price', 'Mean Phase-1 bid price', '€/tCO2')
_bar(axes[1], 'avg_bid_mult', 'Mean qty multiplier on `estimate_need`,
↳ `multiplier')
_bar(axes[2], 'avg_bid_coverage', 'Mean coverage = alloc / need', 'ratio
↳ (clipped to 1)')
plt.suptitle('RQ3.1 - Bidding behaviour: red = pure financial reward; green =
↳ balanced ESG',
            fontsize=12, y=1.04)
plt.tight_layout(); plt.show()

```

Per-(run, agent) rows: 32

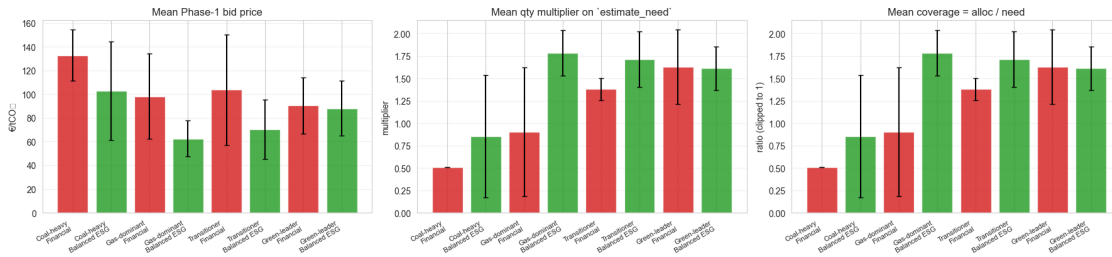
C:\Users\danie\AppData\Local\Temp\ipykernel_28804\309587121.py:55: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

plt.tight_layout(); plt.show()

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.

fig.canvas.print_figure(bytes_io, **kw)

RQ3.1 — Bidding behaviour: red = pure financial reward; green = balanced ESG



1.8.2 RQ3.2 — Investment behaviour

```
[19]: fig, axes = plt.subplots(1, 3, figsize=(18, 4.5))
_bar(axes[0], 'invest_cost', 'Mean episode invest cost', 'M€')
_bar(axes[1], 'green_frac', 'End-of-episode green fraction', 'green frac [0,1]')
_bar(axes[2], 'delta_green', 'Δ green fraction over the episode', 'Δ green frac')
plt.suptitle('RQ3.2 - Investment outcomes: red = pure financial; green = balanced ESG',
             fontsize=12, y=1.04)
plt.tight_layout(); plt.show()

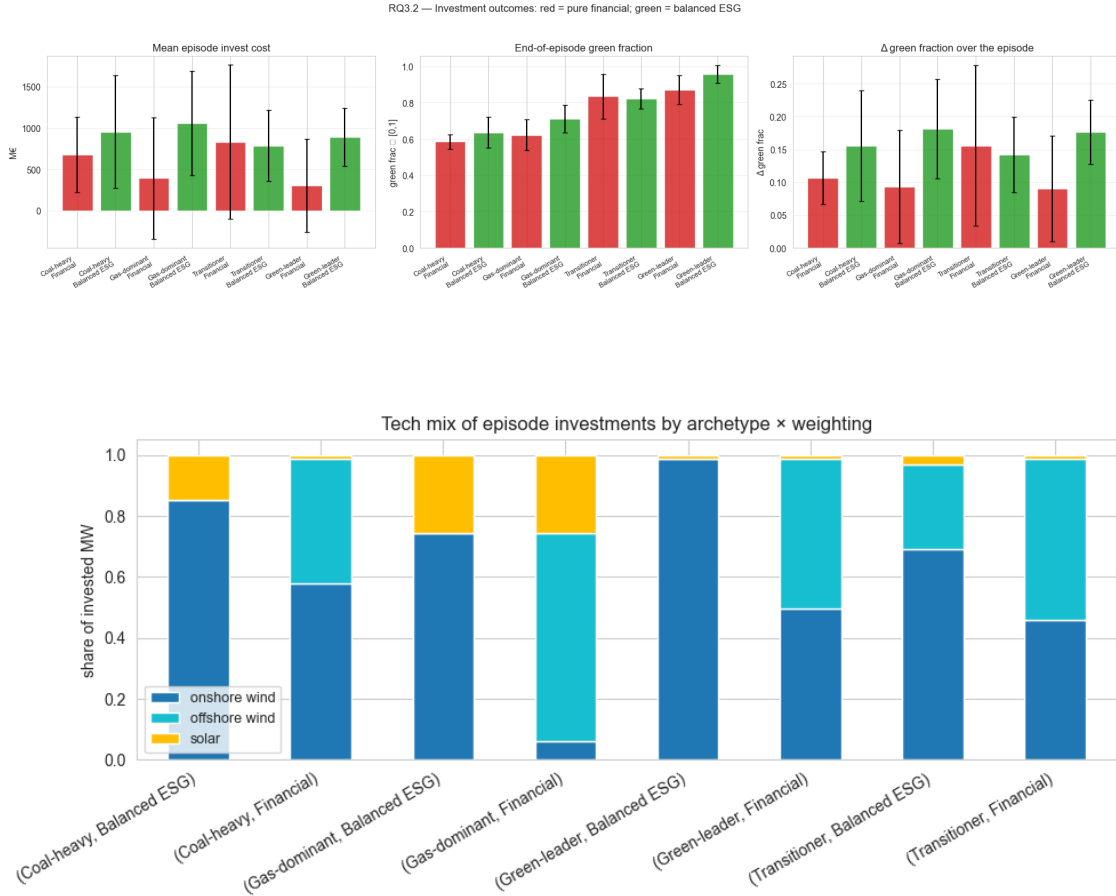
# Tech mix by archetype × weighting (stacked bar).
tech_means = (agent_df.groupby(['archetype', 'weighting'])
              [['inv_onshore_share', 'inv_offshore_share', 'inv_solar_share']].
              mean())
fig, ax = plt.subplots(figsize=(10, 4.5))
tech_means.plot(kind='bar', stacked=True, ax=ax,
                color=['#1f77b4', '#17becf', '#ffbf00'])
ax.set(title='Tech mix of episode investments by archetype × weighting',
       ylabel='share of invested MW', xlabel='')
ax.legend(['onshore wind', 'offshore wind', 'solar'], fontsize=9)
plt.xticks(rotation=30, ha='right')
plt.tight_layout(); plt.show()
```

C:\Users\danie\AppData\Local\Temp\ipykernel_28804\1602023298.py:7: UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.

```
plt.tight_layout(); plt.show()
```

C:\Users\danie\AppData\Roaming\Python\Python312\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8712 (\N{ELEMENT OF}) missing from font(s) Arial.

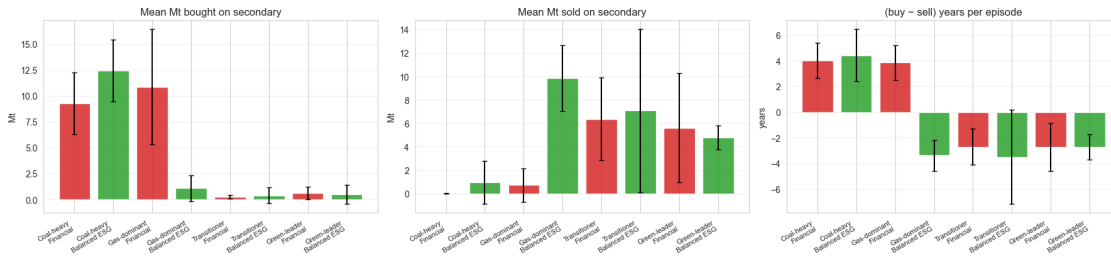
```
fig.canvas.print_figure(bytes_io, **kw)
```



1.8.3 RQ3.3 — Secondary-market role

```
[20]: fig, axes = plt.subplots(1, 3, figsize=(18, 4.5))
_bar(axes[0], 'sec_buy_vol', 'Mean Mt bought on secondary', 'Mt')
_bar(axes[1], 'sec_sell_vol', 'Mean Mt sold on secondary', 'Mt')
# Net buyer/seller years (out of 12).
def _net_buyer_years(row):
    by = row.get('sec_buy_years', np.nan); sy = row.get('sec_sell_years', np.
    ↪nan)
    if pd.isna(by) or pd.isna(sy): return np.nan
    return by - sy
agent_df['net_buyer_years'] = agent_df.apply(_net_buyer_years, axis=1)
_bar(axes[2], 'net_buyer_years', '(buy - sell) years per episode', 'years')
plt.suptitle('RQ3.3 - Who buys, who sells: secondary-market role by archetype ×_
    ↪weighting',
             fontsize=12, y=1.04)
plt.tight_layout(); plt.show()
```

RQ3.3 — Who buys, who sells: secondary-market role by archetype × weighting



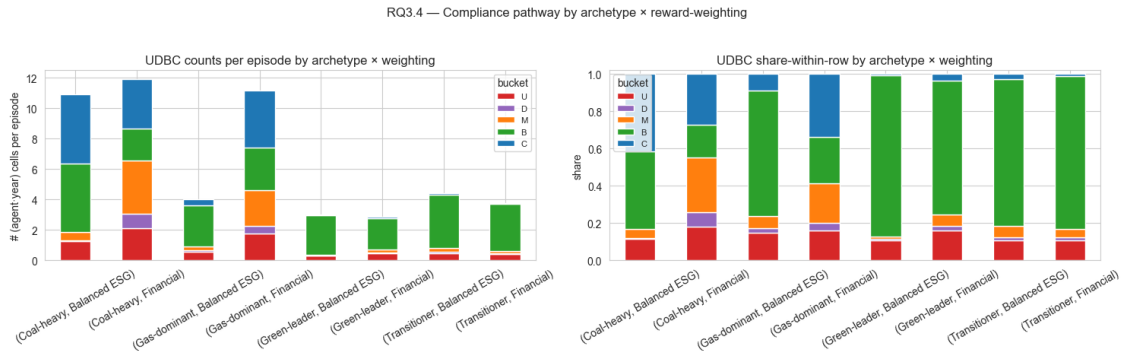
1.8.4 RQ3.4 — Compliance pathway (UDBC) by archetype × weighting

```
[21]: # Mean per-agent UDBC totals + grouped by archetype × weighting + stacked bar
      ↪ of shares.
udbc_buckets = ['U', 'D', 'M', 'B', 'C']
udbc_rows = []
for k, run in runs.items():
    ep = tail_episodes(run['ep_df'])
    for i in range(N_AGENTS):
        row = {'archetype': _agent_archetype(i), 'weighting':
            ↪ _agent_weighting(i)}
        for b in udbc_buckets:
            col = f'udbc_{b}_total_A{i+1}'
            row[b] = float(ep[col].mean()) if col in ep.columns else 0.0
        udbc_rows.append(row)
udbc_df = pd.DataFrame(udbc_rows)
udbc_grouped = udbc_df.groupby(['archetype', 'weighting'])[udbc_buckets].mean()
udbc_share = udbc_grouped.div(udbc_grouped.sum(axis=1).replace(0, np.nan),
    ↪ axis=0).fillna(0.0)

fig, axes = plt.subplots(1, 2, figsize=(15, 4.5))
udbc_grouped.plot(kind='bar', stacked=True, ax=axes[0],
    color=['#d62728', '#9467bd', '#ff7f0e', '#2ca02c', '#1f77b4'])
axes[0].set(title='UDBC counts per episode by archetype × weighting',
    ylabel='# (agent·year) cells per episode', xlabel='')
axes[0].legend(title='bucket', fontsize=8)
axes[0].tick_params(axis='x', rotation=30)

udbc_share.plot(kind='bar', stacked=True, ax=axes[1],
    color=['#d62728', '#9467bd', '#ff7f0e', '#2ca02c', '#1f77b4'])
axes[1].set(title='UDBC share-within-row by archetype × weighting',
    ylabel='share', xlabel='')
axes[1].set_ylim(0, 1.02)
axes[1].legend(title='bucket', fontsize=8)
axes[1].tick_params(axis='x', rotation=30)
```

```
plt.suptitle('RQ3.4 - Compliance pathway by archetype × reward-weighting',
            fontsize=12, y=1.04)
plt.tight_layout(); plt.show()
```



1.8.5 RQ3.5 - A1-A8 reward balance and ESG-vs-financial twin contrast

```
[55]: # RQ3.5 reward channels in the same normalized units as the reward function.
# The year CSV does not flatten reward_channels, so this reconstructs the
# ↪recurring
# channels from logged year-level quantities and leaves banking/soft gaps as
# ↪residual.

from matplotlib.colors import TwoSlopeNorm

def _numeric_column(frame, column, default=0.0):
    if column in frame.columns:
        return pd.to_numeric(frame[column], errors='coerce').astype(float)
    return pd.Series(default, index=frame.index, dtype=float)

def _agent_output_twh(agent_idx):
    output_cfg = REFERENCE_CONFIG.get('companies', {}).get('output_twh', 10.0)
    if isinstance(output_cfg, (list, tuple)):
        return float(output_cfg[agent_idx]) if agent_idx < len(output_cfg) else
# ↪float(output_cfg[-1])
        return float(output_cfg)

def _agent_reward_weights(agent_idx):
    weights = REFERENCE_CONFIG.get('companies', {}).get('reward_weights', [])
# ↪or []
    if agent_idx < len(weights):
```

```

        return float(weights[agent_idx][0]), float(weights[agent_idx][1])
    return (1.0, 0.0) if agent_idx % 2 == 0 else (0.5, 0.5)

def _agent_objective_label(agent_no):
    return 'ESG' if agent_no % 2 == 0 else 'Financial'

tech_efs = np.asarray(REFERENCE_CONFIG.get('technologies', {})).
    ↳get('emission_factors', []), dtype=float)
initial_mix = REFERENCE_CONFIG.get('companies', {}).get('initial_mix', []) or []
initial_efs = []
for agent_idx in range(N_AGENTS):
    if agent_idx < len(initial_mix) and len(tech_efs) > 0:
        initial_efs.append(float(np.dot(np.asarray(initial_mix[agent_idx],
    ↳dtype=float), tech_efs)))
    else:
        initial_efs.append(np.nan)

reward_weights = [_agent_reward_weights(agent_idx) for agent_idx in
    ↳range(N_AGENTS)]
base_penalty_rate = float(REFERENCE_CONFIG.get('penalty', {}).get('rate', 100.
    ↳0))
annual_budgets = REFERENCE_CONFIG.get('budget', {}).get('annual_budgets', [])
    ↳or [1000.0] * N_AGENTS
esg_cfg = REFERENCE_CONFIG.get('esg', {}) or {}
reward_cfg = REFERENCE_CONFIG.get('reward', {}) or {}
banking_cfg = reward_cfg.get('banking_signal', {}) or {}

esg_scale = float(esg_cfg.get('scale', 0.5))
esg_stock_weight = float(esg_cfg.get('stock_weight', 1.0))
esg_flow_weight = float(esg_cfg.get('flow_weight', 1.5))
esg_speed_early = float(esg_cfg.get('speed_coef', 0.3))
esg_speed_late = float(esg_cfg.get('speed_coef_late', esg_speed_early))
gate_threshold = float(reward_cfg.get('compliance_gate_blend_threshold', 0.90))
gate_width = max(float(reward_cfg.get('compliance_gate_blend_width', 0.30)),
    ↳1e-9)
imputed_bank_weight = float(banking_cfg.get('w_imputed', 1.0))
imputed_cap_factor = float(banking_cfg.get('imputed_cap_factor', 5.0))

reward_channel_frames = []
for run_key, run_data in runs.items():
    year_frame = tail_year_rows(run_data).copy()
    if year_frame.empty or 'year' not in year_frame.columns:
        continue

```

```

year_frame = year_frame.sort_values(['episode', 'year']).copy()
if {'episode', 'fundamental_anchor'}.issubset(year_frame.columns):
    year_frame['_anchor_next'] = year_frame.
↳groupby('episode')['fundamental_anchor'].shift(-1)
else:
    year_frame['_anchor_next'] = np.nan
    year_frame['_anchor_next'] = year_frame['_anchor_next'].fillna(year_frame.
↳get('fundamental_anchor', np.nan))

# Focus on recurring annual incentives; final-year liquidation is
↳intentionally
# left out so it does not masquerade as ESG, financial reward, or penalty.
initial_year_label = YEAR_DISPLAY_START if 'YEAR_DISPLAY_START' in
↳globals() else 1
final_year_label = YEAR_DISPLAY_END if 'YEAR_DISPLAY_END' in globals() else
↳N_YEARS
core_year_frame = year_frame[year_frame['year'] < final_year_label].copy()
if core_year_frame.empty:
    core_year_frame = year_frame.copy()

initial_year_rows = year_frame[year_frame['year'] == initial_year_label]
anchor0_real = np.nan
if {'fundamental_anchor', 'inflation_factor'}.issubset(initial_year_rows.
↳columns) and not initial_year_rows.empty:
    anchor0_real = float((initial_year_rows['fundamental_anchor'] /
↳initial_year_rows['inflation_factor']).replace(0, np.nan)).median())
    if not np.isfinite(anchor0_real) or anchor0_real <= 0:
        anchor0_real = 1.0

cap0 = float(initial_year_rows['cap'].median()) if 'cap' in
↳initial_year_rows.columns and not initial_year_rows.empty else np.nan
    if not np.isfinite(cap0) or cap0 <= 0:
        cap0 = float(core_year_frame['cap'].max()) if 'cap' in core_year_frame.
↳columns else 1.0

inflation = _numeric_column(core_year_frame, 'inflation_factor', 1.0).
↳replace(0, np.nan).fillna(1.0)
anchor = _numeric_column(core_year_frame, 'fundamental_anchor', np.nan).
↳replace(0, np.nan)
anchor_next = _numeric_column(core_year_frame, '_anchor_next', np.nan).
↳replace(0, np.nan).fillna(anchor)
clearing_price = _numeric_column(core_year_frame, 'clearing_price', 0.0)
cap = _numeric_column(core_year_frame, 'cap', cap0)
scarcity_amp = 1.0 + np.maximum(0.0, 1.0 - cap / max(cap0, 1e-9))
year_offset = (_numeric_column(core_year_frame, 'year', initial_year_label)
↳- initial_year_label).clip(lower=0.0)

```

```

year_frac = year_offset / max(N_YEARS - 1, 1)
speed_coef = esg_speed_early + year_frac * (esg_speed_late -
↳esg_speed_early)

for agent_idx in range(N_AGENTS):
    agent_no = agent_idx + 1
    reward_col = f'reward_A{agent_no}'
    base_col = f'reward_base_A{agent_no}'
    need_col = f'estimate_need_A{agent_no}'
    if base_col not in core_year_frame.columns or need_col not in
↳core_year_frame.columns:
        continue

    weight_cost, weight_green = reward_weights[agent_idx]
    annual_budget = float(annual_budgets[agent_idx]) if agent_idx <
↳len(annual_budgets) else float(annual_budgets[-1])
    budget_real = annual_budget / inflation
    soft_denom = np.maximum(budget_real, 1.0)

    need = _numeric_column(core_year_frame, need_col, np.nan).
↳clip(lower=1e-6)
    compliance_denom = np.maximum((anchor / inflation) * need, 1.0)
    nominal_compliance_denom = np.maximum(inflation * compliance_denom, 1.0)

    auction_cost = _numeric_column(core_year_frame,
↳f'auction_cost_A{agent_no}', 0.0)
    trade_cost = _numeric_column(core_year_frame,
↳f'trade_cost_A{agent_no}', 0.0)
    mac_cost = _numeric_column(core_year_frame, f'mac_cost_A{agent_no}', 0.
↳0)
    investment_cost = _numeric_column(core_year_frame,
↳f'invest_cost_A{agent_no}', 0.0)
    compliance_norm_cash = ((auction_cost + trade_cost + mac_cost) /
↳inflation) / compliance_denom

    bank_start = _numeric_column(core_year_frame,
↳f'bank_start_A{agent_no}', 0.0).clip(lower=0.0)
    allocation = _numeric_column(core_year_frame, f'alloc_A{agent_no}', 0.
↳0).clip(lower=0.0)
    trade_qty = _numeric_column(core_year_frame, f'trade_qty_A{agent_no}',
↳0.0)
    emissions = _numeric_column(core_year_frame, f'emissions_A{agent_no}',
↳0.0).clip(lower=0.0)
    carry_forward_start = _numeric_column(core_year_frame,
↳f'carry_forward_start_A{agent_no}', 0.0).clip(lower=0.0)

```



```

        carry_forward_end = _numeric_column(core_year_frame,
↪f'carry_forward_end_A{agent_no}', 0.0).clip(lower=0.0)

        net_secondary_buy = trade_qty.clip(lower=0.0)
        total_new_allowances = allocation + net_secondary_buy
        total_obligation = emissions + carry_forward_start
        bank_drawdown = np.minimum(bank_start, (total_obligation -
↪total_new_allowances).clip(lower=0.0)).clip(lower=0.0)
        imputed_bank_real = bank_drawdown * clearing_price / inflation
        imputed_bank_norm = np.minimum(imputed_bank_real, imputed_cap_factor *
↪compliance_denom) / compliance_denom

        compliance_norm = compliance_norm_cash + imputed_bank_weight *
↪imputed_bank_norm
        capital_norm = (investment_cost / inflation) / soft_denom
        cost_norm_centered = compliance_norm + capital_norm - 1.0

        bid_coverage = _numeric_column(core_year_frame,
↪f'bid_coverage_A{agent_no}', np.nan)
        coverage_auction = bid_coverage.fillna(allocation / need).clip(lower=0.
↪0, upper=1.0)
        financial_unweighted_basis = -cost_norm_centered
        financial_unweighted_proxy = financial_unweighted_basis.where(
            cost_norm_centered >= 0.0,
            coverage_auction * financial_unweighted_basis,
        )
        savings_reward = weight_cost * financial_unweighted_basis
        financial_reward = savings_reward.where(cost_norm_centered >= 0.0,
↪coverage_auction * savings_reward)

        shortfall = _numeric_column(core_year_frame, f'shortfall_A{agent_no}',
↪np.nan)
        if shortfall.isna().all():
            effective_penalty_rate = _numeric_column(core_year_frame,
↪f'effective_penalty_rate_A{agent_no}', np.nan).replace(0, np.nan)
            shortfall = _numeric_column(core_year_frame,
↪f'penalty_A{agent_no}', 0.0) / effective_penalty_rate
            shortfall = shortfall.fillna(0.0).clip(lower=0.0)
            penalty_realized_norm = shortfall * base_penalty_rate /
↪nominal_compliance_denom
            penalty_remediation_norm = carry_forward_end * anchor_next *
↪scarcity_amp / nominal_compliance_denom
            penalty_norm = penalty_realized_norm + penalty_remediation_norm

        output_twh = max(_agent_output_twh(agent_idx), 1e-9)
        current_ef = emissions / output_twh

```

```

initial_ef = initial_efs[agent_idx]
if np.isfinite(initial_ef) and initial_ef > 1e-9:
    ef_ratio = ((initial_ef - current_ef) / initial_ef).clip(lower=0.0)
else:
    ef_ratio = pd.Series(0.0, index=core_year_frame.index)
    anchor_ratio = ((anchor / inflation) / max(anchor0_real, 1.0)).
↳clip(lower=0.0)
    green_delta = _numeric_column(core_year_frame,
↳f'delta_green_A{agent_no}', 0.0).clip(lower=0.0)
    pre_compliance_holdings = (bank_start + allocation + trade_qty).
↳clip(lower=0.0)
    coverage_pre_compliance = (pre_compliance_holdings / need).clip(lower=0.
↳0, upper=1.0)
    gate_blend = ((gate_threshold - coverage_pre_compliance) / gate_width).
↳clip(lower=0.0, upper=1.0)
    compliance_gate = coverage_pre_compliance ** (1.0 + gate_blend)

    esg_stock_reward = weight_green * esg_scale * esg_stock_weight *
↳ef_ratio * compliance_gate
    esg_flow_reward = weight_green * esg_scale * esg_flow_weight * ef_ratio
↳* anchor_ratio * compliance_gate
    esg_speed_reward = weight_green * esg_scale * speed_coef * green_delta
↳* compliance_gate
    weighted_esg_reward = esg_stock_reward + esg_flow_reward +
↳esg_speed_reward

    reward_base = _numeric_column(core_year_frame, base_col, np.nan)
    reward = _numeric_column(core_year_frame, reward_col, np.nan)
    reward_shaping = _numeric_column(core_year_frame,
↳f'reward_shaping_A{agent_no}', 0.0)
    residual = reward_base - financial_reward - weighted_esg_reward +
↳penalty_norm
    objective = _agent_objective_label(agent_no)
    archetype = _agent_archetype(agent_idx)

    reward_channel_frames.append(pd.DataFrame({
        'sweep': run_key[0],
        'variant': run_key[1],
        'seed': run_key[2],
        'episode': _numeric_column(core_year_frame, 'episode', np.nan),
        'year': _numeric_column(core_year_frame, 'year', np.nan),
        'agent_no': agent_no,
        'agent': f'A{agent_no}',
        'agent_label': f'A{agent_no} {objective}\n{archetype}',
        'archetype': archetype,
        'objective': objective,

```

```

        'weighting': _agent_weighting(agent_idx),
        'reward': reward,
        'reward_base': reward_base,
        'reward_shaping': reward_shaping,
        'financial_unweighted_proxy': financial_unweighted_proxy,
        'financial_reward_proxy': financial_reward,
        'weighted_esg_proxy': weighted_esg_reward,
        'esg_stock_reward': esg_stock_reward,
        'esg_flow_reward': esg_flow_reward,
        'esg_speed_reward': esg_speed_reward,
        'penalty_norm_proxy': penalty_norm,
        'penalty_realized_norm_proxy': penalty_realized_norm,
        'penalty_remediation_norm_proxy': penalty_remediation_norm,
        'banking_soft_residual': residual,
    })

if not reward_channel_frames:
    print('No year-level reward fields available for RQ3.5.')
else:
    reward_decomp_year = pd.concat(reward_channel_frames, ignore_index=True)
    numeric_cols = reward_decomp_year.select_dtypes(include=[np.number]).columns
    reward_decomp_year[numeric_cols] = reward_decomp_year[numeric_cols].
    ↪replace([np.inf, -np.inf], np.nan)
    reward_decomp_year = reward_decomp_year.dropna(subset=['reward_base',
    ↪'financial_reward_proxy', 'penalty_norm_proxy'])

    channel_cols = [
        'reward', 'reward_base', 'financial_reward_proxy', 'weighted_esg_proxy',
        'esg_stock_reward', 'esg_flow_reward', 'esg_speed_reward',
        'penalty_norm_proxy', 'penalty_realized_norm_proxy',
    ↪'penalty_remediation_norm_proxy',
        'banking_soft_residual', 'reward_shaping',
    ]
    table_channel_cols = [
        'reward', 'reward_base', 'financial_unweighted_proxy',
    ↪'financial_reward_proxy', 'weighted_esg_proxy',
        'esg_stock_reward', 'esg_flow_reward', 'esg_speed_reward',
        'penalty_norm_proxy', 'penalty_realized_norm_proxy',
    ↪'penalty_remediation_norm_proxy',
        'banking_soft_residual', 'reward_shaping',
    ]
    agent_index_cols = ['agent_no', 'agent', 'objective', 'archetype']
    decomp_stats = (
        reward_decomp_year.groupby(agent_index_cols,
    ↪sort=True)[table_channel_cols]
        .agg(['mean', 'median', 'std'])

```

```

        .sort_index(level='agent_no')
        .round(4)
    )
    decomp_print = decomp_stats.copy()
    decomp_print.columns = [f'{column} {stat}' for column, stat in decomp_print.
↪columns]
    print('RQ3.5 A1-A8 normalized reward-channel means, medians, and standard_
↪deviations (agent-year, non-terminal years):')
    print('financial_unweighted_proxy is table-only and removes the objective_
↪weight for level-field financial comparison.')
    display(decomp_print)

    financial_level_stats = (
        reward_decomp_year.groupby(agent_index_cols,
↪sort=True)['financial_unweighted_proxy']
        .agg(['mean', 'median', 'std'])
        .sort_index(level='agent_no')
        .round(4)
        .reset_index()
    )
    level_field_by_agent = financial_level_stats.set_index('agent_no')
    level_field_rows = []
    for financial_agent_no in range(1, N_AGENTS + 1, 2):
        esg_agent_no = financial_agent_no + 1
        if financial_agent_no not in level_field_by_agent.index or esg_agent_no
↪not in level_field_by_agent.index:
            continue
        financial_row = level_field_by_agent.loc[financial_agent_no]
        esg_row = level_field_by_agent.loc[esg_agent_no]
        level_field_rows.append({
            'archetype': str(esg_row['archetype']),
            'financial_agent': str(financial_row['agent']),
            'esg_agent': str(esg_row['agent']),
            'financial_mean': financial_row['mean'],
            'esg_mean': esg_row['mean'],
            'esg_minus_financial_mean': esg_row['mean'] - financial_row['mean'],
            'financial_median': financial_row['median'],
            'esg_median': esg_row['median'],
            'esg_minus_financial_median': esg_row['median'] -
↪financial_row['median'],
            'financial_std': financial_row['std'],
            'esg_std': esg_row['std'],
        })
    financial_level_field_display = pd.DataFrame(level_field_rows)
    if not financial_level_field_display.empty:

```

```

    print('Level-field financial performance proxy by ESG/financial twin_
↪(unweighted by objective weights):')
    display(financial_level_field_display.round(4))

    plot_summary = (
        reward_decomp_year.groupby(['agent_no', 'agent', 'objective',
↪'archetype', 'agent_label'], sort=True)[channel_cols]
        .agg(['mean', 'std'])
        .sort_index(level='agent_no')
    )
    agent_means = plot_summary.xs('mean', level=1, axis=1).reset_index()
    agent_stds = plot_summary.xs('std', level=1, axis=1).reset_index()

    fig, (axis_components, axis_contrast) = plt.subplots(
        1, 2, figsize=(22, 9), constrained_layout=True,
        gridspec_kw={'width_ratios': [1.65, 1.0], 'wspace': 0.25},
    )

    y_positions = np.arange(len(agent_means))
    y_labels = agent_means['agent_label'].tolist()
    positive_left = np.zeros(len(agent_means))
    negative_left = np.zeros(len(agent_means))

    component_specs = [
        ('financial_reward_proxy', 'Financial net', '#1f77b4', 1.0),
        ('esg_stock_reward', 'ESG stock', '#2ca02c', 1.0),
        ('esg_flow_reward', 'ESG flow', '#17becf', 1.0),
        ('esg_speed_reward', 'ESG speed', '#bcbd22', 1.0),
        ('banking_soft_residual', 'Banking / soft residual', '#7f7f7f', 1.0),
        ('penalty_norm_proxy', 'Penalty cost', '#d62728', -1.0),
    ]

    legend_labels_seen = set()
    for column, label, color, sign in component_specs:
        values = (agent_means[column] * sign).to_numpy(dtype=float)
        positive_values = np.clip(values, 0.0, None)
        negative_values = np.clip(values, None, 0.0)
        legend_label = label if label not in legend_labels_seen else
↪'_nolegend_'
        if np.any(np.isfinite(positive_values) & (positive_values != 0.0)):
            axis_components.barh(
                y_positions, positive_values, left=positive_left,
                label=legend_label, color=color, alpha=0.88, height=0.72,
            )
            legend_labels_seen.add(label)
            legend_label = '_nolegend_'
            positive_left += np.nan_to_num(positive_values, nan=0.0)

```

```

        if np.any(np.isfinite(negative_values) & (negative_values != 0.0)):
            axis_components.barh(
                y_positions, negative_values, left=negative_left,
                label=legend_label, color=color, alpha=0.88, height=0.72,
            )
            legend_labels_seen.add(label)
            negative_left += np.nan_to_num(negative_values, nan=0.0)

    reward_base_mean = agent_means['reward_base'].to_numpy(dtype=float)
    axis_components.scatter(
        reward_base_mean, y_positions, marker='D', color='black', s=34,
        label='Observed reward_base mean', zorder=5,
    )

    stacked_extent = np.nanmax(np.abs(np.r_[positive_left, negative_left,
↪reward_base_mean]))
    if not np.isfinite(stacked_extent) or stacked_extent == 0:
        stacked_extent = 1.0
    label_offset = stacked_extent * 0.035
    for y_pos, value in zip(y_positions, reward_base_mean):
        if np.isfinite(value):
            ha = 'left' if value >= 0 else 'right'
            offset = label_offset if value >= 0 else -label_offset
            axis_components.text(value + offset, y_pos, f'{value:+.2f}',
↪va='center', ha=ha, fontsize=8, color='black')

    axis_components.set_xlim(-stacked_extent * 1.18, stacked_extent * 1.18)
    axis_components.axvline(0, color='black', lw=0.9)
    axis_components.set_yticks(y_positions)
    axis_components.set_yticklabels(y_labels, fontsize=9)
    axis_components.invert_yaxis()
    axis_components.set_xlabel('normalized reward contribution (right helps
↪reward; left hurts reward)')
    axis_components.set_title('Mean core reward balance by simulation agent
↪order')
    axis_components.grid(True, alpha=0.28, axis='x')
    handles, labels = axis_components.get_legend_handles_labels()
    legend_items = [(handle, label) for handle, label in zip(handles, labels)]
↪if label != '_nolegend_']
    axis_components.legend(
        [handle for handle, _ in legend_items], [label for _, label in
↪legend_items],
        fontsize=8, loc='lower center', bbox_to_anchor=(0.5, -0.18), ncol=3,
↪frameon=False,
    )

```

```

by_agent = agent_means.set_index('agent_no')
contrast_rows = []
for financial_agent_no in range(1, N_AGENTS + 1, 2):
    esg_agent_no = financial_agent_no + 1
    if financial_agent_no not in by_agent.index or esg_agent_no not in by_agent.index:
        continue
    financial_row = by_agent.loc[financial_agent_no]
    esg_row = by_agent.loc[esg_agent_no]
    contrast_rows.append({
        'archetype': str(esg_row['archetype']),
        'Reward base': esg_row['reward_base'] - financial_row['reward_base'],
        'Financial net': esg_row['financial_reward_proxy'] - esg_row['financial_reward_proxy'],
        'ESG reward': esg_row['weighted_esg_proxy'] - financial_row['weighted_esg_proxy'],
        'Penalty avoided': financial_row['penalty_norm_proxy'] - esg_row['penalty_norm_proxy'],
        'Residual': esg_row['banking_soft_residual'] - financial_row['banking_soft_residual'],
    })

contrast_df = pd.DataFrame(contrast_rows)
if contrast_df.empty:
    axis_contrast.axis('off')
else:
    contrast_cols = ['Reward base', 'Financial net', 'ESG reward', 'Penalty avoided', 'Residual']
    contrast_display = contrast_df.set_index('archetype')[contrast_cols].round(4)
    print('ESG twin minus financial twin by archetype (positive values favor the even-numbered ESG agent):')
    display(contrast_display)

    contrast_matrix = contrast_display.to_numpy(dtype=float)
    max_abs = np.nanmax(np.abs(contrast_matrix))
    if not np.isfinite(max_abs) or max_abs == 0:
        max_abs = 1.0
    norm = TwoSlopeNorm(vmin=-max_abs, vcenter=0.0, vmax=max_abs)
    heat = axis_contrast.imshow(contrast_matrix, cmap='RdYlGn', norm=norm, aspect='auto')
    axis_contrast.set_xticks(np.arange(len(contrast_cols)))
    axis_contrast.set_xticklabels(contrast_cols, rotation=35, ha='right', fontsize=9)
    axis_contrast.set_yticks(np.arange(len(contrast_display.index)))

```

```

axis_contrast.set_yticklabels(contrast_display.index, fontsize=9)
axis_contrast.set_title('Even ESG twin minus odd financial twin')
for row_idx in range(contrast_matrix.shape[0]):
    for col_idx in range(contrast_matrix.shape[1]):
        value = contrast_matrix[row_idx, col_idx]
        if np.isfinite(value):
            text_color = 'white' if abs(value) > max_abs * 0.55 else
↳ 'black'

            axis_contrast.text(col_idx, row_idx, f'{value:+.2f}',
↳ ha='center', va='center', color=text_color, fontsize=8)
        colorbar = fig.colorbar(heat, ax=axis_contrast, fraction=0.046, pad=0.
↳ 04)

        colorbar.set_label('reward-scale delta')
        axis_contrast.set_xlabel('positive = ESG twin has more favorable
↳ contribution')

    fig.suptitle(
        f'RQ3.5 - Reward decomposition on the reward scale, shaping ignored
↳ (last {TAIL_EPISODES} episodes)',
        fontsize=14,
    )
plt.show()

```

RQ3.5 A1-A8 normalized reward-channel means, medians, and standard deviations (agent-year, non-terminal years):
financial_unweighted_proxy is table-only and removes the objective weight for level-field financial comparison.

				reward mean	reward median	reward std	
				reward_base mean	reward_base median	reward_base std	
				financial_unweighted_proxy mean			
agent_no	agent	objective	archetype				
1	A1	Financial	Coal-heavy	-0.6263	-0.3340	0.9873	↳
↳				-0.6263	-0.3340	0.9873	↳
↳				0.1105			
2	A2	ESG	Coal-heavy	-0.2125	0.0751	0.7419	↳
↳				-0.2125	0.0751	0.7419	↳
↳				-0.2934			
3	A3	Financial	Gas-dominant	-0.8600	-0.4765	1.4329	↳
↳				-0.8600	-0.4765	1.4329	↳
↳				-0.1969			
4	A4	ESG	Gas-dominant	0.3649	0.3699	0.7747	↳
↳				0.3649	0.3699	0.7747	↳
↳				0.4808			
5	A5	Financial	Transitioner	0.7366	0.1917	2.0735	↳
↳				0.7366	0.1917	2.0735	↳
↳				0.7398			

6	A6	ESG	Transitioner	0.3908	0.3514	0.8909	␣
↪		0.3908	0.3514	0.8909			␣
↪		0.5163					
7	A7	Financial	Green-leader	0.7569	0.0934	2.5283	␣
↪		0.7569	0.0934	2.5283			␣
↪		0.7843					
8	A8	ESG	Green-leader	0.7444	0.4150	1.6405	␣
↪		0.7444	0.4150	1.6405			␣
↪		0.8642					

financial_unweighted_proxy median ␣

↪financial_unweighted_proxy std financial_reward_proxy mean ␣

↪financial_reward_proxy median financial_reward_proxy std \

agent_no	agent	objective	archetype				
1	A1	Financial	Coal-heavy		0.2036		␣
↪		0.3770		0.1105		0.	
↪		0.2036	0.3770				
2	A2	ESG	Coal-heavy		0.1062		␣
↪		0.9379		-0.1467		0.	
↪		0.0531	0.4689				
3	A3	Financial	Gas-dominant		0.2015		␣
↪		0.9488		-0.1969		0.	
↪		0.2015	0.9488				
4	A4	ESG	Gas-dominant		0.2753		␣
↪		1.1409		0.2404		0.	
↪		0.1376	0.5705				
5	A5	Financial	Transitioner		0.1660		␣
↪		2.0208		0.7398		0.	
↪		0.1660	2.0208				
6	A6	ESG	Transitioner		0.2409		␣
↪		1.5208		0.2581		0.	
↪		0.1205	0.7604				
7	A7	Financial	Green-leader		0.0882		␣
↪		2.4646		0.7843		0.	
↪		0.0882	2.4646				
8	A8	ESG	Green-leader		0.0809		␣
↪		3.3435		0.4321		0.	
↪		0.0404	1.6717				

weighted_esg_proxy mean ␣

↪weighted_esg_proxy median weighted_esg_proxy std esg_stock_reward mean ␣

↪esg_stock_reward median esg_stock_reward std \

agent_no	agent	objective	archetype				
1	A1	Financial	Coal-heavy	0.0000			␣
↪		0.0000	0.0000	0.0000		0.	
↪		0.0000	0.0000				

2	A2	ESG	Coal-heavy	0.1777		
↪	0.1627		0.1218	0.0627	0.	
↪	0.0593		0.0411			
3	A3	Financial	Gas-dominant	0.0000		
↪	0.0000		0.0000	0.0000	0.	
↪	0.0000		0.0000			
4	A4	ESG	Gas-dominant	0.2165		
↪	0.2169		0.1274	0.0765	0.	
↪	0.0789		0.0428			
5	A5	Financial	Transitioner	0.0000		
↪	0.0000		0.0000	0.0000	0.	
↪	0.0000		0.0000			
6	A6	ESG	Transitioner	0.2548		
↪	0.2600		0.1384	0.0908	0.	
↪	0.0941		0.0470			
7	A7	Financial	Green-leader	0.0000		
↪	0.0000		0.0000	0.0000	0.	
↪	0.0000		0.0000			
8	A8	ESG	Green-leader	0.3424		
↪	0.3404		0.1735	0.1216	0.	
↪	0.1232		0.0581			

				esg_flow_reward mean	esg_flow_reward	
				median	esg_flow_reward std	
				esg_speed_reward mean	esg_speed_reward median	
				esg_speed_reward std	\	
agent_no	agent	objective	archetype			
1	A1	Financial	Coal-heavy	0.0000	0.	
↪	0.0000		0.0000	0.0000	0.0000	
↪			0.0000			
2	A2	ESG	Coal-heavy	0.1140	0.	
↪	0.1023		0.0807	0.0009	0.0007	
↪			0.0011			
3	A3	Financial	Gas-dominant	0.0000	0.	
↪	0.0000		0.0000	0.0000	0.0000	
↪			0.0000			
4	A4	ESG	Gas-dominant	0.1388	0.	
↪	0.1366		0.0848	0.0011	0.0009	
↪			0.0012			
5	A5	Financial	Transitioner	0.0000	0.	
↪	0.0000		0.0000	0.0000	0.0000	
↪			0.0000			
6	A6	ESG	Transitioner	0.1631	0.	
↪	0.1642		0.0918	0.0009	0.0005	
↪			0.0012			
7	A7	Financial	Green-leader	0.0000	0.	
↪	0.0000		0.0000	0.0000	0.0000	
↪			0.0000			

8	A8	ESG	Green-leader	0.2197	0.
↵2156			0.1160	0.0011	0.0008
↵			0.0012		

penalty_norm_proxy mean

↵penalty_norm_proxy median	penalty_norm_proxy std	penalty_realized_norm_proxy
↵mean	penalty_realized_norm_proxy median	\

agent_no agent objective archetype

1	A1	Financial	Coal-heavy	0.5664	
↵	0.1395		0.6386	0.3075	
↵			0.079		
2	A2	ESG	Coal-heavy	0.1077	
↵	0.0000		0.3370	0.0593	
↵			0.000		
3	A3	Financial	Gas-dominant	0.4078	
↵	0.0000		0.6161	0.2255	
↵			0.000		
4	A4	ESG	Gas-dominant	0.0737	
↵	0.0000		0.3416	0.0369	
↵			0.000		
5	A5	Financial	Transitioner	0.0459	
↵	0.0000		0.2721	0.0228	
↵			0.000		
6	A6	ESG	Transitioner	0.1006	
↵	0.0000		0.4727	0.0510	
↵			0.000		
7	A7	Financial	Green-leader	0.0554	
↵	0.0000		0.3027	0.0283	
↵			0.000		
8	A8	ESG	Green-leader	0.0305	
↵	0.0000		0.2496	0.0148	
↵			0.000		

penalty_realized_norm_proxy std

↵penalty_remediation_norm_proxy mean	penalty_remediation_norm_proxy median
--------------------------------------	---------------------------------------

↵penalty_remediation_norm_proxy std	\
-------------------------------------	---

agent_no agent objective archetype

1	A1	Financial	Coal-heavy	0.3520	
↵			0.2589	0.0563	
↵			0.2955		
2	A2	ESG	Coal-heavy	0.1885	
↵			0.0485	0.0000	
↵			0.1526		
3	A3	Financial	Gas-dominant	0.3459	
↵			0.1823	0.0000	
↵			0.2776		

4	A4	ESG	Gas-dominant	0.1689	↳
↳			0.0368	0.0000	↳
↳			0.1741		
5	A5	Financial	Transitioner	0.1355	↳
↳			0.0230	0.0000	↳
↳			0.1380		
6	A6	ESG	Transitioner	0.2391	↳
↳			0.0495	0.0000	↳
↳			0.2367		
7	A7	Financial	Green-leader	0.1545	↳
↳			0.0271	0.0000	↳
↳			0.1506		
8	A8	ESG	Green-leader	0.1205	↳
↳			0.0157	0.0000	↳
↳			0.1301		

banking_soft_residual mean ↳

↳banking_soft_residual median banking_soft_residual std reward_shaping mean ↳

↳reward_shaping median reward_shaping std

agent_no agent objective archetype

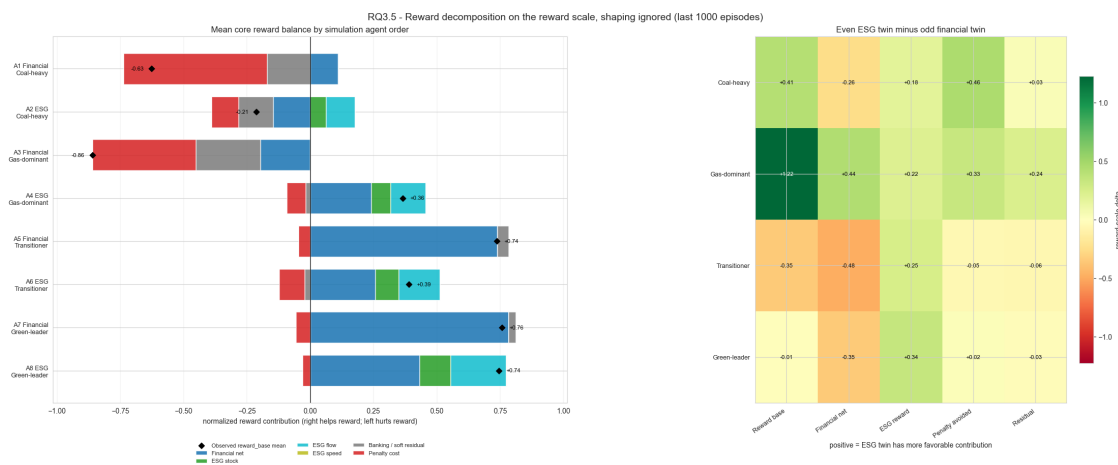
1	A1	Financial	Coal-heavy	-0.1704	↳
↳			0.0157	0.7161	0.0
↳	0.0		0.0		↳
2	A2	ESG	Coal-heavy	-0.1357	↳
↳			-0.0388	0.3651	0.0
↳	0.0		0.0		↳
3	A3	Financial	Gas-dominant	-0.2553	↳
↳			0.0102	0.8292	0.0
↳	0.0		0.0		↳
4	A4	ESG	Gas-dominant	-0.0184	↳
↳			0.0260	0.2559	0.0
↳	0.0		0.0		↳
5	A5	Financial	Transitioner	0.0427	↳
↳			0.0558	0.2543	0.0
↳	0.0		0.0		↳
6	A6	ESG	Transitioner	-0.0216	↳
↳			0.0209	0.4026	0.0
↳	0.0		0.0		↳
7	A7	Financial	Green-leader	0.0280	↳
↳			0.0617	0.4118	0.0
↳	0.0		0.0		↳
8	A8	ESG	Green-leader	0.0004	↳
↳			0.0405	0.6465	0.0
↳	0.0		0.0		↳

Level-field financial performance proxy by ESG/financial twin (unweighted by objective weights):

	archetype	financial_agent	esg_agent	financial_mean	esg_mean	
		esg_minus_financial_mean	financial_median	esg_median		
		esg_minus_financial_median	financial_std	esg_std		
0	Coal-heavy	A1	A2	0.1105	-0.2934	
		-0.4039	0.2036	0.1062	-0.0974	
		0.3770	0.9379			
1	Gas-dominant	A3	A4	-0.1969	0.4808	
		0.6777	0.2015	0.2753	0.0738	
		0.9488	1.1409			
2	Transitioner	A5	A6	0.7398	0.5163	
		-0.2235	0.1660	0.2409	0.0749	
		2.0208	1.5208			
3	Green-leader	A7	A8	0.7843	0.8642	
		0.0799	0.0882	0.0809	-0.0073	
		2.4646	3.3435			

ESG twin minus financial twin by archetype (positive values favor the even-numbered ESG agent):

	Reward base	Financial net	ESG reward	Penalty avoided	Residual
archetype					
Coal-heavy	0.4138	-0.2572	0.1777	0.4587	0.0347
Gas-dominant	1.2249	0.4373	0.2165	0.3342	0.2369
Transitioner	-0.3458	-0.4816	0.2548	-0.0547	-0.0644
Green-leader	-0.0125	-0.3522	0.3424	0.0249	-0.0276



Aggregation note. The system-level outcomes that RQ3 asks about — total clearing price, system green fraction, total penalty volume — are exactly what RQ1.2 / RQ1.3 / RQ2.1 already report. The missing piece RQ3 adds is *attribution*: which archetypes drive those aggregates? The §RQ3 panels above answer that — and in particular, panels 3.1 / 3.4 / 3.5 contrast the pure-financial agent (red) against its balanced-ESG twin (green) within each archetype, which is the direct test of “how do financial vs environmental objectives shape strategy”.

1.8.6 RQ4 — Compliance pressure, debt cascade, and credit stress

These panels read from the v8.6.1 expanded year-log columns. Cells silently no-op on logs from older versions (every column access is guarded by `if col in df.columns`). The four sub-questions are:

- **RQ4.1** — Does coverage gap ($\max(0, \text{need} - \text{alloc})$ Mt) shrink as agents learn to size their auction bids? Is the residual gap eaten by the secondary market or does it propagate as carry-forward debt?
- **RQ4.2** — Auction concentration: does the system stabilise around a diffuse allocation, or does a small subset of bidders capture most volume (HHI / max-agent-share)?
- **RQ4.3** — Secondary-market liquidity: how many agents participate on each side, and what fraction of intent gets executed?
- **RQ4.4** — Credit stress in the converged window: peak emergency-loan balance and peak carry-forward debt by archetype $\times$ reward-weighting.

1.8.7 RQ4.1 — Coverage gap & carry-forward trajectory

```
[64]: # RQ4.1 - Per-year coverage gap and end-of-year carry-forward (debt cascade).
# Cross-seed mean +  $\pm 1$  std fan, computed from the year_log only - works on
# any v8.6.1+ run. Earlier logs silently produce empty figures.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

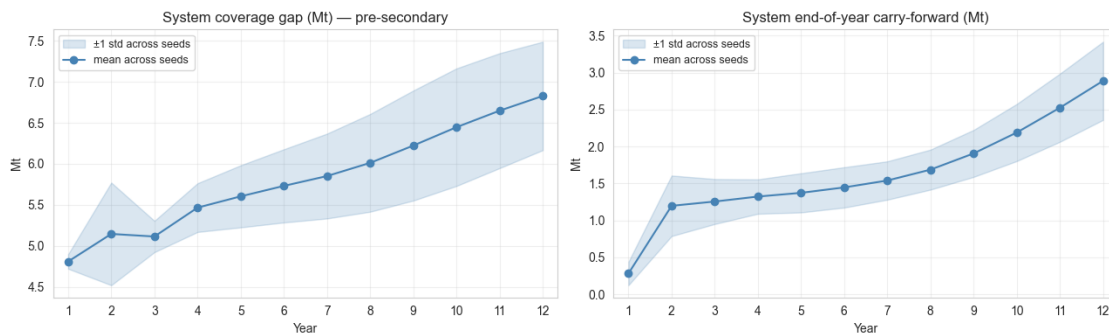
def _stack_year_metric(prefix, max_idx=N_TOTAL, agg="sum"):
    """Return wide DataFrame indexed by year with one column per run, where
    each value is the cross-agent (sum/mean) of `{prefix}_A{i}` for that
    year - averaged across episodes within the run."""
    out = {}
    for k, run in runs.items():
        yr = run["yr_df"]
        if "year" not in yr.columns:
            continue
        cols = [f"{prefix}_A{i+1}" for i in range(max_idx)
                if f"{prefix}_A{i+1}" in yr.columns]
        if not cols:
            continue
        # Per-year mean across episodes of the cross-agent agg.
        if agg == "sum":
            per_year = yr.groupby("year")[cols].mean().sum(axis=1)
        else:
            per_year = yr.groupby("year")[cols].mean().mean(axis=1)
        out[k] = per_year
    return pd.concat(out, axis=1).sort_index() if out else pd.DataFrame()
```

```

cov_gap_df = _stack_year_metric("coverage_gap")           # system-sum (Mt)
cf_end_df  = _stack_year_metric("carry_forward_end")      # system-sum (Mt)
cf_start_df = _stack_year_metric("carry_forward_start")  # system-sum (Mt)

if cov_gap_df.empty and cf_end_df.empty:
    print("[RQ4.1] coverage_gap_A* / carry_forward*_A* columns missing - pre-8.
    ↪6.1 log.")
else:
    fig, axes = plt.subplots(1, 2, figsize=(13, 4))
    for ax, df, ttl in [
        (axes[0], cov_gap_df, "System coverage gap (Mt) - pre-secondary"),
        (axes[1], cf_end_df, "System end-of-year carry-forward (Mt)"),
    ]:
        if df.empty:
            ax.set_title(ttl + "\n(missing)"); continue
        mean = df.mean(axis=1)
        std = df.std(axis=1).fillna(0.0)
        ax.fill_between(mean.index, (mean - std).values, (mean + std).values,
            alpha=0.2, color="steelblue", label="±1 std across_
            ↪seeds")
        ax.plot(mean.index, mean.values, "o-", color="steelblue", label="mean_
            ↪across seeds")
        ax.set_xlabel("Year"); ax.set_ylabel("Mt"); ax.set_title(ttl)
        ax.grid(alpha=0.3); ax.legend(loc="best", fontsize=9)
    for ax in axes:
        format_year_axis(ax)
    plt.tight_layout(); plt.show()

```



1.8.8 RQ4.2 — Auction concentration (HHI, max-agent share)

```

[65]: # RQ4.2 - Auction concentration trajectory. Reads `auction_hhi` and
      # `auction_max_agent_share` (year-level scalars added in v8.6.1).
      import numpy as np
      import pandas as pd

```

```

import matplotlib.pyplot as plt

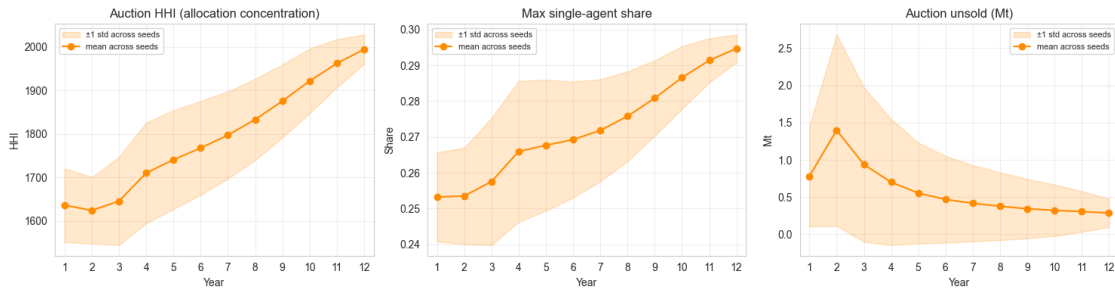
def _stack_year_scalar(col):
    out = {}
    for k, run in runs.items():
        yr = run["yr_df"]
        if "year" not in yr.columns or col not in yr.columns:
            continue
        out[k] = yr.groupby("year")[col].mean()
    return pd.concat(out, axis=1).sort_index() if out else pd.DataFrame()

hhi_df = _stack_year_scalar("auction_hhi")
mas_df = _stack_year_scalar("auction_max_agent_share")
fail_df = _stack_year_scalar("auction_failed")
unsold_df = _stack_year_scalar("auction_unsold")

if hhi_df.empty and mas_df.empty:
    print("[RQ4.2] auction_hhi / auction_max_agent_share missing - pre-8.6.1_
↳log.")
else:
    fig, axes = plt.subplots(1, 3, figsize=(15, 4))
    for ax, df, ttl, ylabel in [
        (axes[0], hhi_df, "Auction HHI (allocation concentration)", "HHI"),
        (axes[1], mas_df, "Max single-agent share", "Share"),
        (axes[2], unsold_df, "Auction unsold (Mt)", "Mt"),
    ]:
        if df.empty:
            ax.set_title(ttl + "\n(missing)"); continue
        mean = df.mean(axis=1)
        std = df.std(axis=1).fillna(0.0)
        ax.fill_between(mean.index, (mean - std).values, (mean + std).values,
            alpha=0.2, color="darkorange", label="±1 std across_
↳seeds")
        ax.plot(mean.index, mean.values, "o-", color="darkorange", label="mean_
↳across seeds")
        ax.set_xlabel("Year"); ax.set_ylabel(ylabel); ax.set_title(ttl)
        ax.grid(alpha=0.3); ax.legend(loc="best", fontsize=8)
    for ax in axes:
        format_year_axis(ax)
    plt.tight_layout(); plt.show()

    if not fail_df.empty:
        n_failures = int(np.nansum(fail_df.values))
        print(f"Total auction-failure (year, seed) cells across loaded runs:_
↳{n_failures}")

```

Total auction-failure (year, seed) cells across loaded runs: 0

1.8.9 RQ4.3 — Secondary-market liquidity (intent vs execution)

```
[66]: # RQ4.3 - Secondary-market liquidity: participation, volume, and match rate.
# The match-rate panel mirrors the trainer's definition: the share of cases
# where at least one secondary trade executed, aggregated by year.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

EPS_TRADE = 1e-6

def _median_by_year(rows):
    if not rows:
        return pd.Series(dtype=float)
    return pd.concat(rows, axis=1).median(axis=1).sort_index()

def _stack_year_column(column):
    rows = []
    for run_key, run in runs.items():
        year_df = run.get("yr_df", pd.DataFrame())
        if year_df is None or "year" not in year_df.columns or column not in year_df.columns:
            continue
        values = pd.to_numeric(year_df[column], errors="coerce")
        rows.append(values.groupby(year_df["year"]).mean())
    return _median_by_year(rows)

count_columns = {
    "buyers_intent": "secondary_n_buyers_intent",
    "sellers_intent": "secondary_n_sellers_intent",
    "buyers_executed": "secondary_n_buyers_executed",
```

```

        "sellers_executed": "secondary_n_sellers_executed",
    }
    agent_count_series = {
        label: series
        for label, column in count_columns.items()
        if not (series := _stack_year_column(column)).empty
    }

    volume_rows = {
        "buy_intent_volume": [],
        "sell_intent_volume": [],
        "buy_executed_volume": [],
        "sell_executed_volume": [],
    }

    trade_active_rows = []

    for run_key, run in runs.items():
        year_df = run.get("yr_df", pd.DataFrame())
        if year_df is None or "year" not in year_df.columns:
            continue

        intent_columns = year_df.filter(regex=r"^sec_qty_action_A\d+$").columns
        trade_columns = year_df.filter(regex=r"^trade_qty_A\d+$").columns
        derived = {"year": year_df["year"]}

        if len(intent_columns):
            intent = year_df[intent_columns].apply(pd.to_numeric, errors="coerce").
            ↪fillna(0.0)
            derived["buy_intent_volume"] = intent.clip(lower=0.0).sum(axis=1)
            derived["sell_intent_volume"] = (-intent.clip(upper=0.0)).sum(axis=1)

        if len(trade_columns):
            traded = year_df[trade_columns].apply(pd.to_numeric, errors="coerce").
            ↪fillna(0.0)
            derived["buy_executed_volume"] = traded.clip(lower=0.0).sum(axis=1)
            derived["sell_executed_volume"] = (-traded.clip(upper=0.0)).sum(axis=1)
            trade_active_rows.append((traded.abs().sum(axis=1) > EPS_TRADE).
            ↪groupby(year_df["year"]).mean())
            elif {"secondary_n_buyers_executed", "secondary_n_sellers_executed"}.
            ↪issubset(year_df.columns):
                buyers_executed = pd.to_numeric(year_df["secondary_n_buyers_executed"],
            ↪errors="coerce").fillna(0.0)
                sellers_executed = pd.
            ↪to_numeric(year_df["secondary_n_sellers_executed"], errors="coerce").
            ↪fillna(0.0)

```

```

        trade_active_rows.append(((buyers_executed + sellers_executed) > 0.0).
↳groupby(year_df["year"]).mean())

    derived_df = pd.DataFrame(derived)
    for label in volume_rows:
        if label in derived_df.columns:
            volume_rows[label].append(derived_df.groupby("year")[label].mean())

volume_series = {
    label: _median_by_year(rows)
    for label, rows in volume_rows.items()
    if rows
}
trade_active_rate = _median_by_year(trade_active_rows) * 100.0

if not agent_count_series and not volume_series and trade_active_rate.empty:
    print("[RQ4.3] secondary-market intent/executed columns missing - cannot_
↳plot liquidity diagnostics.")
else:
    fig, axes = plt.subplots(1, 2, figsize=(18, 6))

    ax = axes[0]
    if agent_count_series:
        for label, series in agent_count_series.items():
            line_style = "-" if "executed" in label else "--"
            ax.plot(series.index, series.values, line_style + "o", label=label.
↳replace("_", " "))
            ax.legend(fontsize=8)
        else:
            ax.text(0.5, 0.5, "missing count columns", ha="center", va="center",
↳transform=ax.transAxes)
        ax.set_xlabel("Year"); ax.set_ylabel("Number of agents")
        ax.set_title("Participation - intent vs executed")
        ax.grid(alpha=0.3)

    ax = axes[1]
    if volume_series:
        for label, series in volume_series.items():
            line_style = "-" if "executed" in label else "--"
            ax.plot(series.index, series.values, line_style + "o", label=label.
↳replace("_", " "))
            ax.legend(fontsize=8)
        else:
            ax.text(0.5, 0.5, "missing volume columns", ha="center", va="center",
↳transform=ax.transAxes)
        ax.set_xlabel("Year"); ax.set_ylabel("Trade volume (Mt)")

```

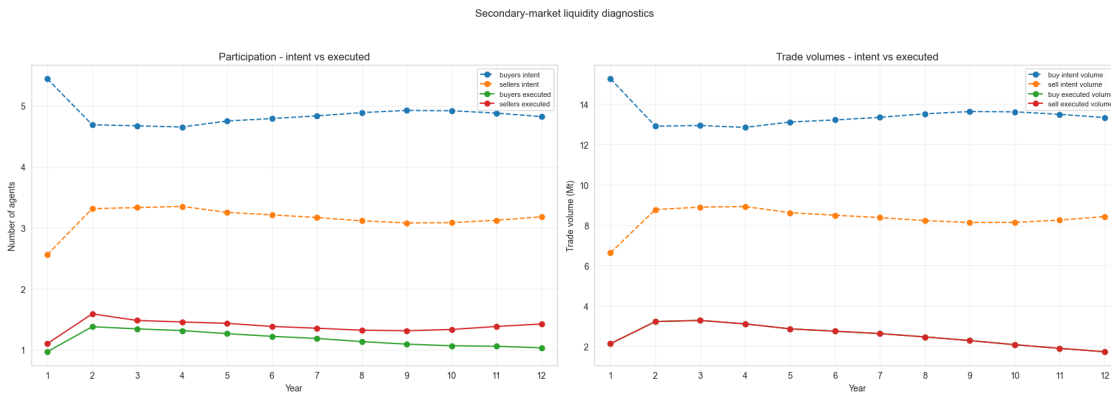
```

ax.set_title("Trade volumes - intent vs executed")
ax.grid(alpha=0.3)

for ax in axes:
    format_year_axis(ax)

fig.suptitle("Secondary-market liquidity diagnostics", y=1.04)
plt.tight_layout(); plt.show()

```



```

[67]: # RQ4.3b - Why is the real secondary-market fill rate low?
# Uses columns loaded by the main loader; no per-cell reloading.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

EPS_TRADE = 1e-6

def _row_weighted_mean(value_df, weight_df):
    weights = weight_df.where(weight_df > EPS_TRADE, 0.0)
    weight_sum = weights.sum(axis=1)
    weighted_sum = value_df.where(weights > 0.0).mul(weights).sum(axis=1,
↪min_count=1)
    return weighted_sum.div(weight_sum.where(weight_sum > EPS_TRADE))

def _summary_row(frame, metric, label, unit):
    values = pd.to_numeric(frame[metric], errors="coerce").replace([np.inf, -np.
↪inf], np.nan).dropna()
    return {
        "Metric": label,
        "Unit": unit,
        "Mean": values.mean(),
    }

```

```

        "Median": values.median(),
        "Std": values.std(),
    }

def _display_table(frame, numeric_columns=None, max_colwidth=180):
    numeric_columns = numeric_columns or []
    formatters = {col: "{:.2f}" for col in numeric_columns if col in frame.
    columns}
    if "Year" in frame.columns:
        formatters["Year"] = "{:.0f}"
    with pd.option_context("display.max_colwidth", max_colwidth):
        display(frame.style.format(formatters))

orderbook_parts = []
missing_price_runs = []
for run_key, run in runs.items():
    year_tail = tail_year_rows(run).copy()
    if year_tail.empty or "year" not in year_tail.columns:
        continue

    qty_cols = agent_cols(year_tail, "sec_qty_action")
    price_cols = agent_cols(year_tail, "sec_price_mult")
    trade_cols = agent_cols(year_tail, "trade_qty")
    if not qty_cols or not trade_cols:
        continue
    if not price_cols:
        missing_price_runs.append(run_key)
        continue

    n_cols = min(len(qty_cols), len(price_cols), len(trade_cols))
    qty = year_tail[qty_cols[:n_cols]].apply(pd.to_numeric, errors="coerce").
    fillna(0.0)
    price = year_tail[price_cols[:n_cols]].apply(pd.to_numeric, errors="coerce")
    trade = year_tail[trade_cols[:n_cols]].apply(pd.to_numeric,
    errors="coerce").fillna(0.0)
    qty.columns = price.columns = trade.columns = range(n_cols)

    buy_qty = qty.clip(lower=0.0)
    raw_sell_qty = (-qty.clip(upper=0.0)).clip(lower=0.0)

    alloc_cols = agent_cols(year_tail, "alloc")[:n_cols]
    bank_cols = agent_cols(year_tail, "bank_start")[:n_cols]
    emissions_cols = agent_cols(year_tail, "emissions")[:n_cols]
    carry_cols = agent_cols(year_tail, "carry_forward_start")[:n_cols]

```

```

    can_clip_sells = len(alloc_cols) == len(bank_cols) == len(emissions_cols)
    ⇨ == n_cols

    if can_clip_sells:
        alloc = year_tail[alloc_cols].apply(pd.to_numeric, errors="coerce").
    ⇨ fillna(0.0)
        bank = year_tail[bank_cols].apply(pd.to_numeric, errors="coerce").
    ⇨ fillna(0.0).clip(lower=0.0)
        emissions = year_tail[emissions_cols].apply(pd.to_numeric,
    ⇨ errors="coerce").fillna(0.0)
        alloc.columns = bank.columns = emissions.columns = range(n_cols)
        if len(carry_cols) == n_cols:
            carry = year_tail[carry_cols].apply(pd.to_numeric, errors="coerce").
    ⇨ fillna(0.0)
            carry.columns = range(n_cols)
        else:
            carry = pd.DataFrame(0.0, index=year_tail.index,
    ⇨ columns=range(n_cols))
            max_sell = (alloc + bank - emissions - carry).clip(lower=0.0)
            feasible_sell_qty = pd.DataFrame(
                np.minimum(raw_sell_qty.to_numpy(), max_sell.to_numpy()),
                index=year_tail.index,
                columns=range(n_cols),
            )
        else:
            feasible_sell_qty = raw_sell_qty.copy()

    buy_intent = buy_qty.sum(axis=1)
    raw_sell_intent = raw_sell_qty.sum(axis=1)
    feasible_sell = feasible_sell_qty.sum(axis=1)
    executed_buy = trade.clip(lower=0.0).sum(axis=1)
    executed_sell = (-trade.clip(upper=0.0)).sum(axis=1)
    executed_once = pd.concat([executed_buy, executed_sell], axis=1).max(axis=1)
    feasible_overlap = pd.concat([buy_intent, feasible_sell], axis=1).
    ⇨ min(axis=1)

    spread_fraction = float((run.get("config") or {}).get("trading", {}).
    ⇨ get("spread_tolerance", 0.0))
    clearing = pd.to_numeric(year_tail.get("clearing_price", 0.0),
    ⇨ errors="coerce").fillna(0.0)
    spread_tolerance = spread_fraction * clearing.clip(lower=1.0)

    buyer_limit = _row_weighted_mean(price.where(buy_qty > EPS_TRADE), buy_qty)
    raw_seller_ask = _row_weighted_mean(price.where(raw_sell_qty > EPS_TRADE),
    ⇨ raw_sell_qty)

```

```

feasible_seller_ask = _row_weighted_mean(price.where(feasible_sell_qty >
↳EPS_TRADE), feasible_sell_qty)
best_buy = price.where(buy_qty > EPS_TRADE).max(axis=1)
best_ask = price.where(feasible_sell_qty > EPS_TRADE).min(axis=1)
gap_after_tolerance = best_ask - best_buy - spread_tolerance

orderbook = pd.DataFrame({
    "run": str(run_key),
    "year": year_tail["year"].to_numpy(),
    "buy_intent_Mt": buy_intent,
    "raw_sell_intent_Mt": raw_sell_intent,
    "feasible_sell_Mt": feasible_sell,
    "executed_Mt": executed_once,
    "clipped_sell_Mt": (raw_sell_intent - feasible_sell).clip(lower=0.0),
    "feasible_overlap_Mt": feasible_overlap,
    "unfilled_feasible_overlap_Mt": (feasible_overlap - executed_once).
↳clip(lower=0.0),
    "buy_demand_above_feasible_sell_Mt": (buy_intent - feasible_sell).
↳clip(lower=0.0),
    "buyer_limit_wavg_EURt": buyer_limit,
    "raw_seller_ask_wavg_EURt": raw_seller_ask,
    "feasible_seller_ask_wavg_EURt": feasible_seller_ask,
    "best_buy_EURt": best_buy,
    "best_feasible_ask_EURt": best_ask,
    "gap_after_tolerance_EURt": gap_after_tolerance,
    "best_prices_cross_pct": (gap_after_tolerance <= 0.0).astype(float) *
↳100.0,
    "real_fill_pct": 100.0 * (executed_buy + executed_sell).div((buy_intent
↳+ raw_sell_intent).clip(lower=EPS_TRADE)),
    "feasible_overlap_fill_pct": 100.0 * executed_once.div(feasible_overlap.
↳clip(lower=EPS_TRADE)),
})

conditions = [
    (orderbook["buy_intent_Mt"] <= EPS_TRADE) &
↳(orderbook["raw_sell_intent_Mt"] <= EPS_TRADE),
    orderbook["buy_intent_Mt"] <= EPS_TRADE,
    orderbook["raw_sell_intent_Mt"] <= EPS_TRADE,
    orderbook["feasible_sell_Mt"] <= EPS_TRADE,
    orderbook["gap_after_tolerance_EURt"] > 0.0,
    orderbook["unfilled_feasible_overlap_Mt"] > EPS_TRADE,
]
labels = [
    "no submitted orders",
    "no buy intent",
    "no sell intent",

```

```

        "sell intent infeasible",
        "best prices do not cross",
        "only marginal prices cross",
    ]
    orderbook["primary_bottleneck"] = np.select(conditions, labels,
↪default="side-limited/mostly filled")
    orderbook_parts.append(orderbook)

orderbook_df = pd.concat(orderbook_parts, ignore_index=True) if orderbook_parts
↪else pd.DataFrame()

if orderbook_df.empty:
    if missing_price_runs:
        print("[RQ4.3b] Secondary price columns are not loaded. Run the main
↪loader cell, then rerun this cell.")
    else:
        print("[RQ4.3b] Missing secondary order columns; cannot diagnose real
↪match rate.")
else:
    if missing_price_runs:
        print(f"[RQ4.3b] Skipped {len(missing_price_runs)} runs without loaded
↪sec_price_mult_A* columns.")
        print("RQ4.3b secondary-market order-book diagnosis")
        print(f"Sample: {len(orderbook_df):,} (run, episode, year) rows across
↪{orderbook_df['run'].nunique()} runs")

    metric_specs = [
        ("real_fill_pct", "Real fill rate", "%", "Executed buy+sell quantity
↪divided by submitted buy+sell quantity."),
        ("feasible_overlap_fill_pct", "Feasible-overlap fill rate", "%",
↪"Executed one-sided volume divided by min(buy intent, feasible sell volume).
↪"),
        ("buy_intent_Mt", "Buy intent volume", "Mt", "Total positive
↪sec_qty_action submitted by buyers."),
        ("raw_sell_intent_Mt", "Raw sell intent volume", "Mt", "Total absolute
↪negative sec_qty_action submitted by sellers before clipping."),
        ("feasible_sell_Mt", "Feasible sell volume", "Mt", "Sell intent
↪remaining after no-short/compliance feasibility clipping."),
        ("executed_Mt", "Executed volume", "Mt", "One-sided matched volume;
↪buyer and seller legs should be equal apart from rounding."),
        ("clipped_sell_Mt", "Clipped sell intent", "Mt", "Submitted sell volume
↪removed by feasibility constraints."),
        ("unfilled_feasible_overlap_Mt", "Unfilled feasible overlap", "Mt",
↪"min(buy intent, feasible sell) minus executed volume."),
        ("buy_demand_above_feasible_sell_Mt", "Buy demand above feasible sell
↪supply", "Mt", "Buy intent exceeding feasible sell quantity."),

```



```

        ("best_prices_cross_pct", "Best-price crossing rate", "%", "Share of
↪rows where highest buyer limit reaches lowest feasible ask after tolerance.
↪"),
        ("buyer_limit_wavg_EURt", "Weighted buy limit price", "EUR/t",
↪"Quantity-weighted average limit price submitted by buyers."),
        ("feasible_seller_ask_wavg_EURt", "Weighted feasible seller ask", "EUR/
↪t", "Quantity-weighted average ask among sellers with feasible volume."),
        ("best_buy_EURt", "Best buy price", "EUR/t", "Highest buyer limit price
↪in the row."),
        ("best_feasible_ask_EURt", "Best feasible ask", "EUR/t", "Lowest ask
↪among sellers with feasible volume."),
        ("gap_after_tolerance_EURt", "Best gap after tolerance", "EUR/t",
↪"Lowest feasible ask minus highest bid minus spread tolerance; positive
↪means no crossing."),
    ]
    metric_definitions = pd.DataFrame([
        {"Metric": label, "Unit": unit, "Definition": definition}
        for _, label, unit, definition in metric_specs
    ])
    summary_stats = pd.DataFrame([
        _summary_row(orderbook_df, metric, label, unit)
        for metric, label, unit, _ in metric_specs
    ])

    print("Metric definitions:")
    _display_table(metric_definitions)

    print("Summary statistics across the converged-window (run, episode, year)
↪sample:")
    _display_table(summary_stats, numeric_columns=["Mean", "Median", "Std"])

    bottleneck_meanings = pd.DataFrame({
        "Primary bottleneck": [
            "best prices do not cross",
            "only marginal prices cross",
            "sell intent infeasible",
            "no buy intent / no sell intent",
            "side-limited/mostly filled",
        ],
        "Meaning": [
            "Even the highest bid is below the lowest feasible ask after
↪tolerance.",
            "At least one buyer and seller can trade, but most submitted volume
↪sits outside the crossed part of the order book.",
            "Agents wanted to sell allowances they could not legally sell after
↪compliance, banking, and no-short constraints.",
        ]
    })

```

```

        "One side of the market is absent, so no bilateral match is
↳possible.",
        "The remaining shortfall is mainly because one side has less
↳feasible volume than the other.",
    ],
})
print("Bottleneck label definitions:")
_display_table(bottleneck_meanings)

bottleneck_summary = (
    orderbook_df.groupby("primary_bottleneck")
    .agg(
        Rows=("primary_bottleneck", "size"),
        Share_pct=("primary_bottleneck", lambda s: 100.0 * len(s) /
↳len(orderbook_df)),
        Real_fill_mean_pct=("real_fill_pct", "mean"),
        Real_fill_median_pct=("real_fill_pct", "median"),
        Real_fill_std_pct=("real_fill_pct", "std"),
        Executed_median_Mt=("executed_Mt", "median"),
        ↳
↳Unfilled_feasible_overlap_median_Mt=("unfilled_feasible_overlap_Mt",
↳"median"),
        Gap_after_tolerance_median_EURt=("gap_after_tolerance_EURt",
↳"median"),
    )
    .sort_values("Share_pct", ascending=False)
    .reset_index()
    .rename(columns={"primary_bottleneck": "Primary bottleneck"})
)
print("Bottleneck summary:")
_display_table(
    bottleneck_summary,
    numeric_columns=[
        "Share_pct", "Real_fill_mean_pct", "Real_fill_median_pct",
↳"Real_fill_std_pct",
        "Executed_median_Mt", "Unfilled_feasible_overlap_median_Mt",
↳"Gap_after_tolerance_median_EURt",
    ],
)

by_year = (
    orderbook_df.groupby("year")
    .agg(
        Buy_intent_median_Mt=("buy_intent_Mt", "median"),
        Raw_sell_intent_median_Mt=("raw_sell_intent_Mt", "median"),
        Feasible_sell_median_Mt=("feasible_sell_Mt", "median"),

```

```

        Executed_median_Mt=("executed_Mt", "median"),
        Real_fill_mean_pct=("real_fill_pct", "mean"),
        Real_fill_median_pct=("real_fill_pct", "median"),
        Real_fill_std_pct=("real_fill_pct", "std"),
        Buyer_limit_median_EURt=("buyer_limit_wavg_EURt", "median"),
        Feasible_seller_ask_median_EURt=("feasible_seller_ask_wavg_EURt",
↪ "median"),
        Best_buy_median_EURt=("best_buy_EURt", "median"),
        Best_feasible_ask_median_EURt=("best_feasible_ask_EURt", "median"),
        Gap_after_tolerance_median_EURt=("gap_after_tolerance_EURt",
↪ "median"),
    )
    Unfilled_feasible_overlap_median_Mt=("unfilled_feasible_overlap_Mt",
↪ "median"),
    Clipped_sell_median_Mt=("clipped_sell_Mt", "median"),
)
.reset_index()
.rename(columns={"year": "Year"})
)
print("By-year medians, with mean/std shown for the real fill rate:")
_display_table(by_year, numeric_columns=[col for col in by_year.columns if
↪ col != "Year"])

fig, axes = plt.subplots(1, 3, figsize=(18, 4))

ax = axes[0]
ax.plot(by_year["Year"], by_year["Buy_intent_median_Mt"], "o--", label="buy
↪ intent")
ax.plot(by_year["Year"], by_year["Raw_sell_intent_median_Mt"], "o--",
↪ label="raw sell intent")
ax.plot(by_year["Year"], by_year["Feasible_sell_median_Mt"], "o-",
↪ label="feasible sell")
ax.plot(by_year["Year"], by_year["Executed_median_Mt"], "o-",
↪ color="black", linewidth=2.0, label="executed")
ax.set_xlabel("Year"); ax.set_ylabel("Median volume (Mt)")
ax.set_title("Wanted vs feasible vs executed volume")
ax.grid(alpha=0.3); ax.legend(fontsize=8)
format_year_axis(ax)

ax = axes[1]
ax.plot(by_year["Year"], by_year["Buyer_limit_median_EURt"], "o-",
↪ label="buy limit, qty-wavg")
ax.plot(by_year["Year"], by_year["Feasible_seller_ask_median_EURt"], "o-",
↪ label="feasible sell ask, qty-wavg")
ax.plot(by_year["Year"], by_year["Best_buy_median_EURt"], "o--",
↪ label="best buy")

```

```

    ax.plot(by_year["Year"], by_year["Best_feasible_ask_median_EURt"], "o--",
    ↪label="best feasible ask")
    ax.set_xlabel("Year"); ax.set_ylabel("Median limit price (EUR/t)")
    ax.set_title("Price wanted by both sides")
    ax.grid(alpha=0.3); ax.legend(fontsize=8)
    format_year_axis(ax)

    ax = axes[2]
    bottleneck_order = [
        "only marginal prices cross",
        "best prices do not cross",
        "sell intent infeasible",
        "no sell intent",
        "no buy intent",
        "no submitted orders",
        "side-limited/mostly filled",
    ]
    bottleneck_share = (
        pd.crosstab(orderbook_df["year"], orderbook_df["primary_bottleneck"],
    ↪normalize="index")
        .mul(100.0)
        .reindex(columns=bottleneck_order, fill_value=0.0)
    )
    bottleneck_share.plot(kind="bar", stacked=True, ax=ax, width=0.85)
    ax.set_xlabel("Year"); ax.set_ylabel("Share of rows (%)")
    ax.set_title("Dominant mismatch reason")
    ax.grid(axis="y", alpha=0.3); ax.legend(fontsize=8, loc="upper left",
    ↪bbox_to_anchor=(1.0, 1.0))

    fig.suptitle("Secondary-market real-match bottleneck diagnosis", y=1.04)
    plt.tight_layout(); plt.show()

```

RQ4.3b secondary-market order-book diagnosis

Sample: 48,000 (run, episode, year) rows across 4 runs

Metric definitions:

<pandas.io.formats.style.Styler at 0x2531da0b740>

Summary statistics across the converged-window (run, episode, year) sample:

<pandas.io.formats.style.Styler at 0x24fa1bba9f0>

Bottleneck label definitions:

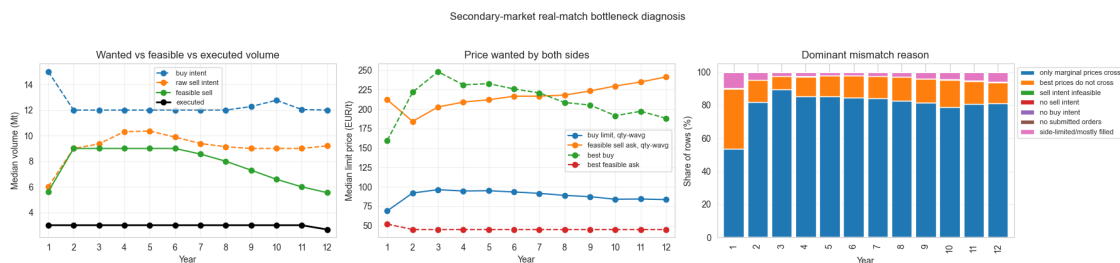
<pandas.io.formats.style.Styler at 0x24fa1bba9f0>

Bottleneck summary:

<pandas.io.formats.style.Styler at 0x24fa9fe6720>

By-year medians, with mean/std shown for the real fill rate:

<pandas.io.formats.style.Styler at 0x24fad3e99d0>



1.8.10 RQ4.4 — Converged-window credit stress by archetype × weighting

```
[27]: # RQ4.4 - Per-(archetype, weighting) cell of the mean peak emergency-loan
# balance and peak carry-forward debt in the converged window. Sourced from
# the per-episode `peak_loan_outstanding_A{i}` / `peak_carry_forward_A{i}`
# (v8.6.1+); pre-8.6.1 logs lack these columns and produce an empty table.
import numpy as np
import pandas as pd

def _converged_credit_metrics():
    rows = []
    for k, run in runs.items():
        ep_tail = tail_episodes(run["ep_df"])
        cfg = run["config"]
        for i in range(N_AGENTS):
            arche = ARCHETYPES[i // 2] if i // 2 < len(ARCHETYPES) else
            f"A{i+1}"
            w_cost, w_green = (cfg.get("companies", {}).get("reward_weights")
            or [[1, 0]] * N_AGENTS)[i]
            weighting = "Financial" if w_green < 0.25 else ("ESG" if w_green >
            0.75 else "Balanced")
            row = {"sweep": k[0], "variant": k[1], "seed": k[2],
                    "archetype": arche, "weighting": weighting, "agent": i + 1}
            for stub in ["peak_loan_outstanding", "peak_carry_forward",
                        "final_treasury_reserve"]:
                col = f"{stub}_A{i+1}"
                row[stub] = float(ep_tail[col].mean()) if col in ep_tail.
            columns else np.nan
            rows.append(row)
    return pd.DataFrame(rows)

credit_df = _converged_credit_metrics()
if credit_df.empty or credit_df[["peak_loan_outstanding",
    "peak_carry_forward"]].isna().all().all():
```

```

print("[RQ4.4] peak_loan_outstanding_A* / peak_carry_forward_A* missing -\n
↳pre-8.6.1 log.")
else:
    pivot_loan = credit_df.pivot_table(
        index="archetype", columns="weighting", values="peak_loan_outstanding",\n
↳aggfunc="mean"
    ).round(3)
    pivot_cf = credit_df.pivot_table(
        index="archetype", columns="weighting", values="peak_carry_forward",\n
↳aggfunc="mean"
    ).round(3)
    print("Mean peak emergency-loan balance (M€) - converged window:")
    display(pivot_loan)
    print("\nMean peak end-of-year carry-forward (Mt) - converged window:")
    display(pivot_cf)

```

Mean peak emergency-loan balance (M€) - converged window:

weighting archetype	Balanced	Financial
Coal-heavy	0.0	0.0
Gas-dominant	0.0	0.0
Green-leader	0.0	0.0
Transitioner	0.0	0.0

Mean peak end-of-year carry-forward (Mt) - converged window:

weighting archetype	Balanced	Financial
Coal-heavy	0.778	2.038
Gas-dominant	0.554	1.366
Green-leader	0.068	0.166
Transitioner	0.490	0.273

1.8.11 RQ4.5 — Why agents go in compliant: per-cell root-cause attribution

The UDBC mix in §RQ3.4 / §RQ2.3 already tells us *how often* each combinatorial state (under-bid / debt / mixed / bank-covered / secondary-covered) occurs. What it does not isolate is the **proximate cause** of each non-compliant cell: did the agent bid too cheaply and lose the auction, did it ask for too few tonnes (`bid_qty_mult` × `estimate_need` short of realised `need`), did the auction itself clear oversubscribed, was it carrying inherited debt forward, or did it submit a buy intent on the secondary that nobody filled?

This panel pulls every non-compliant cell (`episode`, `year`, `agent`) from the converged-tail window only and runs each through a prioritised waterfall:

1. **Inherited debt** — `alloc` `emiss` AND `cf_start` `shortfall`. The agent was auction-compliant *this year*; the deficit is fully explained by debt carried forward from earlier years.

2. **Underbid (price)** — $\text{bid_price} < 99.5 \% \times \text{clearing_price}$. The agent was priced out of the marginal allocation tranche.
3. **Undersized bid (quantity)** — $(\text{bid_qty_mult} \times \text{estimate_need})$ falls short of the realised need by more than half of the shortfall. The agent asked for too few tonnes regardless of price.
4. **Oversubscribed auction** — $\text{auction_total_demand} / \text{auction_volume} > 1.05$. Even bidding at clearing, the agent was rationed by total demand exceeding supply.
5. **Secondary illiquidity** — agent submitted a positive buy intent ($\text{sec_qty_action} > 0$) but the realised buy fell more than 25 % of the shortfall short of intent.
6. **Mixed / cascade** — fallback bucket; no single cause dominates.

The taxonomy is intentionally simple and prioritised — each cell is assigned exactly one dominant cause. The downstream tables report both mean and median diagnostic-feature values per cause and in the compliant/non-compliant contrast, so the central tendency and outlier sensitivity are visible together.

```
[29]: # RQ4.5 - Per-cell root-cause attribution for non-compliant (agent, year)
# cells in the converged tail window. Cells silently no-op on logs lacking
# the v8.6.1+ year-log columns.
import textwrap
from pathlib import Path
from matplotlib.backends.backend_pdf import PdfPages

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

EPS_MT = 1e-6
CAUSE_ORDER = ['Inherited debt', 'Underbid (price)', 'Undersized bid (qty)',
               'Oversubscribed auction', 'Secondary illiquidity', 'Mixed / ↵
               ↵cascade']
CAUSE_COLORS = {
    'Inherited debt': '#9467bd',
    'Underbid (price)': '#d62728',
    'Undersized bid (qty)': '#ff7f0e',
    'Oversubscribed auction': '#1f77b4',
    'Secondary illiquidity': '#17becf',
    'Mixed / cascade': '#7f7f7f',
}
STRESSED_SOLUTION_ORDER = [
    'Banked allowances', 'Secondary market', 'Bank + secondary', 'Other / ↵
    ↵accounting buffer'
]
STRESSED_SOLUTION_COLORS = {
    'Banked allowances': '#2ca02c',
    'Secondary market': '#1f77b4',
    'Bank + secondary': '#17becf',
    'Other / accounting buffer': '#7f7f7f',
```

```

}

EXPORT_RQ45_CELL_PDF = True
RQ45_CELL_PDF_PATH = (PROJECT_ROOT / 'results' / 'analysis' / 'rq45_cell_output.
    ↪pdf'
                      if 'PROJECT_ROOT' in globals() else
    ↪Path('rq45_cell_output.pdf'))
RQ45_AGENT_ORDER = list(range(1, N_AGENTS + 1))
RQ45_BASELINE_ROWS = ['All tail cells (normal/default)', 'Fully compliant',
    ↪cells']
RQ45_DIAG_ROW_ORDER = RQ45_BASELINE_ROWS + CAUSE_ORDER
RQ45_DIAG_FEATURES = [
    'shortfall_Mt', 'inherited_debt_Mt', 'alloc_Mt', 'emissions_Mt',
    'bid_price', 'clearing_price', 'bid_to_clearing', 'bid_qty_mult',
    'bid_coverage_ratio', 'estimate_need_Mt', 'need_realised_Mt',
    'undersized_qty_Mt', 'oversubscription_ratio', 'bank_start_Mt',
    'sec_buy_intent_Mt', 'secondary_buy_unmet_Mt', 'loan_outstanding_Meur',
]

def _rq45_agent_label(agent_num):
    idx0 = agent_num - 1
    weighting = _agent_weighting(idx0)
    weighting_short = 'ESG' if weighting == 'Balanced ESG' else weighting
    return f'A{agent_num} {_agent_archetype(idx0)}\n{weighting_short}'

RQ45_AGENT_LABELS = {agent: _rq45_agent_label(agent) for agent in
    ↪RQ45_AGENT_ORDER}
_rq45_pdf_items = []
_rq45_diag_values = {
    row_label: {feature: [] for feature in RQ45_DIAG_FEATURES}
    for row_label in RQ45_DIAG_ROW_ORDER
}
stressed_rows = []

def _rq45_log(message):
    print(message)
    _rq45_pdf_items.append(('text', str(message)))

def _rq45_record_table(title, df):
    _rq45_pdf_items.append(('table', title, df.copy()))

def _rq45_record_figure(title, fig):

```



```

_rq45_pdf_items.append(('figure', title, fig))

def _rq45_append_diag_values(row_label, mask, feature_values):
    mask = np.asarray(mask, dtype=bool)
    if not mask.any():
        return
    for feature in RQ45_DIAG_FEATURES:
        values = feature_values.get(feature)
        if values is None:
            continue
        arr = np.asarray(values, dtype=float)
        if arr.shape[0] != mask.shape[0]:
            continue
        _rq45_diag_values[row_label][feature].append(arr[mask])

def _rq45_concat_diag(row_label, feature):
    pieces = _rq45_diag_values.get(row_label, {}).get(feature, [])
    if not pieces:
        return np.asarray([], dtype=float)
    return np.concatenate([np.asarray(p, dtype=float) for p in pieces])

def _rq45_stat(values, stat):
    finite = values[np.isfinite(values)]
    if finite.size == 0:
        return np.nan
    if stat == 'mean':
        return float(np.mean(finite))
    if stat == 'median':
        return float(np.median(finite))
    if stat == 'std':
        return float(np.std(finite, ddof=1)) if finite.size > 1 else np.nan
    raise ValueError(stat)

def _rq45_build_diag_summary():
    rows = []
    for row_label in RQ45_DIAG_ROW_ORDER:
        shortfall_values = _rq45_concat_diag(row_label, 'shortfall_Mt')
        row = {'diagnostic_group': row_label, 'n_cells': int(shortfall_values.
↵size)}
        for feature in RQ45_DIAG_FEATURES:
            values = _rq45_concat_diag(row_label, feature)
            for stat in ['mean', 'median', 'std']:
                row[f'{feature}_{stat}'] = _rq45_stat(values, stat)

```

```

        rows.append(row)
    return pd.DataFrame(rows).set_index('diagnostic_group')

def _stressed_solution_label(gap, bank_available, sec_bought):
    if sec_bought >= gap - EPS_MT and sec_bought > EPS_MT:
        return 'Secondary market'
    if bank_available >= gap - EPS_MT and bank_available > EPS_MT:
        return 'Banked allowances'
    if sec_bought > EPS_MT and bank_available > EPS_MT and sec_bought +
↪bank_available >= gap - EPS_MT:
        return 'Bank + secondary'
    if sec_bought > EPS_MT:
        return 'Secondary market'
    if bank_available > EPS_MT:
        return 'Banked allowances'
    return 'Other / accounting buffer'

def _rq45_pdf_text_pages(pdf, title, text, chars_per_line=118,
↪lines_per_page=45):
    lines = []
    for raw_line in str(text).splitlines() or ['']:
        if len(raw_line) <= chars_per_line:
            lines.append(raw_line)
        else:
            lines.extend(textwrap.wrap(raw_line, width=chars_per_line,
                                       break_long_words=False,
                                       replace_whitespace=False) or [''])
    chunks = [lines[i:i + lines_per_page] for i in range(0, len(lines),
↪lines_per_page)] or [[]]
    for page_idx, chunk in enumerate(chunks):
        fig = plt.figure(figsize=(11.69, 8.27))
        page_title = title if len(chunks) == 1 else f'{title} ({page_idx + 1}/
↪{len(chunks)})'
        fig.text(0.02, 0.97, page_title, fontsize=12, weight='bold', va='top')
        fig.text(0.02, 0.91, '\n'.join(chunk), fontsize=8, family='monospace',
↪va='top')
        pdf.savefig(fig, bbox_inches='tight')
        plt.close(fig)

def _rq45_pdf_table_pages(pdf, title, df, cols_per_page=6):
    if df.empty:
        _rq45_pdf_text_pages(pdf, title, '<empty>')
        return
    table_df = df.copy()

```

```

if len(table_df.columns) <= cols_per_page:
    _rq45_pdf_text_pages(pdf, title, table_df.to_string())
    return
for start in range(0, len(table_df.columns), cols_per_page):
    end = min(start + cols_per_page, len(table_df.columns))
    page_df = table_df.iloc[:, start:end]
    page_title = f'{title} | columns {start + 1}-{end} of {len(table_df.
↪columns))}'
    _rq45_pdf_text_pages(pdf, page_title, page_df.to_string())

def _rq45_export_pdf(items, path):
    if not items:
        return None
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with PdfPages(path) as pdf:
        cover = plt.figure(figsize=(11.69, 8.27))
        cover.text(0.05, 0.92, 'RQ4.5 cell output', fontsize=18, weight='bold',
↪va='top')
        cover.text(0.05, 0.84, f'Notebook cell export: {path.name}',
↪fontsize=10, va='top')
        cover.text(0.05, 0.78, 'Includes this cell\'s printed text, displayed
↪tables, and generated figures.',
                    fontsize=10, va='top')
        pdf.savefig(cover, bbox_inches='tight')
        plt.close(cover)
    for item in items:
        kind = item[0]
        if kind == 'text':
            _rq45_pdf_text_pages(pdf, 'Printed output', item[1])
        elif kind == 'table':
            _rq45_pdf_table_pages(pdf, item[1], item[2])
        elif kind == 'figure':
            pdf.savefig(item[2], bbox_inches='tight')
    return path

def _classify_row(sf, cf_start, alloc, emiss, bid_p, clear_p,
                  undersized_qty, oversub, sec_unmet):
    if sf <= EPS_MT:
        return None
    cf = max(0.0, float(cf_start))
    if alloc + EPS_MT >= emiss and cf >= sf - EPS_MT:
        return 'Inherited debt'
    if bid_p > 0 and clear_p > 0 and bid_p < 0.995 * clear_p:
        return 'Underbid (price)'

```

```

if undersized_qty > 0.5 * sf:
    return 'Undersized bid (qty)'
if oversub > 1.05:
    return 'Oversubscribed auction'
if sec_unmet > 0.25 * sf:
    return 'Secondary illiquidity'
return 'Mixed / cascade'

nc_rows = []
rq45_tail_year_rows_by_run = {}
rq45_total_tail_cells = 0
for k, run in runs.items():
    yr = tail_year_rows(run)
    if yr.empty or 'year' not in yr.columns:
        continue
    n = len(yr)
    rq45_tail_year_rows_by_run[k] = n
    rq45_total_tail_cells += n * N_AGENTS
    cp_arr = yr['clearing_price'].to_numpy(float) if 'clearing_price' in yr.columns else np.full(n, np.nan)
    sp_arr = yr['secondary_price'].to_numpy(float) if 'secondary_price' in yr.columns else np.full(n, np.nan)
    atd = yr['auction_total_demand'].to_numpy(float) if 'auction_total_demand' in yr.columns else np.full(n, np.nan)
    avol = yr['auction_volume'].to_numpy(float) if 'auction_volume' in yr.columns else np.full(n, np.nan)
    ns_exec = yr['secondary_n_sellers_executed'].to_numpy(float) if 'secondary_n_sellers_executed' in yr.columns else np.full(n, np.nan)
    yr_arr = yr['year'].to_numpy(int)
    ep_arr = yr['episode'].to_numpy(int) if 'episode' in yr.columns else np.zeros(n, dtype=int)
    oversub_y = np.where(avol > 1e-9, atd / avol, np.nan)

    for i in range(N_AGENTS):
        cols_req = [f'shortfall_A{i+1}', f'alloc_A{i+1}', f'emissions_A{i+1}',
                    f'carry_forward_start_A{i+1}', f'bid_price_A{i+1}',
                    f'estimate_need_A{i+1}', f'bid_qty_mult_A{i+1}',
                    f'bank_start_A{i+1}', f'trade_qty_A{i+1}',
                    f'sec_qty_action_A{i+1}']
        if not all(c in yr.columns for c in cols_req):
            continue
        sf = yr[f'shortfall_A{i+1}'].to_numpy(float)
        alloc = yr[f'alloc_A{i+1}'].to_numpy(float)
        emiss = yr[f'emissions_A{i+1}'].to_numpy(float)
        cf_st = yr[f'carry_forward_start_A{i+1}'].to_numpy(float)

```

```

bidp = yr[f'bid_price_A{i+1}'].to_numpy(float)
en = yr[f'estimate_need_A{i+1}'].to_numpy(float)
qm = yr[f'bid_qty_mult_A{i+1}'].to_numpy(float)
bank = yr[f'bank_start_A{i+1}'].to_numpy(float)
trq = yr[f'trade_qty_A{i+1}'].to_numpy(float)
sqa = yr[f'sec_qty_action_A{i+1}'].to_numpy(float)
cf_e = yr[f'carry_forward_end_A{i+1}'].to_numpy(float) if_
↳f'carry_forward_end_A{i+1}' in yr.columns else np.full(n, np.nan)
loan = yr[f'loan_outstanding_A{i+1}'].to_numpy(float) if_
↳f'loan_outstanding_A{i+1}' in yr.columns else np.full(n, np.nan)
tdr = yr[f'treasury_drawn_A{i+1}'].to_numpy(float) if_
↳f'treasury_drawn_A{i+1}' in yr.columns else np.full(n, np.nan)
cgap = yr[f'coverage_gap_A{i+1}'].to_numpy(float) if_
↳f'coverage_gap_A{i+1}' in yr.columns else np.full(n, np.nan)
sec_net = (yr[f'secondary_net_A{i+1}'].to_numpy(float)
            if f'secondary_net_A{i+1}' in yr.columns else np.full(n, np.
↳nan))

# need_realised = emissions + inherited debt the agent had to cover.
need_realised = emiss + np.maximum(0.0, cf_st)
bid_qty_target = np.maximum(0.0, qm) * np.maximum(0.0, en)
undersized_qty = np.maximum(0.0, need_realised - bid_qty_target)
auction_gap = np.maximum(0.0, need_realised - alloc)
sec_intent_buy = np.maximum(0.0, sqa)
sec_bought = np.maximum(0.0, trq)
sec_unmet = np.maximum(0.0, sec_intent_buy - sec_bought)
bank_available = np.maximum(0.0, bank)
sec_paid_fallback = np.where(np.isfinite(sp_arr), sec_bought * sp_arr,
↳np.nan)
sec_direct_paid = np.where(np.isfinite(sec_net) & (sec_net > EPS_MT),
↳sec_net, np.nan)
sec_paid = np.where(np.isfinite(sec_direct_paid), sec_direct_paid,
↳sec_paid_fallback)
sec_paid = np.where(sec_bought > EPS_MT, sec_paid, 0.0)
residual_after_secondary = np.maximum(0.0, auction_gap - sec_bought)
bank_used_est = np.minimum(bank_available, residual_after_secondary)
with np.errstate(divide='ignore', invalid='ignore'):
    bid_to_clearing = np.where(cp_arr > 1e-9, bidp / cp_arr, np.nan)
    bid_coverage_ratio = np.where(need_realised > 1e-9,
                                bid_qty_target / need_realised,
                                np.nan)
    sec_paid_price = np.where(sec_bought > EPS_MT, sec_paid /
↳sec_bought, np.nan)

feature_values = {
    'shortfall_Mt': sf,

```

```

        'inherited_debt_Mt': cf_st,
        'alloc_Mt': alloc,
        'emissions_Mt': emiss,
        'bid_price': bidp,
        'clearing_price': cp_arr,
        'bid_to_clearing': bid_to_clearing,
        'bid_qty_mult': qm,
        'bid_coverage_ratio': bid_coverage_ratio,
        'estimate_need_Mt': en,
        'need_realised_Mt': need_realised,
        'undersized_qty_Mt': undersized_qty,
        'oversubscription_ratio': oversub_y,
        'bank_start_Mt': bank,
        'sec_buy_intent_Mt': sec_intent_buy,
        'secondary_buy_unmet_Mt': sec_unmet,
        'loan_outstanding_Meur': loan,
    }
    compliant_mask = sf <= EPS_MT
    _rq45_append_diag_values('All tail cells (normal/default)', np.ones(n,
dtype=bool), feature_values)
    _rq45_append_diag_values('Fully compliant cells', compliant_mask,
feature_values)

    stressed_mask = (auction_gap > EPS_MT) & compliant_mask
    for j in np.where(stressed_mask)[0]:
        gap = float(auction_gap[j])
        sec_buy = float(sec_bought[j])
        bank_avail = float(bank_available[j])
        solution = _stressed_solution_label(gap, bank_avail, sec_buy)
        stressed_rows.append({
            'run': f'{k[1]}/s{k[2]}',
            'episode': int(ep_arr[j]),
            'year': int(yr_arr[j]),
            'agent': i + 1,
            'archetype': _agent_archetype(i),
            'weighting': _agent_weighting(i),
            'solution': solution,
            'initial_auction_gap_Mt': gap,
            'need_realised_Mt': float(need_realised[j]),
            'alloc_Mt': float(alloc[j]),
            'bank_start_Mt': bank_avail,
            'bank_used_estimate_Mt': float(bank_used_est[j]),
            'secondary_buy_Mt': sec_buy,
            'secondary_paid_Meur': float(sec_paid[j]) if np.
isfinite(sec_paid[j]) else np.nan,
            'secondary_paid_price_EUR_t': float(sec_paid_price[j]) if np.
isfinite(sec_paid_price[j]) else np.nan,

```

```

    })

    nc_mask = sf > EPS_MT
    if not nc_mask.any():
        continue

    cause_arr = np.full(n, None, dtype=object)
    for j in np.where(nc_mask)[0]:
        cp_j = float(cp_arr[j]) if np.isfinite(cp_arr[j]) else 0.0
        os_j = float(oversub_y[j]) if np.isfinite(oversub_y[j]) else 0.0
        cause = _classify_row(
            sf=float(sf[j]), cf_start=float(cf_st[j]),
            alloc=float(alloc[j]), emiss=float(emiss[j]),
            bid_p=float(bidp[j]), clear_p=cp_j,
            undersized_qty=float(undersized_qty[j]),
            oversub=os_j, sec_unmet=float(sec_unmet[j]),
        )
        cause_arr[j] = cause
        nc_rows.append({
            'run': f'{k[1]}/s{k[2]}',
            'episode': int(ep_arr[j]),
            'year': int(yr_arr[j]),
            'agent': i + 1,
            'archetype': _agent_archetype(i),
            'weighting': _agent_weighting(i),
            'shortfall_Mt': float(sf[j]),
            'inherited_debt_Mt': float(cf_st[j]),
            'alloc_Mt': float(alloc[j]),
            'emissions_Mt': float(emiss[j]),
            'bid_price': float(bidp[j]),
            'clearing_price': cp_j,
            'bid_to_clearing': float(bid_to_clearing[j]) if np.
↳ isfinite(bid_to_clearing[j]) else np.nan,
            'bid_qty_mult': float(qm[j]),
            'bid_coverage_ratio': float(bid_coverage_ratio[j]) if np.
↳ isfinite(bid_coverage_ratio[j]) else np.nan,
            'estimate_need_Mt': float(en[j]),
            'need_realised_Mt': float(need_realised[j]),
            'undersized_qty_Mt': float(undersized_qty[j]),
            'oversubscription_ratio': os_j,
            'bank_start_Mt': float(bank[j]),
            'sec_buy_intent_Mt': float(sec_intent_buy[j]),
            'secondary_buy_unmet_Mt': float(sec_unmet[j]),
            'n_sellers_executed': float(ns_exec[j]) if np.
↳ isfinite(ns_exec[j]) else np.nan,
            'carry_forward_end_Mt': float(cf_e[j]),
            'loan_outstanding_Meur': float(loan[j]),

```

```

        'treasury_drawn_Meur':    float(tdr[j]),
        'coverage_gap_Mt':       float(cgap[j]),
        'cause':                 cause,
    })

    for cause_label in CAUSE_ORDER:
        _rq45_append_diag_values(cause_label, cause_arr == cause_label,
        ↪feature_values)

incomp_df = pd.DataFrame(nc_rows)
stressed_df = pd.DataFrame(stressed_rows)
rq45_expected_cells_per_run = TAIL_EPISODES * N_AGENTS * N_YEARS
rq45_loaded_run_count = len(rq45_tail_year_rows_by_run)
_rq45_log(
    f'[RQ4.5] Denominator: {rq45_loaded_run_count} loaded run(s) × '
    f'{TAIL_EPISODES} tail episodes × {N_AGENTS} agents × {N_YEARS} years '
    f'={rq45_loaded_run_count * rq45_expected_cells_per_run:,} expected cells '
    f'({rq45_expected_cells_per_run:,} per run); actual counted cells = '
    f'{rq45_total_tail_cells:,}.'
)
if rq45_total_tail_cells > 0:
    _rq45_log(
        f'[RQ4.5] Final non-compliant cells: {len(incomp_df):,} '
        f'({100.0 * len(incomp_df) / rq45_total_tail_cells:.1f}%); '
        f'auction-stressed but compliant cells: {len(stressed_df):,} '
        f'({100.0 * len(stressed_df) / rq45_total_tail_cells:.1f}%).'
    )

if incomp_df.empty:
    _rq45_log(['RQ4.5] No non-compliant (agent, year) cells in the converged_
    ↪window - '
            'agents are fully compliant. Skipping attribution panels.'])
else:
    incomp_df['cause'] = pd.Categorical(incomp_df['cause'],
    ↪categories=CAUSE_ORDER, ordered=True)

    # (a) Counts and tonne-weighted shares per cause.
    cause_counts = (incomp_df.groupby('cause', observed=True)
                    .agg(n_cells=('shortfall_Mt', 'size'),
                        sum_shortfall_Mt=('shortfall_Mt', 'sum'),
                        mean_shortfall_Mt=('shortfall_Mt', 'mean'),
                        median_shortfall_Mt=('shortfall_Mt',
    ↪'median'))
                    .reindex(CAUSE_ORDER).fillna(0.0))
    total_cells = max(1, int(cause_counts['n_cells'].sum()))
    total_tons = max(1e-9, float(cause_counts['sum_shortfall_Mt'].sum()))

```



```

    cause_counts['cell_share_pct'] = 100.0 * cause_counts['n_cells'] /
    ↪total_cells
    cause_counts['tonne_share_pct'] = 100.0 * cause_counts['sum_shortfall_Mt'] /
    ↪total_tons

    _rq45_log(f'[RQ4.5] Non-compliant (agent, year, episode) cells in converged_
    ↪window: '
                f'{len(incomp_df):,}'))
    _rq45_log(f'[RQ4.5]  $\Sigma$  shortfall across the window: '
                f'{incomp_df["shortfall_Mt"].sum():.2f} Mt')
    cause_counts_display = cause_counts.round(3)
    _rq45_record_table('Cause counts and shares', cause_counts_display)
    display(cause_counts_display)

    fig, axes = plt.subplots(1, 2, figsize=(14, 4.6))
    colors = [CAUSE_COLORS[c] for c in CAUSE_ORDER]
    axes[0].bar(CAUSE_ORDER, cause_counts['cell_share_pct'].values,
    ↪color=colors)
    axes[0].set(title='Share of non-compliant cells by dominant cause',
                ylabel='% of (agent, year, episode) cells')
    axes[0].tick_params(axis='x', rotation=25); axes[0].grid(alpha=0.3,
    ↪axis='y')

    axes[1].bar(CAUSE_ORDER, cause_counts['tonne_share_pct'].values,
    ↪color=colors)
    axes[1].set(title='Share of total shortfall tonnes by dominant cause',
                ylabel='% of  $\Sigma$  shortfall (Mt)')
    axes[1].tick_params(axis='x', rotation=25); axes[1].grid(alpha=0.3,
    ↪axis='y')
    plt.suptitle(f'RQ4.5 - Why agents go non-compliant in the converged window '
                f'(pooled across {rq45_loaded_run_count} loaded seeds)',
                y=1.03, fontsize=13)
    plt.tight_layout()
    _rq45_record_figure('Cause shares', fig)
    plt.show()

    # (b) Baseline and per-cause diagnostic feature values.
    cause_feature_summary = _rq45_build_diag_summary()
    _rq45_log('RQ4.5] Baseline and per-cause feature values (mean / median /
    ↪std):')
    _rq45_log('
                All tail cells = normal/default-config bid baseline; '
                'Fully compliant cells = regular successful bid baseline.')
    cause_feature_display = cause_feature_summary.round(3)
    _rq45_record_table('Baseline and per-cause feature values',
    ↪cause_feature_display)
    display(cause_feature_display)

```

```

# (c) Simulation-ordered per-agent cause split (A1..A8: archetype x
↳weighting).
agent_cause_counts = (incomp_df.groupby(['agent', 'cause'], observed=True)
                        .size().unstack('cause', fill_value=0)
                        .reindex(index=RQ45_AGENT_ORDER,
                                columns=CAUSE_ORDER,
                                fill_value=0).astype(int))
agent_labels = [RQ45_AGENT_LABELS.get(agent, f'A{agent}')]
                for agent in agent_cause_counts.index]
agent_cause_share = (agent_cause_counts
                     .div(agent_cause_counts.sum(axis=1).replace(0, np.
↳nan), axis=0)
                     .fillna(0.0))

if not agent_cause_share.empty:
    fig, ax = plt.subplots(figsize=(11.8, 4.6))
    agent_cause_share.plot(kind='bar', stacked=True, ax=ax,
                           color=[CAUSE_COLORS[c] for c in
↳agent_cause_share.columns])
    ax.set(title=f'RQ4.5 - Cause mix by simulation archetype order '
                f'(pooled across {rq45_loaded_run_count} loaded seeds)',
           ylabel='Share of non-compliant cells', ylim=(0, 1.0))
    ax.set_xticklabels(agent_labels, rotation=20, ha='right')
    ax.legend(fontsize=8, loc='center left', bbox_to_anchor=(1.02, 0.5))
    ax.grid(alpha=0.3, axis='y')
    plt.tight_layout()
    _rq45_record_figure('Agent-ordered cause mix shares', fig)
    plt.show()

fig_abs, ax_abs = plt.subplots(figsize=(11.8, 4.6))
agent_cause_counts.plot(kind='bar', stacked=True, ax=ax_abs,
                        color=[CAUSE_COLORS[c] for c in
↳agent_cause_counts.columns])
    ax_abs.set(title=f'RQ4.5 - Absolute non-compliant cells by simulation
↳archetype order '
                    f'(pooled across {rq45_loaded_run_count} loaded
↳seeds)',
               ylabel='Non-compliant cells (pooled count)')
    ax_abs.set_xticklabels(agent_labels, rotation=20, ha='right')
    ax_abs.legend(fontsize=8, loc='center left', bbox_to_anchor=(1.02, 0.5))
    ax_abs.grid(alpha=0.3, axis='y')
    plt.tight_layout()
    _rq45_record_figure('Agent-ordered absolute cause counts', fig_abs)
    plt.show()

```

```

# (d) Compliant vs non-compliant feature contrast on the same tail window.
comp_pieces = []
for k, run in runs.items():
    yr = tail_year_rows(run)
    if yr.empty: continue
    for i in range(N_AGENTS):
        sf_col = f'shortfall_A{i+1}'
        if sf_col not in yr.columns: continue
        sf = yr[sf_col].to_numpy(float)
        cmask = sf <= EPS_MT
        if not cmask.any(): continue
        row = {}
        for stub, src_col in [
            ('inherited_debt_Mt', f'carry_forward_start_A{i+1}'),
            ('bank_start_Mt', f'bank_start_A{i+1}'),
            ('bid_qty_mult', f'bid_qty_mult_A{i+1}'),
            ('bid_price', f'bid_price_A{i+1}'),
            ('coverage_gap_Mt', f'coverage_gap_A{i+1}'),
        ]:
            row[stub] = (yr.loc[cmask, src_col].to_numpy(float)
                         if src_col in yr.columns else np.full(int(cmask.
↪sum()), np.nan))
            row['clearing_price'] = (yr.loc[cmask, 'clearing_price'].
↪to_numpy(float)
                                   if 'clearing_price' in yr.columns
                                   else np.full(int(cmask.sum()), np.nan))
            comp_pieces.append(pd.DataFrame(row))
    if comp_pieces:
        comp_df = pd.concat(comp_pieces, ignore_index=True)
        comp_df['bid_to_clearing'] = (comp_df['bid_price']
                                     / comp_df['clearing_price']).replace(0,
↪np.nan))
        nc_with = incomple_df.copy()
        nc_with['bid_to_clearing'] = (nc_with['bid_price']
                                     / nc_with['clearing_price']).replace(0,
↪np.nan))
        contrast_cols = ['inherited_debt_Mt', 'bank_start_Mt',
                        'bid_qty_mult', 'bid_to_clearing', 'coverage_gap_Mt']
        rows = []
        for c in contrast_cols:
            cv = comp_df[c].replace([np.inf, -np.inf], np.nan).dropna() if c in
↪comp_df.columns else pd.Series(dtype=float)
            nv = nc_with[c].replace([np.inf, -np.inf], np.nan).dropna() if c in
↪nc_with.columns else pd.Series(dtype=float)
            cv_mean = float(cv.mean()) if len(cv) else np.nan
            nv_mean = float(nv.mean()) if len(nv) else np.nan

```

```

cv_median = float(cv.median()) if len(cv) else np.nan
nv_median = float(nv.median()) if len(nv) else np.nan
rows.append({
    'feature': c,
    'compliant_mean': cv_mean,
    'non_compliant_mean': nv_mean,
    'delta_mean(nc - c)': (nv_mean - cv_mean if len(cv) and
↪len(nv) else np.nan),
    'compliant_median': cv_median,
    'non_compliant_median': nv_median,
    'delta_median(nc - c)': (nv_median - cv_median if len(cv) and
↪len(nv) else np.nan),
})
_rq45_log(['RQ4.5] Compliant vs non-compliant cell means and medians
↪(key discriminators):'])
contrast_display = pd.DataFrame(rows).round(3)
_rq45_record_table('Compliant vs non-compliant feature contrast',
↪contrast_display)
display(contrast_display)

# (e) Auction-stressed but ultimately compliant cells: how did agents save the
↪year?
if stressed_df.empty:
    _rq45_log(['RQ4.5] No auction-stressed but ultimately compliant cells found
↪in the converged window.'])
else:
    stressed_df['solution'] = pd.Categorical(
        stressed_df['solution'], categories=STRESSED_SOLUTION_ORDER,
↪ordered=True
    )
    stressed_summary = (stressed_df.groupby('solution', observed=False)
        .agg(n_cells=('initial_auction_gap_Mt',
↪'size'),
            share_pct=('initial_auction_gap_Mt',
↪lambda s: 100.0 * len(s) / len(stressed_df)),
            mean_initial_gap_Mt=('initial_auction_gap_Mt', 'mean'),
            median_initial_gap_Mt=('initial_auction_gap_Mt', 'median'),
            sum_initial_gap_Mt=('initial_auction_gap_Mt', 'sum'),
            mean_bank_start_Mt=('bank_start_Mt',
↪'mean'),
            mean_bank_used_estimate_Mt=('bank_used_estimate_Mt', 'mean'),

```

```

        ↪mean_secondary_buy_Mt=('secondary_buy_Mt', 'mean'),
        ↪sum_secondary_buy_Mt=('secondary_buy_Mt', 'sum'),
        ↪mean_secondary_paid_Meur=('secondary_paid_Meur', 'mean'),
        ↪sum_secondary_paid_Meur=('secondary_paid_Meur', 'sum'),
        ↪mean_secondary_paid_price_EUR_t=('secondary_paid_price_EUR_t', 'mean'))
        .reindex(STRESSED_SOLUTION_ORDER).fillna(0.
        ↪0))
    _rq45_log(f'[RQ4.5] Auction-stressed but ultimately compliant cells:↪
    ↪{len(stressed_df):,}')
    _rq45_log('        Stressed = auction allocation < emissions +↪
    ↪carry-forward start; final shortfall = 0.')
    stressed_display = stressed_summary.round(3)
    _rq45_record_table('Auction-stressed compliant cells by solution',↪
    ↪stressed_display)
    display(stressed_display)

    fig_stress, ax_stress = plt.subplots(figsize=(8.8, 4.0))
    stress_counts = stressed_summary['n_cells'].astype(int)
    stress_colors = [STRESSED_SOLUTION_COLORS[s] for s in stress_counts.index]
    bars = ax_stress.bar(stress_counts.index.astype(str), stress_counts.values,↪
    ↪color=stress_colors)
    total_stressed = max(1, int(stress_counts.sum()))
    for bar, count in zip(bars, stress_counts.values):
        if count <= 0:
            continue
        ax_stress.text(bar.get_x() + bar.get_width() / 2,
            bar.get_height(),
            f'{count:,}\n({100.0 * count / total_stressed:.1f}%)',
            ha='center', va='bottom', fontsize=8)
    ax_stress.set(title=f'RQ4.5 - Auction-stressed compliant cells: resolution↪
    ↪channel '
        f'(pooled across {rq45_loaded_run_count} loaded seeds)',
        ylabel='Cell count (pooled)')
    ax_stress.tick_params(axis='x', rotation=15)
    ax_stress.grid(alpha=0.3, axis='y')
    plt.tight_layout()
    _rq45_record_figure('Auction-stressed compliant resolution counts',↪
    ↪fig_stress)
    plt.show()

if EXPORT_RQ45_CELL_PDF:

```

```

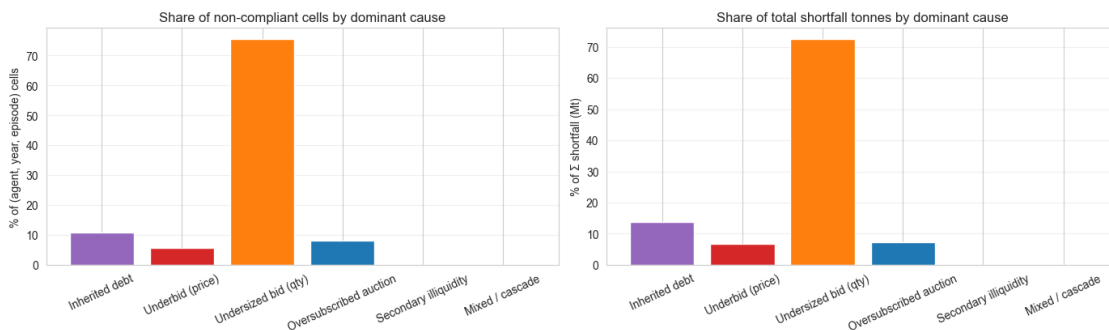
_rq45_log(f'[RQ4.5] Writing cell output PDF to: {RQ45_CELL_PDF_PATH}')
_rq45_pdf_path = _rq45_export_pdf(_rq45_pdf_items, RQ45_CELL_PDF_PATH)
if _rq45_pdf_path is not None:
    print(f'[RQ4.5] PDF export complete: {_rq45_pdf_path}')

```

[RQ4.5] Denominator: 4 loaded run(s) × 1000 tail episodes × 8 agents × 12 years = 384,000 expected cells (96,000 per run); actual counted cells = 384,000.
[RQ4.5] Final non-compliant cells: 66,112 (17.2%); auction-stressed but compliant cells: 142,175 (37.0%).
[RQ4.5] Non-compliant (agent, year, episode) cells in converged window: 66,112
[RQ4.5] Σ shortfall across the window: 79,149.41 Mt

	n_cells	sum_shortfall_Mt	mean_shortfall_Mt	
↳ median_shortfall_Mt	cell_share_pct	tonne_share_pct		
cause				
Inherited debt	7070	10841.089	1.533	↳
↳ 1.686	10.694	13.697		
Underbid (price)	3825	5321.238	1.391	↳
↳ 0.740	5.786	6.723		
Undersized bid (qty)	49874	57323.488	1.149	↳
↳ 1.099	75.439	72.424		
Oversubscribed auction	5333	5659.925	1.061	↳
↳ 0.718	8.067	7.151		
Secondary illiquidity	4	2.079	0.520	↳
↳ 0.547	0.006	0.003		
Mixed / cascade	6	1.589	0.265	↳
↳ 0.231	0.009	0.002		

RQ4.5 — Why agents go non-compliant in the converged window (pooled across 4 loaded seeds)



[RQ4.5] Baseline and per-cause feature values (mean / median / std):
All tail cells = normal/default-config bid baseline; Fully compliant cells = regular successful bid baseline.

	n_cells	shortfall_Mt_mean	shortfall_Mt_median	shortfall_Mt_std	inherited_debt_Mt_mean	inherited_debt_Mt_median	inherited_debt_Mt_std	alloc_Mt_mean
diagnostic_group								
All tail cells (normal/default)	384000	0.206	0.000	0.593	0.177	0.000		
				0.543	1.739			
Fully compliant cells	317888	0.000	0.000	0.000	0.060	0.000		
				0.295	1.797			
Inherited debt	7070	1.533	1.686	0.805	1.951	2.010		
				0.809	2.250			
Underbid (price)	3825	1.391	0.740	1.735	0.681	0.000		
				1.429	0.000			
Undersized bid (qty)	49874	1.149	1.099	0.800	0.611	0.078		
				0.797	1.556			
Oversubscribed auction	5333	1.061	0.718	1.177	0.370	0.000		
				0.894	0.509			
Secondary illiquidity	4	0.520	0.547	0.362	0.157	0.049		
				0.252	1.614			
Mixed / cascade	6	0.265	0.231	0.180	0.030	0.000		
				0.075	1.133			

	alloc_Mt_median	alloc_Mt_std	emissions_Mt_mean	emissions_Mt_median	emissions_Mt_std	bid_price_mean	bid_price_median	bid_price_std	clearing_price_mean
diagnostic_group									
All tail cells (normal/default)	1.563	1.217	1.675	0.868	96.857	91.032			
			46.931	54.383					
Fully compliant cells	1.582	1.282	1.525	0.835	91.997	84.665			
			44.816	53.839					
Inherited debt	2.260	0.702	1.986	0.584	148.542	165.621			
			44.188	69.600					
Underbid (price)	0.000	0.000	1.535	0.584	61.477	45.000			
			26.881	98.740					

Undersized bid (qty)			1.541	0.586	2.	
↳498	2.596		0.781	128.014	135.097	↳
↳	44.419	52.539				
Oversubscribed auction			0.270	0.609	1.	
↳847	1.807		0.614	52.076	45.000	↳
↳	19.354	52.077				
Secondary illiquidity			1.826	0.896	1.	
↳976	1.994		0.667	45.000	45.000	↳
↳	0.000	45.000				
Mixed / cascade			1.096	0.542	1.	
↳368	1.327		0.503	45.000	45.000	↳
↳	0.000	45.000				

			clearing_price_median	clearing_price_std	↳	
↳bid_to_clearing_mean	bid_to_clearing_median	bid_to_clearing_std	↳			
↳bid_qty_mult_mean	bid_qty_mult_median	\				
diagnostic_group						
All tail cells (normal/default)			45.000	20.575	↳	
↳	1.874	1.671	0.934	1.		
↳298	1.433					
Fully compliant cells			45.000	19.689	↳	
↳	1.789	1.545	0.891	1.		
↳418	1.814					
Inherited debt			61.282	28.262	↳	
↳	2.329	2.226	0.906	0.		
↳676	0.500					
Underbid (price)			94.685	33.604	↳	
↳	0.647	0.639	0.201	1.		
↳731	2.000					
Undersized bid (qty)			45.000	18.948	↳	
↳	2.537	2.535	0.911	0.		
↳522	0.500					
Oversubscribed auction			45.000	19.357	↳	
↳	1.000	1.000	0.000	1.		
↳890	2.000					
Secondary illiquidity			45.000	0.000	↳	
↳	1.000	1.000	0.000	1.		
↳097	1.111					
Mixed / cascade			45.000	0.000	↳	
↳	1.000	1.000	0.000	1.		
↳356	1.214					

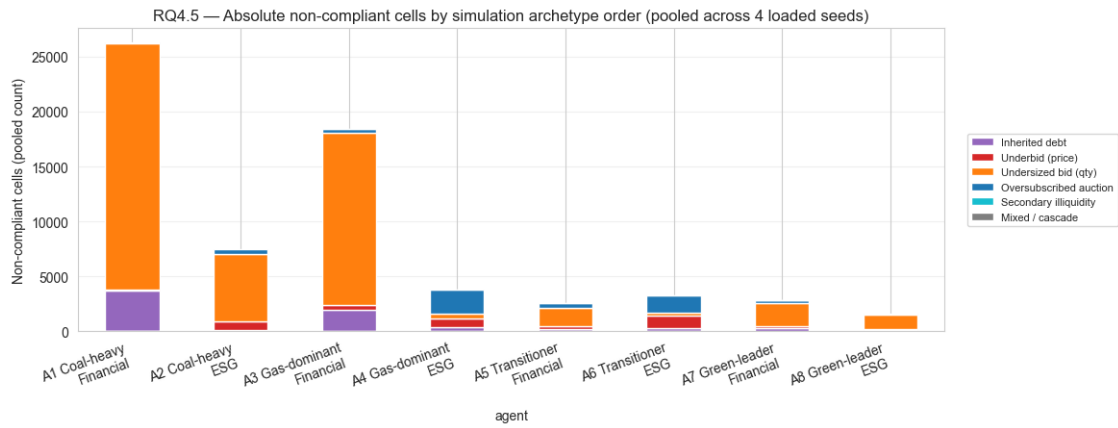
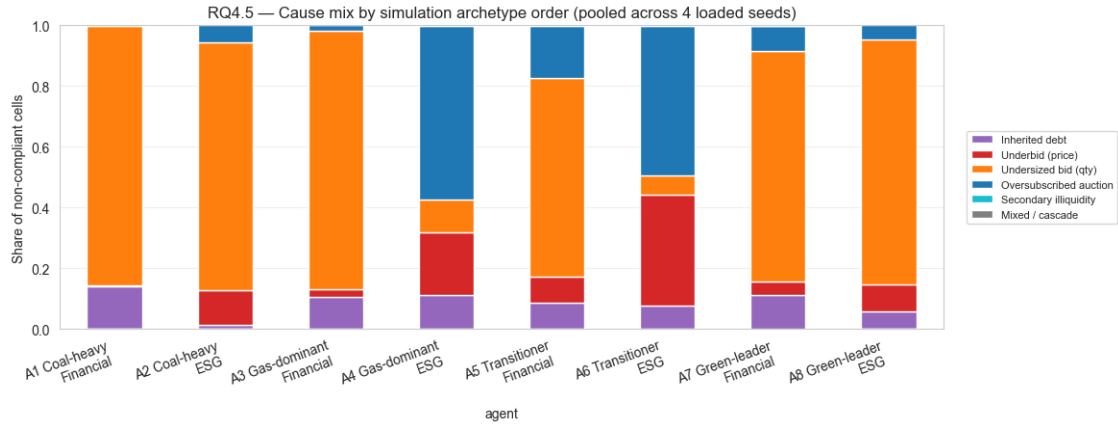
			bid_qty_mult_std	bid_coverage_ratio_mean	↳	
↳bid_coverage_ratio_median	bid_coverage_ratio_std	estimate_need_Mt_mean	↳			
↳estimate_need_Mt_median	estimate_need_Mt_std	\				
diagnostic_group						

All tail cells (normal/default)	0.681	1.346	␣
↪ 1.403	0.771	1.971	␣
↪ 1.742	1.139		
Fully compliant cells	0.652	1.476	␣
↪ 1.657	0.752	1.737	␣
↪ 1.549	0.943		
Inherited debt	0.424	0.713	␣
↪ 0.553	0.416	4.179	␣
↪ 4.365	1.375		
Underbid (price)	0.465	1.781	␣
↪ 1.935	0.554	2.247	␣
↪ 1.835	1.557		
Undersized bid (qty)	0.109	0.517	␣
↪ 0.501	0.081	3.100	␣
↪ 3.085	1.182		
Oversubscribed auction	0.286	1.885	␣
↪ 1.933	0.363	2.190	␣
↪ 1.957	1.127		
Secondary illiquidity	0.084	1.011	␣
↪ 1.019	0.133	1.985	␣
↪ 2.036	0.836		
Mixed / cascade	0.530	1.106	␣
↪ 1.141	0.166	1.301	␣
↪ 1.309	0.657		

	need_realised_Mt_mean	need_realised_Mt_median	␣
↪ need_realised_Mt_std	undersized_qty_Mt_mean	undersized_qty_Mt_median	␣
↪ undersized_qty_Mt_std	\		
diagnostic_group			
All tail cells (normal/default)	1.942	1.726	␣
↪ 1.129	0.509	0.000	␣
↪ 0.745			
Fully compliant cells	1.708	1.540	␣
↪ 0.936	0.338	0.000	␣
↪ 0.599			
Inherited debt	3.889	4.000	␣
↪ 1.276	1.539	1.733	␣
↪ 0.853			
Underbid (price)	2.200	1.818	␣
↪ 1.526	0.081	0.000	␣
↪ 0.323			
Undersized bid (qty)	3.109	3.089	␣
↪ 1.194	1.540	1.543	␣
↪ 0.658			
Oversubscribed auction	2.217	1.986	␣
↪ 1.141	0.017	0.000	␣
↪ 0.138			

Secondary illiquidity		2.134	2.036	□
↪	0.887	0.125	0.049	□
↪	0.191			
Mixed / cascade		1.398	1.327	□
↪	0.522	0.030	0.000	□
↪	0.074			
		oversubscription_ratio_mean		□
	oversubscription_ratio_median	oversubscription_ratio_std	bank_start_Mt_mean	□
	bank_start_Mt_median	bank_start_Mt_std		\
diagnostic_group				
All tail cells (normal/default)		1.275		□
↪	1.247	0.332	2.017	1.
↪595	1.991			
Fully compliant cells		1.260		□
↪	1.234	0.314	2.372	1.
↪956	2.000			
Inherited debt		1.543		□
↪	1.538	0.346	0.000	0.
↪000	0.000			
Underbid (price)		1.938		□
↪	1.746	0.646	0.624	0.
↪317	0.713			
Undersized bid (qty)		1.258		□
↪	1.234	0.329	0.303	0.
↪000	0.498			
Oversubscribed auction		1.474		□
↪	1.423	0.305	0.552	0.
↪226	0.667			
Secondary illiquidity		1.036		□
↪	1.036	0.008	0.000	0.
↪000	0.000			
Mixed / cascade		1.033		□
↪	1.037	0.014	0.000	0.
↪000	0.000			
		sec_buy_intent_Mt_mean		□
	sec_buy_intent_Mt_median	sec_buy_intent_Mt_std	secondary_buy_unmet_Mt_mean	□
	secondary_buy_unmet_Mt_median	secondary_buy_unmet_Mt_std		\
diagnostic_group				
All tail cells (normal/default)		1.669	3.	
↪000	1.474	1.301		□
↪	0.0	1.449		
Fully compliant cells		1.444	0.	
↪071	1.482	1.021		□
↪	0.0	1.384		

Inherited debt			2.768	3.
↳000	0.786		2.663	↳
↳ 3.0		0.864		
Underbid (price)			2.407	3.
↳000	1.176		2.223	↳
↳ 3.0		1.276		
Undersized bid (qty)			2.815	3.
↳000	0.706		2.714	↳
↳ 3.0		0.797		
Oversubscribed auction			2.406	3.
↳000	1.175		2.312	↳
↳ 3.0		1.233		
Secondary illiquidity			3.000	3.
↳000	0.000		3.000	↳
↳ 3.0		0.000		
Mixed / cascade			0.000	0.
↳000	0.000		0.000	↳
↳ 0.0		0.000		
			loan_outstanding_Meur_mean	↳
↳loan_outstanding_Meur_median	loan_outstanding_Meur_std			
diagnostic_group				
All tail cells (normal/default)			0.0	↳
↳ 0.0		0.0		
Fully compliant cells			0.0	↳
↳ 0.0		0.0		
Inherited debt			0.0	↳
↳ 0.0		0.0		
Underbid (price)			0.0	↳
↳ 0.0		0.0		
Undersized bid (qty)			0.0	↳
↳ 0.0		0.0		
Oversubscribed auction			0.0	↳
↳ 0.0		0.0		
Secondary illiquidity			0.0	↳
↳ 0.0		0.0		
Mixed / cascade			0.0	↳
↳ 0.0		0.0		



[RQ4.5] Compliant vs non-compliant cell means and medians (key discriminators):

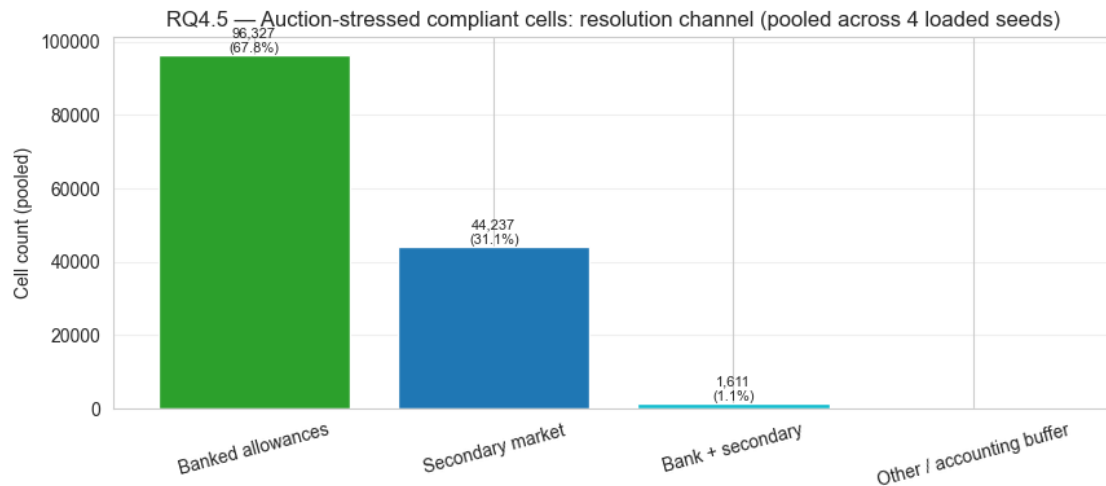
	feature	compliant_mean	non_compliant_mean	delta_mean_(nc - c)	
	↪ compliant_median		non_compliant_median	delta_median_(nc - c)	
0	inherited_debt_Mt	0.060	0.739	0.678	↪
↪	0.000	0.173	0.173		
1	bank_start_Mt	2.372	0.309	-2.063	↪
↪	1.956	0.000	-1.956		
2	bid_qty_mult	1.418	0.719	-0.700	↪
↪	1.814	0.500	-1.314		
3	bid_to_clearing	1.789	2.281	0.492	↪
↪	1.545	2.271	0.726		
4	coverage_gap_Mt	0.481	2.104	1.623	↪
↪	0.000	2.107	2.107		

[RQ4.5] Auction-stressed but ultimately compliant cells: 142,175

Stressed = auction allocation < emissions + carry-forward start; final shortfall = 0.

	n_cells	share_pct	mean_initial_gap_Mt	median_initial_gap_Mt	sum_initial_gap_Mt	mean_bank_start_Mt	mean_bank_used_estimate_Mt	mean_secondary_buy_Mt	\
solution									
Banked allowances	96327	67.752	0.890						
↪	0.840	85738.173	3.020				0.870		
↪		0.020							
Secondary market	44237	31.114	1.650						
↪	1.627	72984.968	0.937				0.000		
↪		2.837							
Bank + secondary	1611	1.133	1.932						
↪	1.722	3113.214	1.228				0.478		
↪		1.455							
Other / accounting buffer	0	0.000	0.000						
↪	0.000	0.000	0.000				0.000		
↪		0.000							

	sum_secondary_buy_Mt	mean_secondary_paid_Meur	sum_secondary_paid_Meur	mean_secondary_paid_price_EUR_t
solution				
Banked allowances	1948.997	1.849		
↪	1.780747e+05	89.952		
Secondary market	125519.232	410.794		
↪	1.817231e+07	142.228		
Bank + secondary	2343.612	215.366		
↪	3.469541e+05	127.979		
Other / accounting buffer	0.000	0.000		
↪	0.000000e+00	0.000		



[RQ4.5] Writing cell output PDF to:

C:\Users\danie\Documents\Python_Scripts\Master Thesis\Thesis-Energy-Auction\results\analysis\rq45_cell_output.pdf
[RQ4.5] PDF export complete: C:\Users\danie\Documents\Python_Scripts\Master Thesis\Thesis-Energy-Auction\results\analysis\rq45_cell_output.pdf

RQ4 summary. RQ4.1–4.3 give the run-level mechanics: did the agents close the coverage gap, did the auction stay diffuse, and did the secondary market actually clear the residual? RQ4.4 attributes the residual debt and emergency-loan usage to the (archetype $\times$ weighting) cells — the financial vs ESG split is the most informative cut here, since coal-heavy financial agents are the ones for whom the gap-penalty reward (driven by `effective_penalty_rate`) actively dominates the bid head.

1.9 Year-over-Year Dynamics in the Converged Window — Cross-Sweep Comparison

All preceding sections pool every loaded run into a single default-config population. This section instead splits by **sweep** and overlays every loaded sweep on the same axes, so cross-sweep differences become visible at a glance. The x-axis is always the simulation year (1..N_YEARS) and every series is restricted to the converged tail window (PELT and plateau diagnostics confirm convergence well within the final 10 000 episodes; the default tail of 200 episodes is enough to capture the steady-state shape, but the knob can be raised below).

Each panel shows a labelled cross-seed median line (within a sweep) plus a labelled shaded min/max fan, so the spread is bounded by the observed seed extremes within that sweep. The companion tables list the mean alongside the median for each year, while the vertical separation between coloured medians reflects the cross-sweep effect.

```
[68]: # Cross-sweep year-over-year helpers.
#
# YOY_TAIL_EPISODES bounds how many of the final episodes per run feed
# the year-axis aggregations below. 200 is the working default; raising
# it adds samples but does not change the steady-state shape because the
# runs are already converged at that point (PELT/plateau diagnostics).
import itertools
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

YOY_TAIL_EPISODES = 1000
YOY_SWEEP_FILTER = ['default_seeds']

# Stable sweep ordering and colour map so every panel below uses the
# same colour for a given sweep.
AVAILABLE_SWEEP_NAMES = sorted({k[0] for k in runs.keys()})
if YOY_SWEEP_FILTER:
```

```

SWEEP_NAMES = [sw for sw in AVAILABLE_SWEEP_NAMES if sw in YOY_SWEEP_FILTER]
if not SWEEP_NAMES:
    raise ValueError(
        f'YOY_SWEEP_FILTER={YOY_SWEEP_FILTER} matched no loaded sweeps. '
        f'Available: {AVAILABLE_SWEEP_NAMES}'
    )
else:
    SWEEP_NAMES = AVAILABLE_SWEEP_NAMES
    _palette = plt.get_cmap('tab10').colors
    SWEEP_COLORS = {sw: _palette[i % len(_palette)] for i, sw in
        enumerate(SWEEP_NAMES)}
    print(f'YoY converged-window section - tail = {YOY_TAIL_EPISODES} episodes per_
        run')
    print(f'Available sweeps: {AVAILABLE_SWEEP_NAMES}')
    print(f'Sweeps overlaid: {SWEEP_NAMES}')
    for sw in SWEEP_NAMES:
        n_runs = sum(1 for k in runs if k[0] == sw)
        print(f' {sw}: {n_runs} run(s)')

def _format_yoy_year_axis(ax):
    if 'format_year_axis' in globals():
        format_year_axis(ax)

def _reference_reserve_price():
    ets_cfg = REFERENCE_CONFIG.get('ets', {}) or {}
    return float(ets_cfg.get('reserve_price', ets_cfg.
        get('auction_reserve_price', np.nan)))

def _penalty_rate_by_years(years):
    penalty_cfg = REFERENCE_CONFIG.get('penalty', {}) or {}
    ets_cfg = REFERENCE_CONFIG.get('ets', {}) or {}
    base = penalty_cfg.get('rate', ets_cfg.get('penalty_rate', np.nan))
    inflation = penalty_cfg.get('inflation_rate', penalty_cfg.get('inflation',
        0.0))
    years = pd.Index(years, name='year')
    if len(years) == 0 or pd.isna(base):
        return pd.Series(dtype=float, name='penalty_rate')
    year_origin = YEAR_DISPLAY_START if 'YEAR_DISPLAY_START' in globals() else 1
    year_offset = np.asarray(years, dtype=float) - float(year_origin)
    return pd.Series(
        float(base) * np.power(1.0 + float(inflation), np.clip(year_offset, 0.
            0, None)),
        index=years,
        name='penalty_rate',

```

```

)

def _years_from_sweep_data(sweep_data):
    years = []
    for df in (sweep_data or {}).values():
        if df is not None and not df.empty:
            years.extend([y for y in df.index if pd.notna(y)])
    return pd.Index(sorted(set(years)), name='year')

def _yoy_year_rows(run, n=YOY_TAIL_EPISODES):
    """Return year_log rows whose episode is in the last `n` episodes."""
    ep = run['ep_df']
    yr = run['yr_df']
    if ep.empty or yr.empty or 'episode' not in yr.columns or 'episode' not in ep.columns:
        return yr.head(0)
    last_eps = ep['episode'].tail(min(n, len(ep))).unique()
    return yr[yr['episode'].isin(last_eps)]

def _per_sweep_year_stack(metric_fn):
    """For each sweep, build a wide DataFrame indexed by year with one
    column per (variant, seed). `metric_fn(yr_rows)` must return a
    pd.Series indexed by year (or None to skip the run)."""
    out = {}
    for sw in SWEEP_NAMES:
        cols = {}
        for k, run in runs.items():
            if k[0] != sw:
                continue
            yr = _yoy_year_rows(run)
            if yr.empty or 'year' not in yr.columns:
                continue
            s = metric_fn(yr)
            if s is None or len(s) == 0:
                continue
            cols[(k[1], k[2])] = s
        if cols:
            out[sw] = pd.concat(cols, axis=1).sort_index()
    return out

def _overlay_yoy(ax, sweep_data, ylabel, title, *, show_band=True, marker='o'):
    """Overlay one median line plus a labelled min/max fan per sweep on `ax`."""
    if not sweep_data:

```



```

        ax.text(0.5, 0.5, 'no data', ha='center', va='center', transform=ax.
↳transAxes)
        ax.set_title(title); ax.set_ylabel(ylabel); ax.set_xlabel('Year')
        _format_yoy_year_axis(ax)
        return
    for sw in SWEEP_NAMES:
        df = sweep_data.get(sw)
        if df is None or df.empty:
            continue
        color = SWEEP_COLORS[sw]
        med = df.median(axis=1)
        lo = df.min(axis=1)
        hi = df.max(axis=1)
        n = df.shape[1]
        if show_band and n > 1:
            ax.fill_between(med.index, lo.values, hi.values,
                            color=color, alpha=0.15,
                            label=f'{sw} min/max fan')
            ax.plot(med.index, med.values, marker=marker, lw=2.0, color=color,
                    label=f'{sw} median line (n={n})')
        ax.set_title(title); ax.set_xlabel('Year'); ax.set_ylabel(ylabel)
        ax.grid(alpha=0.3)
        ax.legend(fontsize=8, loc='best')
        _format_yoy_year_axis(ax)

def _overlay_yoy_reference(ax, sweep_data, label, *, color='black',
↳linestyle='--', lw=1.6, alpha=0.75):
    """Overlay a dashed median reference line from the same per-sweep stack
↳format."""
    if not sweep_data:
        return
    for sw in SWEEP_NAMES:
        df = sweep_data.get(sw)
        if df is None or df.empty:
            continue
        med = df.median(axis=1)
        line_color = SWEEP_COLORS[sw] if color is None else color
        line_label = f'{sw} {label}' if len(SWEEP_NAMES) > 1 else label
        ax.plot(med.index, med.values, linestyle=linestyle, lw=lw,
                color=line_color, alpha=alpha, label=line_label)

def _agent_cols_in(yr_df, prefix, max_idx=N_TOTAL):
    """Return existing `{prefix}_A{i}` columns in yr_df, sorted by i."""
    return [f'{prefix}_A{i+1}' for i in range(max_idx)
            if f'{prefix}_A{i+1}' in yr_df.columns]

```

```

def _system_sum_by_year(yr_df, prefix, max_idx=N_TOTAL):
    """Per-year mean across episodes of the cross-agent SUM of {prefix}_A*."""
    cols = _agent_cols_in(yr_df, prefix, max_idx)
    if not cols:
        return None
    return yr_df.groupby('year')[cols].mean().sum(axis=1)

def _agent_mean_by_year(yr_df, prefix, max_idx=N_TOTAL):
    """Per-year mean across episodes of the cross-agent MEAN of {prefix}_A*."""
    cols = _agent_cols_in(yr_df, prefix, max_idx)
    if not cols:
        return None
    return yr_df.groupby('year')[cols].mean().mean(axis=1)

def _scalar_mean_by_year(yr_df, col):
    if col not in yr_df.columns:
        return None
    return yr_df.groupby('year')[col].mean()

def _scalar_std_by_year(yr_df, col):
    """Cross-episode std of `col` per year - the within-run volatility."""
    if col not in yr_df.columns:
        return None
    return yr_df.groupby('year')[col].std()

def _yoy_panel_tables(panels, *, units=None):
    """Build companion data tables for a 2x2 YoY figure.

    Parameters
    -----
    panels : list of (title, sweep_data)
        `sweep_data` is the dict returned by `_per_sweep_year_stack`,
        i.e. `{sweep_name: wide_df indexed by year, columns = (variant,
        seed)}`.
    units : optional dict {title: unit_str}, embedded in the printout.

    Returns
    -----
    (per_year_df, summary_df)
        `per_year_df` - one row per (panel, sweep, year) with the
        mean and median across seeds, the cross-seed std, the min/max

```

```

    fan bounds, and n.
    `summary_df` - one row per (panel, sweep) summarising the
    per-year median series across years (mean / median / std / min /
    max), i.e. the scalar headline numbers for the panel.
    """
    py_rows, ag_rows = [], []
    for title, sd in panels:
        unit = (units or {}).get(title, '')
        if not sd:
            ag_rows.append({'panel': title, 'unit': unit, 'sweep': '(no data)',
                           'n_seeds': 0, 'years': 0,
                           'mean': np.nan, 'median': np.nan,
                           'std': np.nan, 'min': np.nan, 'max': np.nan})
            continue
        for sw, df in sd.items():
            mean = df.mean(axis=1)
            med = df.median(axis=1)
            std = df.std(axis=1)
            mn = df.min(axis=1)
            mx = df.max(axis=1)
            n = df.shape[1]
            for y in med.index:
                py_rows.append({
                    'panel': title, 'unit': unit, 'sweep': sw,
                    'year': int(y),
                    'mean_across_seeds': mean.loc[y],
                    'median_across_seeds': med.loc[y],
                    'std_across_seeds': std.loc[y],
                    'min_across_seeds': mn.loc[y],
                    'max_across_seeds': mx.loc[y],
                    'n_seeds': n,
                })
            ag_rows.append({
                'panel': title, 'unit': unit, 'sweep': sw,
                'n_seeds': n, 'years': int(len(med)),
                'mean': med.mean(), 'median': med.median(),
                'std': med.std(), 'min': med.min(), 'max': med.max(),
            })
    py_df = pd.DataFrame(py_rows)
    ag_df = pd.DataFrame(ag_rows)
    return py_df, ag_df

def _show_yoy_tables(panels, *, units=None, float_fmt='{:, .4g}'):
    """Compute and display the per-year and across-year summary tables
    for a 2x2 YoY figure. Returns the two DataFrames so calling cells
    can re-use them if needed."""

```

```

py_df, ag_df = _yoy_panel_tables(panels, units=units)
with pd.option_context('display.float_format', float_fmt.format,
                        'display.max_rows', 200):
    print(f'Across-year summary (plotted median series across_
↪{YOY_TAIL_EPISODES}-episode tail, per sweep):')
    display(ag_df)
    print('Per-year detail (median line; shaded min/max fan; mean listed_
↪for comparison):')
    display(py_df)
return py_df, ag_df

```

YoY converged-window section - tail = 1000 episodes per run
Available sweeps: ['default_seeds']
Sweeps overlaid: ['default_seeds']
default_seeds: 4 run(s)

1.9.1 Price dynamics over the simulation year

Clearing price and secondary price (level + cross-seed spread per sweep), price realism measured by the clearing/fundamental-anchor ratio (1 means the auction tracks the fundamental), and the within-run cross-episode std of the clearing price (a proxy for year-conditional price volatility within the converged window).

```

[69]: fig, axes = plt.subplots(2, 2, figsize=(14, 8))

_clearing_price_yoy = _per_sweep_year_stack(lambda yr: _scalar_mean_by_year(yr,
↪'clearing_price'))
_overlay_yoy(
    axes[0, 0],
    _clearing_price_yoy,
    ylabel='€/tonne', title='Auction clearing price')

_fundamental_anchor_yoy = _per_sweep_year_stack(lambda yr:
↪_scalar_mean_by_year(yr, 'fundamental_anchor'))
_overlay_yoy_reference(
    axes[0, 0],
    _fundamental_anchor_yoy,
    label='fundamental price anchor',
    color='black', linestyle='--', alpha=0.7)
_reserve_price = _reference_reserve_price()
if np.isfinite(_reserve_price):
    axes[0, 0].axhline(_reserve_price, color='dimgray', lw=1.2,
                        linestyle=':', alpha=0.8,
                        label=f'reserve price ({_reserve_price:g} €/t)')
axes[0, 0].legend(fontsize=8, loc='best')

```

```

_secondary_price_yoy = _per_sweep_year_stack(lambda yr:
    ↪ _scalar_mean_by_year(yr, 'secondary_price'))
_overlay_yoy(
    axes[0, 1],
    _secondary_price_yoy,
    ylabel='€/tonne', title='Secondary-market price')
_penalty_rate =
    ↪ _penalty_rate_by_years(_years_from_sweep_data(_secondary_price_yoy))
if not _penalty_rate.empty:
    axes[0, 1].plot(_penalty_rate.index, _penalty_rate.values,
                    color='red', lw=1.6, linestyle='--', alpha=0.75,
                    label='penalty rate')
axes[0, 1].legend(fontsize=8, loc='best')

def _clearing_over_anchor(yr):
    if 'clearing_price' not in yr.columns or 'fundamental_anchor' not in yr.
    ↪ columns:
        return None
    g = yr.groupby('year')[['clearing_price', 'fundamental_anchor']].mean()
    anchor = g['fundamental_anchor'].replace(0, np.nan)
    return (g['clearing_price'] / anchor)

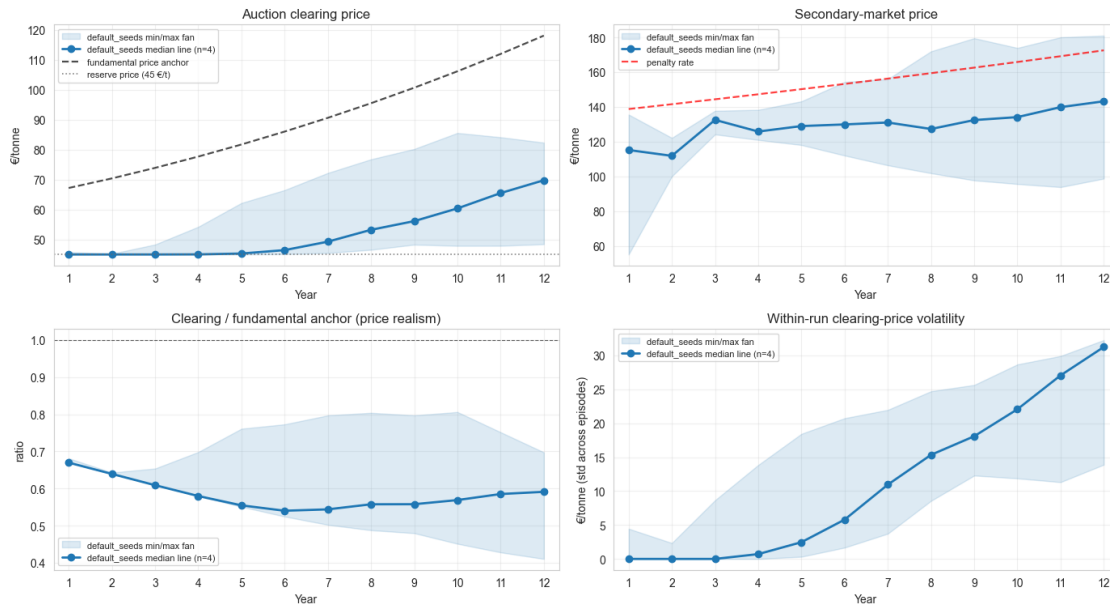
_overlay_yoy(
    axes[1, 0],
    _per_sweep_year_stack(_clearing_over_anchor),
    ylabel='ratio', title='Clearing / fundamental anchor (price realism)')
axes[1, 0].axhline(1.0, color='black', lw=0.8, linestyle='--', alpha=0.6)

_overlay_yoy(
    axes[1, 1],
    _per_sweep_year_stack(lambda yr: _scalar_std_by_year(yr, 'clearing_price')),
    ylabel='€/tonne (std across episodes)',
    title='Within-run clearing-price volatility')

plt.suptitle(f'Price dynamics by simulation year (last {YOY_TAIL_EPISODES} eps
    ↪ per run)',
             fontsize=13, y=1.00)
plt.tight_layout(); plt.show()

```

Price dynamics by simulation year (last 1000 eps per run)



[70]: *# Companion data tables for the price-dynamics figure above.
Each row of the per-year table corresponds to one plotted point;
the across-year summary collapses the median series into mean/std/
min/max so the thesis text can quote both shape and headline numbers.*

```
def _clearing_over_anchor_tbl(yr):
    if 'clearing_price' not in yr.columns or 'fundamental_anchor' not in yr.
    ↪columns:
        return None
    g = yr.groupby('year')[['clearing_price', 'fundamental_anchor']].mean()
    anchor = g['fundamental_anchor'].replace(0, np.nan)
    return g['clearing_price'] / anchor

_panels_price = [
    ('Auction clearing price',
     _per_sweep_year_stack(lambda yr: _scalar_mean_by_year(yr,
     ↪'clearing_price'))),
    ('Secondary-market price',
     _per_sweep_year_stack(lambda yr: _scalar_mean_by_year(yr,
     ↪'secondary_price'))),
    ('Clearing / fundamental anchor (price realism)',
     _per_sweep_year_stack(_clearing_over_anchor_tbl)),
    ('Within-run clearing-price volatility',
     _per_sweep_year_stack(lambda yr: _scalar_std_by_year(yr,
     ↪'clearing_price'))),
]
```

```

_units_price = {
    'Auction clearing price': '€/tonne',
    'Secondary-market price': '€/tonne',
    'Clearing / fundamental anchor (price realism)': 'ratio',
    'Within-run clearing-price volatility': '€/tonne (std across episodes)',
}
_price_yoy_tables = _show_yoy_tables(_panels_price, units=_units_price)

```

Across-year summary (plotted median series across 1000-episode tail, per sweep):

					panel				unit
	sweep	n_seeds	years	mean	median	std	min	max	
0					Auction clearing price				€/tonne
	↪ default_seeds	4	12	52.17	47.86	8.868	45	69.78	
1					Secondary-market price				€/tonne
	↪ default_seeds	4	12	129.3	130.4	8.906	111.8	143.2	
2					Clearing / fundamental anchor (price realism)				ratio
	↪ default_seeds	4	12	0.5829	0.5742	0.03946	0.54	0.6696	
3					Within-run clearing-price volatility				€/tonne (std across episodes)
	↪ default_seeds	4	12	11.15	8.376	11.4	0	31.25	

Per-year detail (median line; shaded min/max fan; mean listed for comparison):

					panel		unit
	sweep	year	mean_across_seeds	median_across_seeds	std_across_seeds		
	↪ min_across_seeds		max_across_seeds	\			
0					Auction clearing price		€/tonne
	↪ default_seeds	1		45.2		45	0.3955
	↪		45	45.79			
1					Auction clearing price		€/tonne
	↪ default_seeds	2		45.06		45	0.1272
	↪		45	45.25			
2					Auction clearing price		€/tonne
	↪ default_seeds	3		45.83		45	1.659
	↪		45	48.32			
3					Auction clearing price		€/tonne
	↪ default_seeds	4		47.32		45.03	4.608
	↪		45	54.24			
4					Auction clearing price		€/tonne
	↪ default_seeds	5		49.46		45.31	8.492
	↪		45.02	62.19			
5					Auction clearing price		€/tonne
	↪ default_seeds	6		51.13		46.45	10.3
	↪		45.11	66.49			
6					Auction clearing price		€/tonne
	↪ default_seeds	7		54.07		49.27	12.42
	↪		45.5	72.22			

7		Auction clearing price		€/tonne
↪ default_seeds	8	57.45	53.23	13.79
↪	46.56	76.77		
8		Auction clearing price		€/tonne
↪ default_seeds	9	60.19	56.13	15.24
↪	48.27	80.23		
9		Auction clearing price		€/tonne
↪ default_seeds	10	63.55	60.38	17.65
↪	47.87	85.57		
10		Auction clearing price		€/tonne
↪ default_seeds	11	65.75	65.5	17.51
↪	47.88	84.14		
11		Auction clearing price		€/tonne
↪ default_seeds	12	67.58	69.78	16.98
↪	48.42	82.35		
12		Secondary-market price		€/tonne
↪ default_seeds	1	105.2	115.2	35.98
↪	55.08	135.6		
13		Secondary-market price		€/tonne
↪ default_seeds	2	111.4	111.8	9.363
↪	100	122.2		
14		Secondary-market price		€/tonne
↪ default_seeds	3	131.7	132.5	6.687
↪	124.1	137.7		
15		Secondary-market price		€/tonne
↪ default_seeds	4	127.7	125.8	8.336
↪	121	138.3		
16		Secondary-market price		€/tonne
↪ default_seeds	5	129.8	129	10.32
↪	118	143.2		
17		Secondary-market price		€/tonne
↪ default_seeds	6	131.5	129.9	18.46
↪	112	154.3		
18		Secondary-market price		€/tonne
↪ default_seeds	7	131.1	131	22.07
↪	106.4	155.9		
19		Secondary-market price		€/tonne
↪ default_seeds	8	132.1	127.3	31.09
↪	101.8	171.9		
20		Secondary-market price		€/tonne
↪ default_seeds	9	135.5	132.4	36.38
↪	97.7	179.5		
21		Secondary-market price		€/tonne
↪ default_seeds	10	134.4	134.1	36.48
↪	95.62	173.9		

22		Secondary-market price		€/tonne
↪	default_seeds	11	138.4	139.8
↪		93.83	180	38.62
23		Secondary-market price		€/tonne
↪	default_seeds	12	141.5	143.2
↪		98.72	181	36.74
24	Clearing / fundamental anchor (price realism)			ratio
↪	default_seeds	1	0.6726	0.6696
↪		0.6696	0.6814	0.005885
25	Clearing / fundamental anchor (price realism)			ratio
↪	default_seeds	2	0.6401	0.6392
↪		0.6392	0.6429	0.001807
26	Clearing / fundamental anchor (price realism)			ratio
↪	default_seeds	3	0.6201	0.6088
↪		0.6088	0.6537	0.02245
27	Clearing / fundamental anchor (price realism)			ratio
↪	default_seeds	4	0.6092	0.5797
↪		0.5793	0.6981	0.05931
28	Clearing / fundamental anchor (price realism)			ratio
↪	default_seeds	5	0.6052	0.5545
↪		0.5508	0.761	0.1039
29	Clearing / fundamental anchor (price realism)			ratio
↪	default_seeds	6	0.5943	0.54
↪		0.5243	0.7729	0.1197
30	Clearing / fundamental anchor (price realism)			ratio
↪	default_seeds	7	0.5966	0.5437
↪		0.5021	0.7969	0.1371
31	Clearing / fundamental anchor (price realism)			ratio
↪	default_seeds	8	0.6016	0.5574
↪		0.4875	0.8039	0.1444
32	Clearing / fundamental anchor (price realism)			ratio
↪	default_seeds	9	0.5979	0.5576
↪		0.4795	0.7969	0.1514
33	Clearing / fundamental anchor (price realism)			ratio
↪	default_seeds	10	0.5986	0.5688
↪		0.451	0.8061	0.1663
34	Clearing / fundamental anchor (price realism)			ratio
↪	default_seeds	11	0.5874	0.585
↪		0.4277	0.7516	0.1564
35	Clearing / fundamental anchor (price realism)			ratio
↪	default_seeds	12	0.5724	0.591
↪		0.4101	0.6974	0.1438
36	Within-run clearing-price volatility		€/tonne (std across episodes)	
↪	default_seeds	1	1.114	0
↪		0	4.455	2.227

37	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	2	0.5861	0 1.172
↪	0	2.344	
38	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	3	2.164	0 4.327
↪	0	8.655	
39	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	4	3.818	0.7098 6.698
↪	0	13.85	
40	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	5	5.924	2.467 8.458
↪	0.3167	18.44	
41	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	6	8.503	5.796 8.662
↪	1.667	20.75	
42	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	7	11.9	10.96 7.942
↪	3.721	21.98	
43	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	8	16	15.35 7.174
↪	8.557	24.73	
44	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	9	18.53	18.09 6.605
↪	12.29	25.65	
45	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	10	21.18	22.08 7.264
↪	11.87	28.69	
46	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	11	23.85	27.07 8.513
↪	11.32	29.93	
47	Within-run clearing-price volatility	€/tonne (std across episodes)	
↪ default_seeds	12	27.17	31.25 8.895
↪	13.89	32.28	

	n_seeds
0	4
1	4
2	4
3	4
4	4
5	4
6	4
7	4
8	4
9	4
10	4
11	4

12	4
13	4
14	4
15	4
16	4
17	4
18	4
19	4
20	4
21	4
22	4
23	4
24	4
25	4
26	4
27	4
28	4
29	4
30	4
31	4
32	4
33	4
34	4
35	4
36	4
37	4
38	4
39	4
40	4
41	4
42	4
43	4
44	4
45	4
46	4
47	4

1.9.2 Compliance and penalty pressure over the simulation year

System-aggregate coverage gap (Mt remaining after the auction), end-of-year carry-forward (debt cascade indicator), compliance rate (share of (agent, year) cells with non-negative compliance surplus), and the system-aggregate penalty paid per year.

```
[71]: fig, axes = plt.subplots(2, 2, figsize=(14, 8))

      _overlay_yoy(
          axes[0, 0],
          _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'coverage_gap')),
```

```

        ylabel='Mt', title='System coverage gap (pre-secondary)')

_overlay_yoy(
    axes[0, 1],
    _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr,
↳ 'carry_forward_end')),
    ylabel='Mt', title='End-of-year carry-forward (system)')

def _compliance_rate_by_year(yr):
    sf_cols = _agent_cols_in(yr, 'shortfall')
    if sf_cols:
        long = yr.melt(id_vars=['year'], value_vars=sf_cols,
↳ value_name='shortfall')
        long['compliant'] = (long['shortfall'] <= 1e-6).astype(float)
        return long.groupby('year')['compliant'].mean()

    surplus_cols = _agent_cols_in(yr, 'compliance_surplus')
    if not surplus_cols:
        return None
    long = yr.melt(id_vars=['year'], value_vars=surplus_cols,
↳ value_name='surplus')
    long['compliant'] = (long['surplus'].fillna(0) >= -1e-6).astype(float)
    return long.groupby('year')['compliant'].mean()

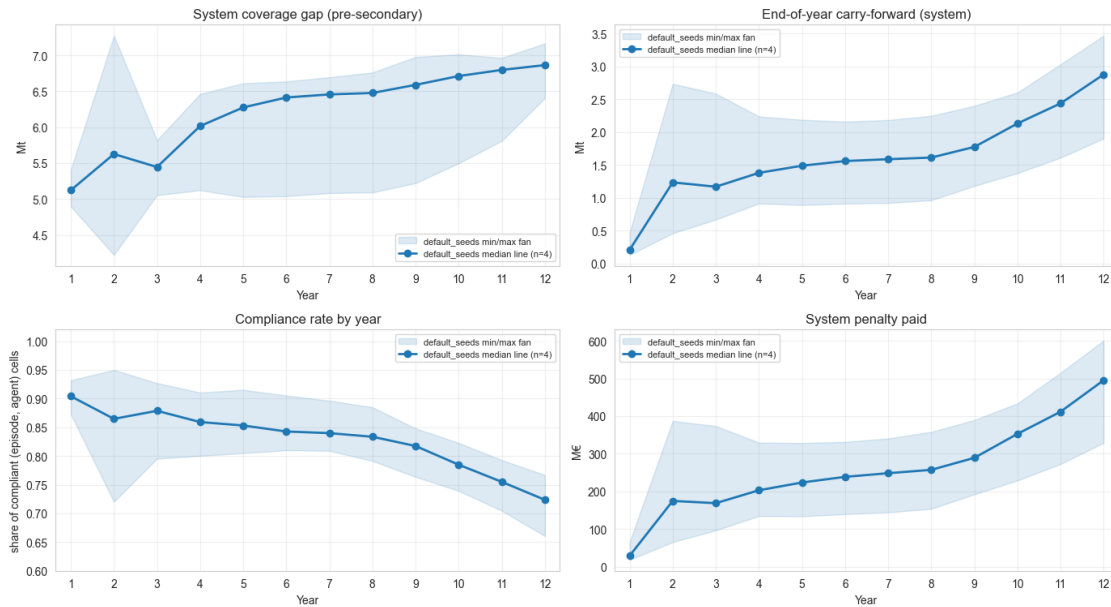
_overlay_yoy(
    axes[1, 0],
    _per_sweep_year_stack(_compliance_rate_by_year),
    ylabel='share of compliant (episode, agent) cells',
    title='Compliance rate by year')
axes[1, 0].set_ylim(0.6, 1.02)

_overlay_yoy(
    axes[1, 1],
    _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'penalty')),
    ylabel='M€', title='System penalty paid')

plt.suptitle(f'Compliance pressure by simulation year (last {YOY_TAIL_EPISODES}
↳ eps per run)',
            fontsize=13, y=1.00)
plt.tight_layout(); plt.show()

```

Compliance pressure by simulation year (last 1000 eps per run)



```
[72]: # Companion data tables for the compliance / penalty-pressure figure.
def _compliance_rate_tbl(yr):
    sf_cols = _agent_cols_in(yr, 'shortfall')
    if sf_cols:
        long = yr.melt(id_vars=['year'], value_vars=sf_cols,
            value_name='shortfall')
        long['compliant'] = (long['shortfall'] <= 1e-6).astype(float)
        return long.groupby('year')['compliant'].mean()

    surplus_cols = _agent_cols_in(yr, 'compliance_surplus')
    if not surplus_cols:
        return None
    long = yr.melt(id_vars=['year'], value_vars=surplus_cols,
        value_name='surplus')
    long['compliant'] = (long['surplus'].fillna(0) >= -1e-6).astype(float)
    return long.groupby('year')['compliant'].mean()

_panels_compliance = [
    ('System coverage gap (pre-secondary)',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'coverage_gap'))),
    ('End-of-year carry-forward (system)',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr,
        'carry_forward_end'))),
    ('Compliance rate by year',
     _per_sweep_year_stack(_compliance_rate_tbl)),
    ('System penalty paid',
```

```

    _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'penalty'))),
]
_units_compliance = {
    'System coverage gap (pre-secondary)': 'Mt',
    'End-of-year carry-forward (system)': 'Mt',
    'Compliance rate by year': 'share of compliant (episode, agent) cells',
    'System penalty paid': 'M€',
}
_compliance_yoy_tables = _show_yoy_tables(_panels_compliance,
    ↪units=_units_compliance)

```

Across-year summary (plotted median series across 1000-episode tail, per sweep):

			panel						
↪unit	sweep	n_seeds	years	mean	median	std	min	max	↪
0	System coverage gap (pre-secondary)								↪
↪Mt	default_seeds	4	12	6.233	6.435	0.5627	5.123	6.864	
1	End-of-year carry-forward (system)								↪
↪Mt	default_seeds	4	12	1.621	1.573	0.6715	0.2078	2.877	
2	Compliance rate by year	share of compliant (episode, agent)							↪
↪cells	default_seeds	4	12	0.8297	0.8413	0.05209	0.7237	0.9041	
3	System penalty paid								↪
↪M€	default_seeds	4	12	257.6	243.5	121.4	28.83	495.3	

Per-year detail (median line; shaded min/max fan; mean listed for comparison):

			panel						
↪unit	sweep	year	mean_across_seeds	median_across_seeds	↪				↪
↪std_across_seeds	min_across_seeds	max_across_seeds	\						
0	System coverage gap (pre-secondary)								↪
↪Mt	default_seeds	1	5.138		5.123		0.		
↪2108	4.896		5.41						
1	System coverage gap (pre-secondary)								↪
↪Mt	default_seeds	2	5.687		5.626		1.		
↪262	4.222		7.273						
2	System coverage gap (pre-secondary)								↪
↪Mt	default_seeds	3	5.441		5.445		0.		
↪4141	5.053		5.819						
3	System coverage gap (pre-secondary)								↪
↪Mt	default_seeds	4	5.904		6.016		0.		
↪5657	5.122		6.462						
4	System coverage gap (pre-secondary)								↪
↪Mt	default_seeds	5	6.047		6.275		0.		
↪6991	5.028		6.612						
5	System coverage gap (pre-secondary)								↪
↪Mt	default_seeds	6	6.125		6.413		0.		
↪7338	5.039		6.636						

6	System coverage gap (pre-secondary)				▮
↳Mt	default_seeds	7	6.172	6.456	0.
↳7477		5.083	6.695		
7	System coverage gap (pre-secondary)				▮
↳Mt	default_seeds	8	6.201	6.476	0.
↳756		5.092	6.761		
8	System coverage gap (pre-secondary)				▮
↳Mt	default_seeds	9	6.344	6.589	0.
↳7726		5.221	6.977		
9	System coverage gap (pre-secondary)				▮
↳Mt	default_seeds	10	6.483	6.711	0.
↳6731		5.497	7.013		
10	System coverage gap (pre-secondary)				▮
↳Mt	default_seeds	11	6.592	6.797	0.
↳5285		5.809	6.964		
11	System coverage gap (pre-secondary)				▮
↳Mt	default_seeds	12	6.824	6.864	0.
↳3149		6.403	7.164		
12	End-of-year carry-forward (system)				▮
↳Mt	default_seeds	1	0.2539	0.2078	0.
↳1495		0.1301	0.4701		
13	End-of-year carry-forward (system)				▮
↳Mt	default_seeds	2	1.416	1.235	0.
↳9586		0.4599	2.735		
14	End-of-year carry-forward (system)				▮
↳Mt	default_seeds	3	1.398	1.168	0.
↳8351		0.6681	2.587		
15	End-of-year carry-forward (system)				▮
↳Mt	default_seeds	4	1.478	1.38	0.
↳5969		0.9134	2.238		
16	End-of-year carry-forward (system)				▮
↳Mt	default_seeds	5	1.513	1.488	0.
↳6206		0.8896	2.186		
17	End-of-year carry-forward (system)				▮
↳Mt	default_seeds	6	1.547	1.559	0.
↳6265		0.9103	2.16		
18	End-of-year carry-forward (system)				▮
↳Mt	default_seeds	7	1.57	1.587	0.
↳6128		0.9211	2.183		
19	End-of-year carry-forward (system)				▮
↳Mt	default_seeds	8	1.609	1.612	0.
↳5948		0.9624	2.249		
20	End-of-year carry-forward (system)				▮
↳Mt	default_seeds	9	1.782	1.774	0.
↳5848		1.18	2.4		

21	End-of-year carry-forward (system)				□
↳Mt	default_seeds	10	2.06	2.131	0.
↳6216		1.374	2.603		
22	End-of-year carry-forward (system)				□
↳Mt	default_seeds	11	2.379	2.437	0.
↳6475		1.61	3.032		
23	End-of-year carry-forward (system)				□
↳Mt	default_seeds	12	2.781	2.877	0.
↳6705		1.902	3.467		
24	Compliance rate by year share of compliant (episode, agent)				□
↳cells	default_seeds	1	0.9032	0.9041	0.
↳02768		0.8726	0.932		
25	Compliance rate by year share of compliant (episode, agent)				□
↳cells	default_seeds	2	0.8498	0.8647	0.
↳09569		0.7201	0.9497		
26	Compliance rate by year share of compliant (episode, agent)				□
↳cells	default_seeds	3	0.8699	0.8787	0.
↳05489		0.7953	0.927		
27	Compliance rate by year share of compliant (episode, agent)				□
↳cells	default_seeds	4	0.8573	0.8591	0.
↳04703		0.8003	0.9105		
28	Compliance rate by year share of compliant (episode, agent)				□
↳cells	default_seeds	5	0.8566	0.8531	0.
↳04834		0.8049	0.9153		
29	Compliance rate by year share of compliant (episode, agent)				□
↳cells	default_seeds	6	0.8502	0.8427	0.
↳04419		0.81	0.9052		
30	Compliance rate by year share of compliant (episode, agent)				□
↳cells	default_seeds	7	0.8462	0.8398	0.
↳04033		0.8089	0.8964		
31	Compliance rate by year share of compliant (episode, agent)				□
↳cells	default_seeds	8	0.8358	0.8335	0.
↳04137		0.7914	0.8849		
32	Compliance rate by year share of compliant (episode, agent)				□
↳cells	default_seeds	9	0.8116	0.8174	0.
↳04013		0.7635	0.8481		
33	Compliance rate by year share of compliant (episode, agent)				□
↳cells	default_seeds	10	0.7829	0.7849	0.
↳04332		0.7392	0.8227		
34	Compliance rate by year share of compliant (episode, agent)				□
↳cells	default_seeds	11	0.7519	0.7549	□
↳0.0403		0.7046	0.793		
35	Compliance rate by year share of compliant (episode, agent)				□
↳cells	default_seeds	12	0.7187	0.7237	0.
↳04532		0.6604	0.7669		

36		System penalty paid			
↵M€	default_seeds	1	35.23	28.83	20.
↵74		18.05	65.23		
37		System penalty paid			
↵M€	default_seeds	2	200.4	174.6	
↵135.7		65.14	387.3		
38		System penalty paid			
↵M€	default_seeds	3	201.8	168.6	
↵120.7		96.39	373.7		
39		System penalty paid			
↵M€	default_seeds	4	217.5	202.9	87.
↵95		134.4	329.7		
40		System penalty paid			
↵M€	default_seeds	5	227.3	223.6	93.
↵12		133.7	328.4		
41		System penalty paid			
↵M€	default_seeds	6	237	238.6	95.
↵94		139.5	331.5		
42		System penalty paid			
↵M€	default_seeds	7	245.4	248.3	95.
↵63		144.1	340.7		
43		System penalty paid			
↵M€	default_seeds	8	256.6	257.2	
↵94.7		153.7	358.1		
44		System penalty paid			
↵M€	default_seeds	9	290	289.2	95.
↵03		192.1	389.5		
45		System penalty paid			
↵M€	default_seeds	10	342	352.8	
↵103.1		228.8	433.7		
46		System penalty paid			
↵M€	default_seeds	11	402.8	411.7	
↵109.9		272.8	515.1		
47		System penalty paid			
↵M€	default_seeds	12	480	495.3	
↵116		328.9	600.6		

	n_seeds
0	4
1	4
2	4
3	4
4	4
5	4
6	4
7	4
8	4

9	4
10	4
11	4
12	4
13	4
14	4
15	4
16	4
17	4
18	4
19	4
20	4
21	4
22	4
23	4
24	4
25	4
26	4
27	4
28	4
29	4
30	4
31	4
32	4
33	4
34	4
35	4
36	4
37	4
38	4
39	4
40	4
41	4
42	4
43	4
44	4
45	4
46	4
47	4

1.9.3 Decarbonisation and investment over the simulation year

Cross-agent mean green generation share (does the fleet green up over the year axis?), system emissions (Mt), system invest cost (M€), and MAC fuel-switching reductions (Mt abated). All four panels read from per-agent year_log columns aggregated across the converged tail.

```
[73]: fig, axes = plt.subplots(2, 2, figsize=(14, 8))
```

```

_overlay_yoy(
    axes[0, 0],
    _per_sweep_year_stack(lambda yr: _agent_mean_by_year(yr, 'green_frac')),
    ylabel='green frac in [0, 1]', title='Mean green generation share')
axes[0, 0].set_ylim(0, 1)

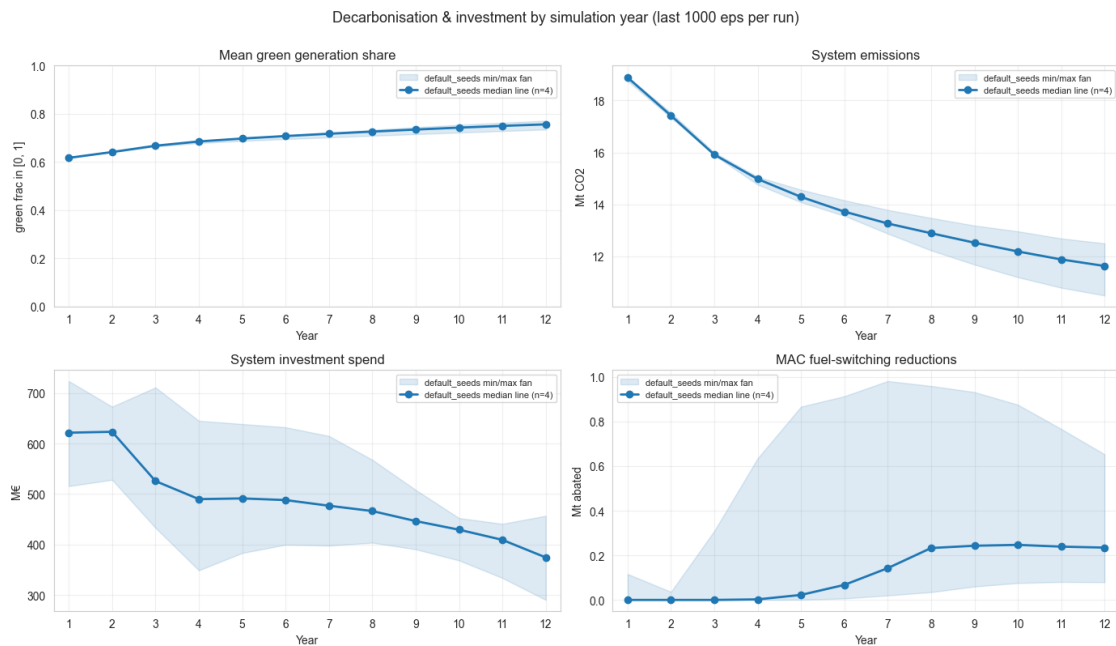
_overlay_yoy(
    axes[0, 1],
    _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'emissions')),
    ylabel='Mt CO2', title='System emissions')

_overlay_yoy(
    axes[1, 0],
    _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'invest_cost')),
    ylabel='M€', title='System investment spend')

_overlay_yoy(
    axes[1, 1],
    _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'mac_reduction')),
    ylabel='Mt abated', title='MAC fuel-switching reductions')

plt.suptitle(f'Decarbonisation & investment by simulation year (last_
↳{YOY_TAIL_EPISODES} eps per run)',
            fontsize=13, y=1.00)
plt.tight_layout(); plt.show()

```



```
[74]: # Companion data tables for the decarbonisation / investment figure.
_panels_decarb = [
    ('Mean green generation share',
     _per_sweep_year_stack(lambda yr: _agent_mean_by_year(yr, 'green_frac'))),
    ('System emissions',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'emissions'))),
    ('System investment spend',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'invest_cost'))),
    ('MAC fuel-switching reductions',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr,
     ↪ 'mac_reduction'))),
]
_units_decarb = {
    'Mean green generation share': 'green frac [0, 1]',
    'System emissions': 'Mt CO ',
    'System investment spend': 'M€',
    'MAC fuel-switching reductions': 'Mt abated',
}
_decarb_yoy_tables = _show_yoy_tables(_panels_decarb, units=_units_decarb)
```

Across-year summary (plotted median series across 1000-episode tail, per sweep):

			panel		unit	sweep	n_seeds	
↪ years	mean	median	std	min	max			
0	Mean green generation share	green frac	[0, 1]	default_seeds		4		
↪ 12	0.7036	0.7125	0.04388	0.6171	0.7563			
1	System emissions	Mt CO	default_seeds		4			
↪ 12	14.13	13.49	2.283	11.63	18.88			
2	System investment spend	M€	default_seeds		4			
↪ 12	486.9	482.5	75.19	374.7	623.1			
3	MAC fuel-switching reductions	Mt abated	default_seeds		4			
↪ 12	0.1192	0.1049	0.1132	0	0.2468			

Per-year detail (median line; shaded min/max fan; mean listed for comparison):

		panel		unit	sweep	year	
↪ mean_across_seeds	median_across_seeds	std_across_seeds	min_across_seeds				
↪ max_across_seeds	n_seeds						
0	Mean green generation share	green frac	[0, 1]	default_seeds	1		
↪	0.617	0.6171	0.0002988	0.6165			
↪	0.6171	4					
1	Mean green generation share	green frac	[0, 1]	default_seeds	2		
↪	0.6413	0.6413	0.001056	0.64			
↪	0.6424	4					
2	Mean green generation share	green frac	[0, 1]	default_seeds	3		
↪	0.667	0.6675	0.002309	0.6639			
↪	0.6692	4					

3	Mean green generation share	green frac	[0, 1]	default_seeds	4	
↪	0.684	0.6853	0.00381	0.6784		
↪	0.6871	4				
4	Mean green generation share	green frac	[0, 1]	default_seeds	5	
↪	0.6955	0.6974	0.005549	0.6874		
↪	0.6998	4				
5	Mean green generation share	green frac	[0, 1]	default_seeds	6	
↪	0.7053	0.7077	0.007364	0.695		
↪	0.7107	4				
6	Mean green generation share	green frac	[0, 1]	default_seeds	7	
↪	0.7147	0.7172	0.009378	0.702		
↪	0.7222	4				
7	Mean green generation share	green frac	[0, 1]	default_seeds	8	
↪	0.7237	0.7262	0.0114	0.7089		
↪	0.7335	4				
8	Mean green generation share	green frac	[0, 1]	default_seeds	9	
↪	0.7324	0.7348	0.01321	0.7156		
↪	0.7443	4				
9	Mean green generation share	green frac	[0, 1]	default_seeds	10	
↪	0.7404	0.7427	0.01476	0.7221		
↪	0.7542	4				
10	Mean green generation share	green frac	[0, 1]	default_seeds	11	
↪	0.7478	0.7499	0.01587	0.7284		
↪	0.7632	4				
11	Mean green generation share	green frac	[0, 1]	default_seeds	12	
↪	0.7546	0.7563	0.01664	0.7347		
↪	0.7712	4				
12	System emissions		Mt CO	default_seeds	1	
↪	18.86	18.88	0.07374	18.75		
↪	18.91	4				
13	System emissions		Mt CO	default_seeds	2	
↪	17.43	17.42	0.08008	17.34		
↪	17.53	4				
14	System emissions		Mt CO	default_seeds	3	
↪	15.93	15.91	0.04706	15.89		
↪	16	4				
15	System emissions		Mt CO	default_seeds	4	
↪	14.94	14.98	0.128	14.76		
↪	15.06	4				
16	System emissions		Mt CO	default_seeds	5	
↪	14.31	14.29	0.2033	14.08		
↪	14.56	4				
17	System emissions		Mt CO	default_seeds	6	
↪	13.79	13.72	0.2654	13.56		
↪	14.16	4				

18		System emissions	Mt CO	default_seeds	7	□
↪	13.3	13.27	0.3764	12.88		□
↪	13.79	4				
19		System emissions	Mt CO	default_seeds	8	□
↪	12.87	12.89	0.5115	12.23		□
↪	13.48	4				
20		System emissions	Mt CO	default_seeds	9	□
↪	12.48	12.53	0.626	11.69		□
↪	13.19	4				
21		System emissions	Mt CO	default_seeds	10	□
↪	12.14	12.19	0.7425	11.2		□
↪	12.96	4				
22		System emissions	Mt CO	default_seeds	11	□
↪	11.81	11.88	0.8154	10.8		□
↪	12.69	4				
23		System emissions	Mt CO	default_seeds	12	□
↪	11.57	11.63	0.8746	10.5		□
↪	12.5	4				
24		System investment spend	M€	default_seeds	1	□
↪	620.4	621.3	88.52	515.6		□
↪	723.4	4				
25		System investment spend	M€	default_seeds	2	□
↪	611.8	623.1	61.95	528.2		□
↪	672.8	4				
26		System investment spend	M€	default_seeds	3	□
↪	548.8	525.7	125.2	433.1		□
↪	710.8	4				
27		System investment spend	M€	default_seeds	4	□
↪	493.5	490	139.4	349.2		□
↪	644.9	4				
28		System investment spend	M€	default_seeds	5	□
↪	501.2	491.4	129.7	383.5		□
↪	638.5	4				
29		System investment spend	M€	default_seeds	6	□
↪	502	488.1	119.7	399.8		□
↪	632	4				
30		System investment spend	M€	default_seeds	7	□
↪	491.7	476.9	109.3	398		□
↪	614.8	4				
31		System investment spend	M€	default_seeds	8	□
↪	476.1	466.4	82.86	403.8		□
↪	567.8	4				
32		System investment spend	M€	default_seeds	9	□
↪	448.1	446.6	58.19	390.8		□
↪	508.3	4				

33	System investment spend	M€	default_seeds	10	□
↪	420.1	429.5	37.65	368.7	□
↪	452.6	4			
34	System investment spend	M€	default_seeds	11	□
↪	398.5	409.4	45.49	334.1	□
↪	441.1	4			
35	System investment spend	M€	default_seeds	12	□
↪	374.3	374.7	68.47	290.7	□
↪	457	4			
36	MAC fuel-switching reductions	Mt abated	default_seeds	1	□
↪	0.02904	0	0.05807	0	□
↪	0.1161	4			
37	MAC fuel-switching reductions	Mt abated	default_seeds	2	□
↪	0.009059	0	0.01812	0	□
↪	0.03624	4			
38	MAC fuel-switching reductions	Mt abated	default_seeds	3	□
↪	0.07737	0	0.1547	0	□
↪	0.3095	4			
39	MAC fuel-switching reductions	Mt abated	default_seeds	4	□
↪	0.16	0.002459	0.3167	0	□
↪	0.635	4			
40	MAC fuel-switching reductions	Mt abated	default_seeds	5	□
↪	0.2282	0.02244	0.4259	0.001155	□
↪	0.8667	4			
41	MAC fuel-switching reductions	Mt abated	default_seeds	6	□
↪	0.2636	0.06738	0.4352	0.006317	□
↪	0.9133	4			
42	MAC fuel-switching reductions	Mt abated	default_seeds	7	□
↪	0.3216	0.1425	0.4483	0.01998	□
↪	0.9814	4			
43	MAC fuel-switching reductions	Mt abated	default_seeds	8	□
↪	0.3649	0.2327	0.416	0.03489	□
↪	0.9593	4			
44	MAC fuel-switching reductions	Mt abated	default_seeds	9	□
↪	0.3695	0.243	0.4009	0.06004	□
↪	0.932	4			
45	MAC fuel-switching reductions	Mt abated	default_seeds	10	□
↪	0.3612	0.2468	0.3772	0.07569	□
↪	0.8754	4			
46	MAC fuel-switching reductions	Mt abated	default_seeds	11	□
↪	0.3314	0.239	0.3242	0.08065	□
↪	0.7667	4			
47	MAC fuel-switching reductions	Mt abated	default_seeds	12	□
↪	0.3005	0.2346	0.2675	0.07955	□
↪	0.6531	4			

1.9.4 Secondary-market activity over the simulation year

Total secondary volume per year (sum of net buys across agents — by construction equal to the sum of net sells), executed buyer / seller counts vs the corresponding intent counts (a wide gap means agents post uncrossable orders), and the training-consistent secondary match rate (share of episode-years with any executed trade). If the required match-rate fields are unavailable in a run, the panel falls back to the secondary-vs-clearing price spread.

```
[75]: fig, axes = plt.subplots(2, 2, figsize=(14, 8))

def _secondary_volume(yr):
    cols = _agent_cols_in(yr, 'secondary_net')
    if not cols:
        # Fallback: signed trade_qty across agents.
        cols = _agent_cols_in(yr, 'trade_qty')
    if not cols:
        return None
    # Mean per (year, agent) across episodes, then sum positives across agents.
    by_year = yr.groupby('year')[cols].mean()
    return by_year.clip(lower=0).sum(axis=1)

_overlay_yoy(
    axes[0, 0],
    _per_sweep_year_stack(_secondary_volume),
    ylabel='Mt traded', title='Total secondary volume')

def _intent_exec_gap_buyers(yr):
    if not {'secondary_n_buyers_intent', 'secondary_n_buyers_executed'}.
    <isubset(yr.columns):
        return None
    g = yr.groupby('year')[['secondary_n_buyers_intent',
    <'secondary_n_buyers_executed']].mean()
    return g['secondary_n_buyers_intent'] - g['secondary_n_buyers_executed']

def _intent_exec_gap_sellers(yr):
    if not {'secondary_n_sellers_intent', 'secondary_n_sellers_executed'}.
    <isubset(yr.columns):
        return None
    g = yr.groupby('year')[['secondary_n_sellers_intent',
    <'secondary_n_sellers_executed']].mean()
    return g['secondary_n_sellers_intent'] - g['secondary_n_sellers_executed']

_overlay_yoy(
    axes[0, 1],
    _per_sweep_year_stack(_intent_exec_gap_buyers),
    ylabel='agents (intent - executed)',
    title='Buy-side intent-execution gap')
```



```

_overlay_yoy(
    axes[1, 0],
    _per_sweep_year_stack(_intent_exec_gap_sellers),
    ylabel='agents (intent - executed)',
    title='Sell-side intent-execution gap')

def _training_match_rate_by_year(yr):
    # Match the training metric definition: a year is matched if any trade
    ↪ executed.
    cols = {'secondary_n_buyers_executed', 'secondary_n_sellers_executed'}
    if cols.issubset(yr.columns):
        any_trade = (yr['secondary_n_buyers_executed'].fillna(0)
                     + yr['secondary_n_sellers_executed'].fillna(0)) > 0
        return any_trade.groupby(yr['year']).mean()
    trade_cols = _agent_cols_in(yr, 'trade_qty')
    if not trade_cols:
        return None
    any_trade = yr[trade_cols].fillna(0).abs().gt(1e-6).any(axis=1)
    return any_trade.groupby(yr['year']).mean()

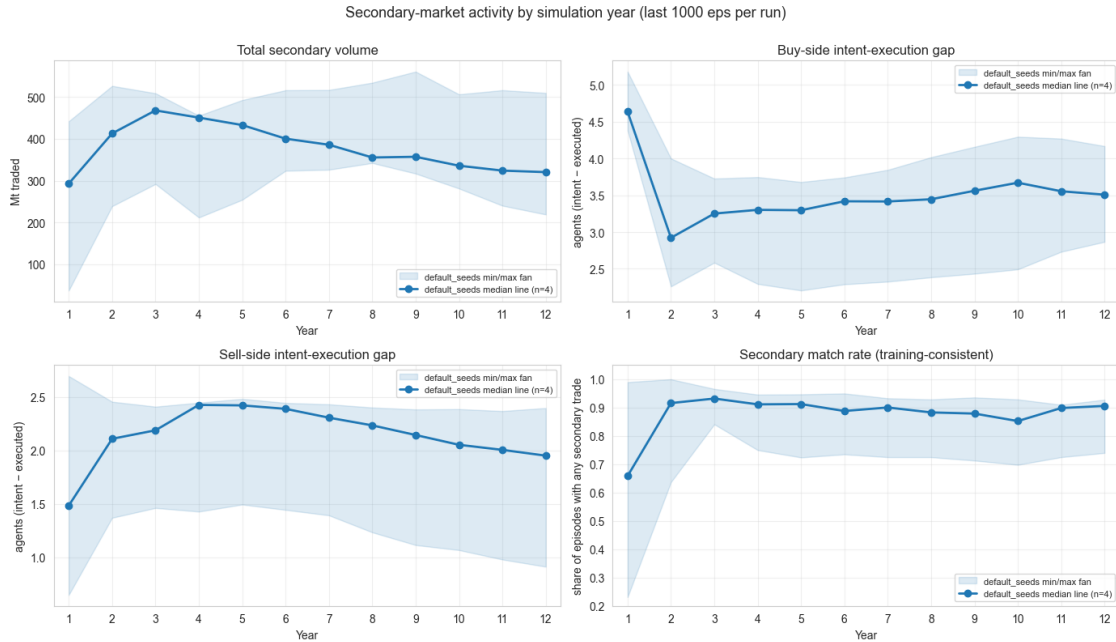
def _sec_minus_clearing(yr):
    if not {'secondary_price', 'clearing_price'}.issubset(yr.columns):
        return None
    g = yr.groupby('year')[['secondary_price', 'clearing_price']].mean()
    return g['secondary_price'] - g['clearing_price']

_match_rate_yoy = _per_sweep_year_stack(_training_match_rate_by_year)
if _match_rate_yoy:
    _secondary_panel_data = _match_rate_yoy
    _secondary_panel_ylabel = 'share of episodes with any secondary trade'
    _secondary_panel_title = 'Secondary match rate (training-consistent)'
else:
    _secondary_panel_data = _per_sweep_year_stack(_sec_minus_clearing)
    _secondary_panel_ylabel = '€/tonne'
    _secondary_panel_title = 'Secondary - clearing price spread'

_overlay_yoy(
    axes[1, 1],
    _secondary_panel_data,
    ylabel=_secondary_panel_ylabel,
    title=_secondary_panel_title)
if _match_rate_yoy:
    axes[1, 1].set_ylim(0.2, 1.05)
else:
    axes[1, 1].axhline(0, color='black', lw=0.8, linestyle='--', alpha=0.6)

```

```
plt.suptitle(f'Secondary-market activity by simulation year (last_{
    ↳{YOY_TAIL_EPISODES} eps per run)',
            fontsize=13, y=1.00)
plt.tight_layout(); plt.show()
```



```
[76]: # Companion data tables for the secondary-market activity figure.
def _secondary_volume_tbl(yr):
    cols = _agent_cols_in(yr, 'secondary_net')
    if not cols:
        cols = _agent_cols_in(yr, 'trade_qty')
        if not cols:
            return None
    by_year = yr.groupby('year')[cols].mean()
    return by_year.clip(lower=0).sum(axis=1)

def _intent_exec_gap_buyers_tbl(yr):
    if not {'secondary_n_buyers_intent', 'secondary_n_buyers_executed'}.
    ↳issubset(yr.columns):
        return None
    g = yr.groupby('year')[['secondary_n_buyers_intent',
    ↳'secondary_n_buyers_executed']].mean()
    return g['secondary_n_buyers_intent'] - g['secondary_n_buyers_executed']

def _intent_exec_gap_sellers_tbl(yr):
    if not {'secondary_n_sellers_intent', 'secondary_n_sellers_executed'}.
    ↳issubset(yr.columns):
```

```

        return None
    g = yr.groupby('year')[['secondary_n_sellers_intent',
↪ 'secondary_n_sellers_executed']].mean()
    return g['secondary_n_sellers_intent'] - g['secondary_n_sellers_executed']

def _training_match_rate_by_year_tbl(yr):
    cols = {'secondary_n_buyers_executed', 'secondary_n_sellers_executed'}
    if cols.issubset(yr.columns):
        any_trade = (yr['secondary_n_buyers_executed'].fillna(0)
                     + yr['secondary_n_sellers_executed'].fillna(0)) > 0
        return any_trade.groupby(yr['year']).mean()
    trade_cols = _agent_cols_in(yr, 'trade_qty')
    if not trade_cols:
        return None
    any_trade = yr[trade_cols].fillna(0).abs().gt(1e-6).any(axis=1)
    return any_trade.groupby(yr['year']).mean()

def _sec_minus_clearing_tbl(yr):
    if not {'secondary_price', 'clearing_price'}.issubset(yr.columns):
        return None
    g = yr.groupby('year')[['secondary_price', 'clearing_price']].mean()
    return g['secondary_price'] - g['clearing_price']

_match_rate_yoy_tbl = _per_sweep_year_stack(_training_match_rate_by_year_tbl)
if _match_rate_yoy_tbl:
    _secondary_panel_label = 'Secondary match rate (training-consistent)'
    _secondary_panel_unit = 'share of episodes with any secondary trade'
    _secondary_panel_stack = _match_rate_yoy_tbl
else:
    _secondary_panel_label = 'Secondary - clearing price spread'
    _secondary_panel_unit = '€/tonne'
    _secondary_panel_stack = _per_sweep_year_stack(_sec_minus_clearing_tbl)

_panels_secondary = [
    ('Total secondary volume',
     _per_sweep_year_stack(_secondary_volume_tbl)),
    ('Buy-side intent-execution gap',
     _per_sweep_year_stack(_intent_exec_gap_buyers_tbl)),
    ('Sell-side intent-execution gap',
     _per_sweep_year_stack(_intent_exec_gap_sellers_tbl)),
    (_secondary_panel_label,
     _secondary_panel_stack),
]
_units_secondary = {
    'Total secondary volume': 'Mt traded',
    'Buy-side intent-execution gap': 'agents (intent - executed)',
    'Sell-side intent-execution gap': 'agents (intent - executed)',
}

```

```

    _secondary_panel_label: _secondary_panel_unit,
}
_secondary_yoy_tables = _show_yoy_tables(_panels_secondary,
↳units=_units_secondary)

```

Across-year summary (plotted median series across 1000-episode tail, per sweep):

				panel					
↳	unit	sweep	n_seeds	years	mean	median	std	min	max
0	Total secondary volume								
↳	Mt traded	default_seeds	4	12	378.1	371.5	55.99	293	468.2
1	Buy-side intent-execution gap agents (intent -								
↳	executed)	default_seeds	4	12	3.498	3.432	0.4066	2.922	4.636
2	Sell-side intent-execution gap agents (intent -								
↳	executed)	default_seeds	4	12	2.142	2.166	0.2618	1.484	2.425
3	Secondary match rate (training-consistent) share of episodes with any								
↳	secondary trade	default_seeds	4	12	0.8782	0.8995	0.0722	0.6585	0.932

Per-year detail (median line; shaded min/max fan; mean listed for comparison):

				panel				
↳	unit	sweep	year	mean_across_seeds	median_across_seeds	↳		
↳	std_across_seeds	min_across_seeds	\					
0	Total secondary volume							
↳	Mt traded	default_seeds	1	266.3		293		
↳	171.7	36.96						
1	Total secondary volume							
↳	Mt traded	default_seeds	2	397.9		413		
↳	140.6	238.8						
2	Total secondary volume							
↳	Mt traded	default_seeds	3	434.4		468.2		
↳	99.44	291.9						
3	Total secondary volume							
↳	Mt traded	default_seeds	4	392.3		451		
↳	120.4	211.7						
4	Total secondary volume							
↳	Mt traded	default_seeds	5	403.4		433.2		
↳	103.4	254.1						
5	Total secondary volume							
↳	Mt traded	default_seeds	6	410.3		400.5		
↳	79.64	323.6						
6	Total secondary volume							
↳	Mt traded	default_seeds	7	403.8		385.9		
↳	81.71	326.2						
7	Total secondary volume							
↳	Mt traded	default_seeds	8	396.9		355.5		
↳	92.53	342.2						

8	Total secondary volume					
↪Mt traded	default_seeds	9	398.1	357.1	↪	
↪	111.7	316.9				
9	Total secondary volume				↪	
↪Mt traded	default_seeds	10	364.8	335.8	↪	
↪	105.2	281.1				
10	Total secondary volume				↪	
↪Mt traded	default_seeds	11	351.2	324	↪	
↪	125.9	240				
11	Total secondary volume				↪	
↪Mt traded	default_seeds	12	342.5	320.4	↪	
↪	127.2	219				
12	Buy-side intent-execution gap			agents (intent	↪	
↪executed)	default_seeds	1	4.708	4.636	↪	
↪	0.3679	4.38				
13	Buy-side intent-execution gap			agents (intent	↪	
↪executed)	default_seeds	2	3.026	2.922	↪	
↪	0.7471	2.261				
14	Buy-side intent-execution gap			agents (intent	↪	
↪executed)	default_seeds	3	3.203	3.251	↪	
↪	0.5113	2.584				
15	Buy-side intent-execution gap			agents (intent	↪	
↪executed)	default_seeds	4	3.161	3.302	↪	
↪	0.6835	2.295				
16	Buy-side intent-execution gap			agents (intent	↪	
↪executed)	default_seeds	5	3.12	3.296	↪	
↪	0.6501	2.206				
17	Buy-side intent-execution gap			agents (intent	↪	
↪executed)	default_seeds	6	3.217	3.418	↪	
↪	0.6789	2.29				
18	Buy-side intent-execution gap			agents (intent	↪	
↪executed)	default_seeds	7	3.251	3.415	↪	
↪	0.7344	2.326				
19	Buy-side intent-execution gap			agents (intent	↪	
↪executed)	default_seeds	8	3.323	3.446	↪	
↪	0.7429	2.385				
20	Buy-side intent-execution gap			agents (intent	↪	
↪executed)	default_seeds	9	3.428	3.561	↪	
↪	0.7885	2.434				
21	Buy-side intent-execution gap			agents (intent	↪	
↪executed)	default_seeds	10	3.532	3.67	↪	
↪	0.8284	2.493				
22	Buy-side intent-execution gap			agents (intent	↪	
↪executed)	default_seeds	11	3.528	3.554	↪	
↪	0.7525	2.732				

23	Buy-side intent-execution gap	agents (intent -	
↪executed)	default_seeds 12	3.513	3.508
↪	0.7357 2.867		
24	Sell-side intent-execution gap	agents (intent -	
↪executed)	default_seeds 1	1.578	1.484
↪	0.8434 0.649		
25	Sell-side intent-execution gap	agents (intent -	
↪executed)	default_seeds 2	2.01	2.108
↪	0.5232 1.37		
26	Sell-side intent-execution gap	agents (intent -	
↪executed)	default_seeds 3	2.062	2.188
↪	0.4303 1.462		
27	Sell-side intent-execution gap	agents (intent -	
↪executed)	default_seeds 4	2.181	2.425
↪	0.5025 1.428		
28	Sell-side intent-execution gap	agents (intent -	
↪executed)	default_seeds 5	2.204	2.421
↪	0.4748 1.494		
29	Sell-side intent-execution gap	agents (intent -	
↪executed)	default_seeds 6	2.166	2.389
↪	0.484 1.444		
30	Sell-side intent-execution gap	agents (intent -	
↪executed)	default_seeds 7	2.109	2.306
↪	0.4824 1.393		
31	Sell-side intent-execution gap	agents (intent -	
↪executed)	default_seeds 8	2.026	2.234
↪	0.5337 1.234		
32	Sell-side intent-execution gap	agents (intent -	
↪executed)	default_seeds 9	1.947	2.144
↪	0.5658 1.115		
33	Sell-side intent-execution gap	agents (intent -	
↪executed)	default_seeds 10	1.89	2.052
↪	0.5706 1.068		
34	Sell-side intent-execution gap	agents (intent -	
↪executed)	default_seeds 11	1.839	2.005
↪	0.6029 0.98		
35	Sell-side intent-execution gap	agents (intent -	
↪executed)	default_seeds 12	1.803	1.951
↪	0.6379 0.913		
36	Secondary match rate (training-consistent)	share of episodes with any	
↪secondary trade	default_seeds 1	0.6345	0.6585
↪	0.3324 0.231		
37	Secondary match rate (training-consistent)	share of episodes with any	
↪secondary trade	default_seeds 2	0.8672	0.916
↪	0.1609 0.637		

38	Secondary match rate (training-consistent)	share of episodes with any	┐
↪	secondary trade default_seeds 3	0.9177	0.932 ┐
↪	0.05539 0.841		
39	Secondary match rate (training-consistent)	share of episodes with any	┐
↪	secondary trade default_seeds 4	0.8798	0.9115 ┐
↪	0.09008 0.75		
40	Secondary match rate (training-consistent)	share of episodes with any	┐
↪	secondary trade default_seeds 5	0.874	0.9125 ┐
↪	0.1023 0.724		
41	Secondary match rate (training-consistent)	share of episodes with any	┐
↪	secondary trade default_seeds 6	0.8653	0.888 ┐
↪	0.09352 0.735		
42	Secondary match rate (training-consistent)	share of episodes with any	┐
↪	secondary trade default_seeds 7	0.8648	0.9005 ┐
↪	0.09457 0.725		
43	Secondary match rate (training-consistent)	share of episodes with any	┐
↪	secondary trade default_seeds 8	0.855	0.883 ┐
↪	0.09164 0.725		
44	Secondary match rate (training-consistent)	share of episodes with any	┐
↪	secondary trade default_seeds 9	0.8518	0.879 ┐
↪	0.0983 0.713		
45	Secondary match rate (training-consistent)	share of episodes with any	┐
↪	secondary trade default_seeds 10	0.833	0.8525 ┐
↪	0.09868 0.698		
46	Secondary match rate (training-consistent)	share of episodes with any	┐
↪	secondary trade default_seeds 11	0.858	0.8985 ┐
↪	0.08933 0.725		
47	Secondary match rate (training-consistent)	share of episodes with any	┐
↪	secondary trade default_seeds 12	0.87	0.906 ┐
↪	0.08851 0.74		

	max_across_seeds	n_seeds
0	442.2	4
1	526.9	4
2	509.2	4
3	455.7	4
4	493	4
5	516.5	4
6	517.2	4
7	534.6	4
8	561.1	4
9	506.7	4
10	516.6	4
11	510.1	4
12	5.18	4
13	4	4
14	3.727	4

15	3.747	4
16	3.68	4
17	3.742	4
18	3.845	4
19	4.017	4
20	4.158	4
21	4.295	4
22	4.27	4
23	4.168	4
24	2.695	4
25	2.455	4
26	2.409	4
27	2.446	4
28	2.482	4
29	2.444	4
30	2.431	4
31	2.401	4
32	2.383	4
33	2.386	4
34	2.368	4
35	2.396	4
36	0.99	4
37	1	4
38	0.966	4
39	0.946	4
40	0.947	4
41	0.95	4
42	0.933	4
43	0.929	4
44	0.936	4
45	0.929	4
46	0.91	4
47	0.928	4

1.9.5 Reward and reward-component dynamics over the simulation year

System-aggregate per-year reward, the logged `reward_base` component (financial – penalty + ESG), system penalty paid as a concrete reward cost driver, plus the cross-agent reward dispersion per year (std across the agent panel), which is a proxy for outcome equity. The per-episode `quality_score` does not have a within-episode year axis and is therefore covered in the convergence panels above; the reward-decomposition panels here are its per-year analogue.

```
[77]: fig, axes = plt.subplots(2, 2, figsize=(14, 8))

      _overlay_yoy(
          axes[0, 0],
          _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'reward')),
          ylabel='reward (units of training-time reward)',
```



```

        title='System reward')

_overlay_yoy(
    axes[0, 1],
    _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'reward_base')),
    ylabel='reward_base', title='System reward_base (financial - penalty +
↳ESG)')

_overlay_yoy(
    axes[1, 0],
    _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'penalty')),
    ylabel='M€', title='System penalty paid (reward cost driver)')

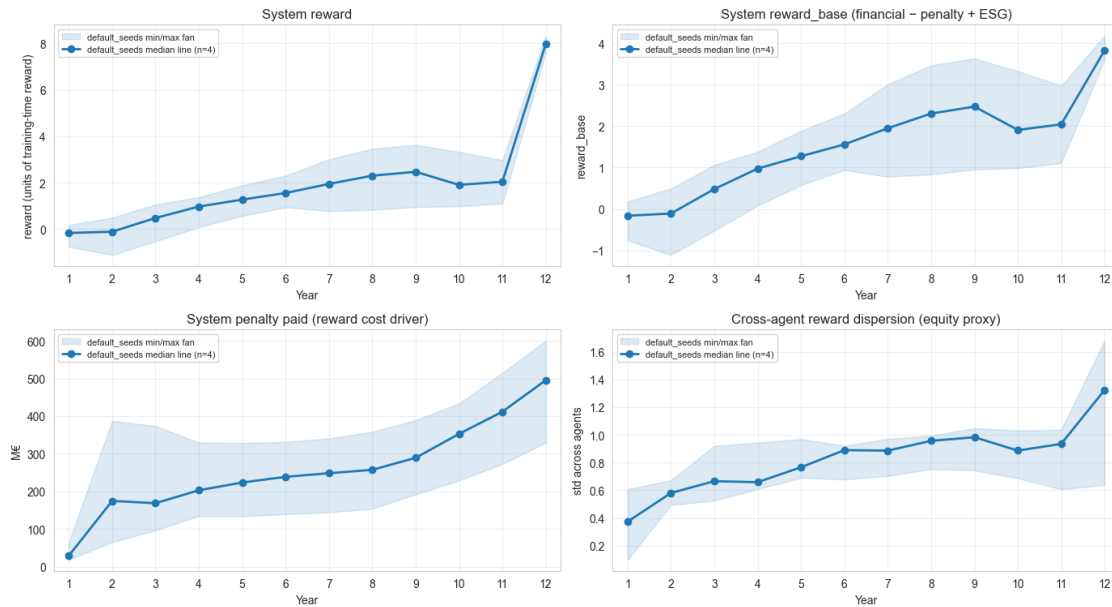
def _reward_dispersion(yr):
    cols = _agent_cols_in(yr, 'reward')
    if not cols:
        return None
    by_year = yr.groupby('year')[cols].mean()
    return by_year.std(axis=1)

_overlay_yoy(
    axes[1, 1],
    _per_sweep_year_stack(_reward_dispersion),
    ylabel='std across agents',
    title='Cross-agent reward dispersion (equity proxy)')

plt.suptitle(f'Reward and reward components by simulation year (last
↳{YOY_TAIL_EPISODES} eps per run)',
            fontsize=13, y=1.00)
plt.tight_layout(); plt.show()

```

Reward and reward components by simulation year (last 1000 eps per run)



```
[78]: # Companion data tables for the reward-dynamics figure.
def _reward_dispersion_tbl(yr):
    cols = _agent_cols_in(yr, 'reward')
    if not cols:
        return None
    by_year = yr.groupby('year')[cols].mean()
    return by_year.std(axis=1)

_panels_reward = [
    ('System reward',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'reward'))),
    ('System reward_base (financial - penalty + ESG)',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'reward_base'))),
    ('System penalty paid (reward cost driver)',
     _per_sweep_year_stack(lambda yr: _system_sum_by_year(yr, 'penalty'))),
    ('Cross-agent reward dispersion (equity proxy)',
     _per_sweep_year_stack(_reward_dispersion_tbl)),
]

_units_reward = {
    'System reward': 'reward (training-time units)',
    'System reward_base (financial - penalty + ESG)': 'reward_base',
    'System penalty paid (reward cost driver)': 'M€',
    'Cross-agent reward dispersion (equity proxy)': 'std across agents',
}

_reward_yoy_tables = _show_yoy_tables(_panels_reward, units=_units_reward)
```

Across-year summary (plotted median series across 1000-episode tail, per sweep):

				panel					unit
	sweep	n_seeds	years	mean	median	std	min	max	
0				System reward		reward (training-time units)			
	↪ default_seeds	4	12	1.888	1.733	2.111	-0.1621	7.969	
1	System reward_base (financial - penalty + ESG)								reward_base
	↪ default_seeds	4	12	1.543	1.733	1.142	-0.1621	3.825	
2	System penalty paid (reward cost driver)								M€
	↪ default_seeds	4	12	257.6	243.5	121.4	28.83	495.3	
3	Cross-agent reward dispersion (equity proxy)								std across agents
	↪ default_seeds	4	12	0.8256	0.8865	0.2401	0.3737	1.323	

Per-year detail (median line; shaded min/max fan; mean listed for comparison):

				panel			unit
	sweep	year	mean_across_seeds	median_across_seeds	std_across_seeds		
	↪ min_across_seeds					↪ max_across_seeds	\
0			System reward		reward (training-time units)		
	↪ default_seeds	1	-0.2246		-0.1621		0.4109
	↪		-0.7557	0.1814			
1			System reward		reward (training-time units)		
	↪ default_seeds	2	-0.208		-0.1089		0.7438
	↪		-1.108	0.4936			
2			System reward		reward (training-time units)		
	↪ default_seeds	3	0.3754		0.4815		0.6776
	↪		-0.5241	1.063			
3			System reward		reward (training-time units)		
	↪ default_seeds	4	0.8532		0.9753		0.5717
	↪		0.0826	1.38			
4			System reward		reward (training-time units)		
	↪ default_seeds	5	1.253		1.274		0.5556
	↪		0.5751	1.889			
5			System reward		reward (training-time units)		
	↪ default_seeds	6	1.59		1.558		0.5669
	↪		0.9405	2.304			
6			System reward		reward (training-time units)		
	↪ default_seeds	7	1.921		1.949		1.032
	↪		0.778	3.008			
7			System reward		reward (training-time units)		
	↪ default_seeds	8	2.227		2.303		1.185
	↪		0.8342	3.467			
8			System reward		reward (training-time units)		
	↪ default_seeds	9	2.382		2.47		1.223
	↪		0.9536	3.632			
9			System reward		reward (training-time units)		
	↪ default_seeds	10	2.032		1.908		1.105
	↪		0.9855	3.326			

10		System reward	reward (training-time units)	┐
↪	default_seeds	11	2.041	2.042
↪		1.107	2.974	0.8403┐
11		System reward	reward (training-time units)	┐
↪	default_seeds	12	7.965	7.969
↪		7.592	8.33	0.3057┐
12	System reward_base (financial - penalty + ESG)			reward_base┐
↪	default_seeds	1	-0.2246	-0.1621
↪		-0.7557	0.1814	0.4109┐
13	System reward_base (financial - penalty + ESG)			reward_base┐
↪	default_seeds	2	-0.208	-0.1089
↪		-1.108	0.4936	0.7438┐
14	System reward_base (financial - penalty + ESG)			reward_base┐
↪	default_seeds	3	0.3754	0.4815
↪		-0.5241	1.063	0.6776┐
15	System reward_base (financial - penalty + ESG)			reward_base┐
↪	default_seeds	4	0.8532	0.9753
↪		0.0826	1.38	0.5717┐
16	System reward_base (financial - penalty + ESG)			reward_base┐
↪	default_seeds	5	1.253	1.274
↪		0.5751	1.889	0.5556┐
17	System reward_base (financial - penalty + ESG)			reward_base┐
↪	default_seeds	6	1.59	1.558
↪		0.9405	2.304	0.5669┐
18	System reward_base (financial - penalty + ESG)			reward_base┐
↪	default_seeds	7	1.921	1.949
↪		0.778	3.008	1.032┐
19	System reward_base (financial - penalty + ESG)			reward_base┐
↪	default_seeds	8	2.227	2.303
↪		0.8342	3.467	1.185┐
20	System reward_base (financial - penalty + ESG)			reward_base┐
↪	default_seeds	9	2.382	2.47
↪		0.9536	3.632	1.223┐
21	System reward_base (financial - penalty + ESG)			reward_base┐
↪	default_seeds	10	2.032	1.908
↪		0.9855	3.326	1.105┐
22	System reward_base (financial - penalty + ESG)			reward_base┐
↪	default_seeds	11	2.041	2.042
↪		1.107	2.974	0.8403┐
23	System reward_base (financial - penalty + ESG)			reward_base┐
↪	default_seeds	12	3.859	3.825
↪		3.605	4.181	0.2551┐
24	System penalty paid (reward cost driver)			M€┐
↪	default_seeds	1	35.23	28.83
↪		18.05	65.23	20.74┐

25	System penalty paid (reward cost driver)			M€ _□
↪	default_seeds	2	200.4	174.6
↪		65.14	387.3	135.7 _□
26	System penalty paid (reward cost driver)			M€ _□
↪	default_seeds	3	201.8	168.6
↪		96.39	373.7	120.7 _□
27	System penalty paid (reward cost driver)			M€ _□
↪	default_seeds	4	217.5	202.9
↪		134.4	329.7	87.95 _□
28	System penalty paid (reward cost driver)			M€ _□
↪	default_seeds	5	227.3	223.6
↪		133.7	328.4	93.12 _□
29	System penalty paid (reward cost driver)			M€ _□
↪	default_seeds	6	237	238.6
↪		139.5	331.5	95.94 _□
30	System penalty paid (reward cost driver)			M€ _□
↪	default_seeds	7	245.4	248.3
↪		144.1	340.7	95.63 _□
31	System penalty paid (reward cost driver)			M€ _□
↪	default_seeds	8	256.6	257.2
↪		153.7	358.1	94.7 _□
32	System penalty paid (reward cost driver)			M€ _□
↪	default_seeds	9	290	289.2
↪		192.1	389.5	95.03 _□
33	System penalty paid (reward cost driver)			M€ _□
↪	default_seeds	10	342	352.8
↪		228.8	433.7	103.1 _□
34	System penalty paid (reward cost driver)			M€ _□
↪	default_seeds	11	402.8	411.7
↪		272.8	515.1	109.9 _□
35	System penalty paid (reward cost driver)			M€ _□
↪	default_seeds	12	480	495.3
↪		328.9	600.6	116 _□
36	Cross-agent reward dispersion (equity proxy)			std across agents _□
↪	default_seeds	1	0.3633	0.3737
↪		0.09824	0.6073	0.2211 _□
37	Cross-agent reward dispersion (equity proxy)			std across agents _□
↪	default_seeds	2	0.582	0.5809
↪		0.4935	0.6728	0.09337 _□
38	Cross-agent reward dispersion (equity proxy)			std across agents _□
↪	default_seeds	3	0.6946	0.6659
↪		0.5265	0.9201	0.1931 _□
39	Cross-agent reward dispersion (equity proxy)			std across agents _□
↪	default_seeds	4	0.7177	0.6589
↪		0.6082	0.9447	0.1558 _□

40	Cross-agent reward dispersion (equity proxy)		std across agents
↪	default_seeds 5	0.7981	0.7663 0.1201
↪	0.6902	0.9696	
41	Cross-agent reward dispersion (equity proxy)		std across agents
↪	default_seeds 6	0.845	0.8899 0.1122
↪	0.6787	0.9214	
42	Cross-agent reward dispersion (equity proxy)		std across agents
↪	default_seeds 7	0.8619	0.8863 0.1168
↪	0.7035	0.9716	
43	Cross-agent reward dispersion (equity proxy)		std across agents
↪	default_seeds 8	0.9154	0.9572 0.1104
↪	0.753	0.9942	
44	Cross-agent reward dispersion (equity proxy)		std across agents
↪	default_seeds 9	0.9395	0.983 0.137
↪	0.7439	1.048	
45	Cross-agent reward dispersion (equity proxy)		std across agents
↪	default_seeds 10	0.8734	0.8868 0.1504
↪	0.6872	1.033	
46	Cross-agent reward dispersion (equity proxy)		std across agents
↪	default_seeds 11	0.8787	0.9352 0.2015
↪	0.6077	1.037	
47	Cross-agent reward dispersion (equity proxy)		std across agents
↪	default_seeds 12	1.241	1.323 0.4364
↪	0.6368	1.68	

	n_seeds
0	4
1	4
2	4
3	4
4	4
5	4
6	4
7	4
8	4
9	4
10	4
11	4
12	4
13	4
14	4
15	4
16	4
17	4
18	4
19	4
20	4

21	4
22	4
23	4
24	4
25	4
26	4
27	4
28	4
29	4
30	4
31	4
32	4
33	4
34	4
35	4
36	4
37	4
38	4
39	4
40	4
41	4
42	4
43	4
44	4
45	4
46	4
47	4

Reading these panels. Each coloured solid line is one sweep’s cross-seed median, with a shaded min/max fan when the sweep contains more than one seed; both are labelled directly in the legend. The price panels also add dashed or dotted reference lines for the fundamental anchor, reserve price, and penalty rate where they help orient the scale. Vertical separation between coloured medians is the cross-sweep effect; band overlap indicates that the sweeps are indistinguishable at the loaded sample size. The companion tables list cross-seed means alongside medians for every year, so mean-sensitive outliers remain visible even though the plotted line uses the median.

The line stays as the median because the loaded sample is small and seed outliers can pull the mean into a path no actual seed followed. The min/max fan shows the full observed envelope, while the tables keep the mean available for thesis prose and checks where outlier sensitivity is itself informative. Because every series is drawn from the converged tail, year-to-year structure here reflects the *shape* of the steady-state policy, not transient learning dynamics.

1.10 What this notebook does not answer

- Cross-config sensitivity — needs sweep variants. *Out of scope here.*
- Causal attribution of regulatory mechanisms (e.g. counterfactual “what if MSR were off”) — needs sweep variants.

- Real-world EU-ETS empirical validation beyond the qualitative price-band check in RQ1.3.
- Strategy clustering, theory tests (LCOE crossover, archetype heterogeneity), and predictive models of clearing price — these are in the analysis plan (`planning/data_science_analysis_plan.md` §B1, B2, B4) and will get their own notebooks where they can be evaluated cleanly with cross-validation.

The default-config evidence base assembled above is the *baseline* that those follow-on analyses will perturb and test against.